

HiDiTingV100 UIKit

# 开发指南

|      |            |
|------|------------|
| 文档版本 | 00B03      |
| 发布日期 | 2025-07-30 |

# 前言

## 概述

本文档主要介绍 UIKit 基础架构，构建基于轻量 OS 应用框架，满足硬件资源较小的物联网设备，指导用户开发 Native 应用。

## 产品版本

与本文档相对应的产品版本如下。

| 产品名称     | 产品版本 |
|----------|------|
| HiDiTing | V100 |

## 读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

## 符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

| 符号 | 说明 |
|----|----|
|----|----|

| 符号                                                                                   | 说明                                                                    |
|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
|  危险 | 表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。                                        |
|  警告 | 表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。                                        |
|  注意 | 表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。                                        |
| 须知                                                                                   | 用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。<br>“须知”不涉及人身伤害。 |
|  说明 | 对正文中重点信息的补充说明。<br>“说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。                       |

修改记录

| 文档版本  | 发布日期       | 修改说明                                                                                                                                                     |
|-------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 00B03 | 2025-07-30 | <ul style="list-style-type: none"><li>更新 “2.3.26 UITransformGroup、2.4.2 UIImageAnimator、2.4.19 UICanvasExt” 。</li><li>新增 “8 在线表盘、9 桌面样式” 两个章节。</li></ul> |
| 00B02 | 2025-06-27 | 更新 “2.3.15 UIPickerView” 。                                                                                                                               |
| 00B01 | 2025-06-05 | 第一次临时版本发布。                                                                                                                                               |

# 目 录

|                                    |    |
|------------------------------------|----|
| 前言.....                            | i  |
| 1 概述.....                          | 1  |
| 2 组件介绍.....                        | 3  |
| 2.1 概述.....                        | 3  |
| 2.2 UIView 和 UIViewGroup.....      | 4  |
| 2.2.1 UIView 函数介绍.....             | 4  |
| 2.2.2 UIView 的监听器介绍.....           | 11 |
| 2.2.3 UIViewGroup 介绍.....          | 13 |
| 2.3 容器类组件.....                     | 14 |
| 2.3.1 RootView.....                | 14 |
| 2.3.2 GridLayout.....              | 16 |
| 2.3.3 FlexLayout.....              | 16 |
| 2.3.4 ListLayout.....              | 17 |
| 2.3.5 UIAbstractScroll.....        | 17 |
| 2.3.6 UIScrollView.....            | 18 |
| 2.3.7 UISwipeView.....             | 19 |
| 2.3.8 UISwipeViewNested.....       | 20 |
| 2.3.9 UICollection.....            | 20 |
| 2.3.10 UICollectionViewExt.....    | 22 |
| 2.3.11 UICollectionViewNested..... | 22 |
| 2.3.12 UIAnalogClock.....          | 23 |
| 2.3.13 UISweepClock.....           | 24 |
| 2.3.14 UIDigitalClock.....         | 28 |
| 2.3.15 UIPickerView.....           | 29 |
| 2.3.16 UITimePicker.....           | 31 |
| 2.3.17 UIAxis.....                 | 31 |
| 2.3.18 UIChart.....                | 33 |
| 2.3.19 UICrossView.....            | 35 |

|                                   |           |
|-----------------------------------|-----------|
| 2.3.20 UIRollerView .....         | 38        |
| 2.3.21 UICoverflowView .....      | 40        |
| 2.3.22 UICoverflowView2 .....     | 42        |
| 2.3.23 UISphereView .....         | 43        |
| 2.3.24 UISlipflowView .....       | 46        |
| 2.3.25 UIIcosahedronView .....    | 47        |
| 2.3.26 UITransformGroup .....     | 48        |
| 2.3.27 UITransformList .....      | 49        |
| 2.3.28 UIHexagonList .....        | 49        |
| 2.3.29 UICustomHexagonsList ..... | 51        |
| 2.3.30 UIWaterfallList .....      | 51        |
| 2.3.31 UIParticleView .....       | 52        |
| 2.3.32 UIMapView .....            | 56        |
| 2.4 非容器类组件 .....                  | 59        |
| 2.4.1 UIImageView .....           | 59        |
| 2.4.2 UIImageAnimator .....       | 60        |
| 2.4.3 UILabel .....               | 62        |
| 2.4.4 UILabelExt .....            | 63        |
| 2.4.5 UIButton .....              | 63        |
| 2.4.6 UILabelButtonExt .....      | 64        |
| 2.4.7 UILabelButton .....         | 64        |
| 2.4.8 UICheckBox .....            | 65        |
| 2.4.9 UIRadioButton .....         | 66        |
| 2.4.10 UIToggleButton .....       | 66        |
| 2.4.11 UIArcLabel .....           | 66        |
| 2.4.12 UIBoxProgress .....        | 68        |
| 2.4.13 UICircleProgress .....     | 69        |
| 2.4.14 UISlider .....             | 70        |
| 2.4.15 UITextureMapper .....      | 71        |
| 2.4.16 UIQrcode .....             | 72        |
| 2.4.17 UIBarcode .....            | 74        |
| 2.4.18 UICanvas .....             | 74        |
| 2.4.19 UICanvasExt .....          | 76        |
| 2.4.20 UILiteSurfaceView .....    | 78        |
| 2.4.21 UIEditText .....           | 80        |
| 2.4.22 UIKaleidoscopeView .....   | 83        |
| <b>3 支持振动 .....</b>               | <b>85</b> |
| 3.1 功能介绍 .....                    | 85        |
| 3.2 振动类型 .....                    | 85        |
| 3.3 对外接口 .....                    | 86        |

|                                     |            |
|-------------------------------------|------------|
| <b>4 多语言切换</b>                      | <b>87</b>  |
| 4.1 功能介绍                            | 87         |
| 4.2 对外接口                            | 88         |
| 4.2.1 FontGlobalManager             | 88         |
| 4.2.2 UILabelExt 和 UILabelButtonExt | 90         |
| 4.3 XML APIs                        | 90         |
| 4.3.1 根元素                           | 90         |
| 4.3.2 字符串元素                         | 91         |
| <b>5 多语言文本显示</b>                    | <b>93</b>  |
| 5.1 功能介绍                            | 93         |
| 5.2 关键对外接口                          | 94         |
| 5.3 开发建议                            | 96         |
| <b>6 图片资源管理</b>                     | <b>97</b>  |
| 6.1 方案目的                            | 97         |
| 6.2 整体功能介绍                          | 97         |
| 6.3 服务器要求                           | 99         |
| 6.4 图片资源打包                          | 99         |
| 6.4.1 配置文件说明                        | 99         |
| 6.4.2 工具输入要求                        | 100        |
| 6.4.3 打包使用                          | 101        |
| 6.5 资源使用                            | 101        |
| <b>7 离线表盘</b>                       | <b>104</b> |
| 7.1 方案介绍                            | 104        |
| 7.2 AOD 离线表盘                        | 105        |
| 7.2.1 状态机                           | 105        |
| 7.2.2 AodView                       | 108        |
| 7.2.3 AOD 离线表盘                      | 108        |
| 7.3 表盘制作                            | 109        |
| 7.4 数据类型 model                      | 124        |
| 7.5 相关类介绍                           | 125        |
| 7.5.1 DialViewGroup                 | 125        |

|                                   |            |
|-----------------------------------|------------|
| 7.5.2 DialBaseModel.....          | 125        |
| 7.5.3 ModelDialDataFactory .....  | 126        |
| 7.6 协议说明.....                     | 127        |
| <b>8 在线表盘.....</b>                | <b>128</b> |
| 8.1 简介 .....                      | 128        |
| 8.2 参考实现.....                     | 129        |
| 8.3 注册方案.....                     | 130        |
| <b>9 桌面样式.....</b>                | <b>131</b> |
| 9.1 简介 .....                      | 131        |
| 9.2 参考实现.....                     | 132        |
| 9.3 注册方案.....                     | 135        |
| <b>10 Graphic service .....</b>   | <b>136</b> |
| 10.1 整体方案.....                    | 136        |
| 10.2 对外接口.....                    | 137        |
| <b>11 表情包支持 .....</b>             | <b>138</b> |
| 11.1 方案介绍 .....                   | 138        |
| 11.2 制作表情包数据库和表情包映射表 .....        | 138        |
| 11.3 绘制原理 .....                   | 139        |
| <b>12 DFX.....</b>                | <b>140</b> |
| 12.1 内存占用.....                    | 140        |
| 12.2 帧率 .....                     | 141        |
| 12.3 截图 .....                     | 141        |
| 12.4 模拟事件.....                    | 142        |
| 12.5 时延统计 .....                   | 144        |
| 12.6 控件树打印 .....                  | 146        |
| 12.7 内存泄漏定位 .....                 | 146        |
| 12.8 Native 状态打印 .....            | 146        |
| <b>13 支持标准解码 JPEG 和 PNG .....</b> | <b>148</b> |

## 插图目录

|                                       |    |
|---------------------------------------|----|
| 图 1-1 图形子系统架构图 .....                  | 1  |
| 图 2-1 组件继承关系图 .....                   | 4  |
| 图 2-2 UISweepClock 效果 .....           | 25 |
| 图 2-3 垂直指针图片 .....                    | 27 |
| 图 2-4 center 位置示意图 .....              | 28 |
| 图 2-5 UIDigitalClock 效果 .....         | 29 |
| 图 2-6 UIChartPillar 效果 .....          | 34 |
| 图 2-7 UIChartPolyline 效果 .....        | 34 |
| 图 2-8 UIRollerView Display 模式效果 ..... | 39 |
| 图 2-9 覆盖流效果 .....                     | 41 |
| 图 2-10 UICoverflowView 实例显示效果 .....   | 42 |
| 图 2-11 球形效果 .....                     | 44 |
| 图 2-12 滑动流效果 .....                    | 46 |
| 图 2-13 足球效果 .....                     | 47 |
| 图 2-14 足球面的图片要求 .....                 | 48 |
| 图 2-15 粒子效果 .....                     | 53 |
| 图 2-16 私有格式结构图 .....                  | 57 |
| 图 2-17 显示效果 .....                     | 58 |
| 图 2-18 UIArcLabel 效果 .....            | 67 |
| 图 2-19 UISlider 效果 .....              | 70 |



|                                     |     |
|-------------------------------------|-----|
| 图 2-20 UIQrcode 效果 .....            | 73  |
| 图 2-21 UIBarcode 效果.....            | 74  |
| 图 2-22 UIEditText 两种模式显示效果图.....    | 81  |
| 图 2-23 软键盘效果图 .....                 | 82  |
| 图 2-24 万花筒效果示意图 .....               | 84  |
| 图 4-1 原理框图 .....                    | 88  |
| 图 5-1 多字言文本混排效果 .....               | 94  |
| 图 6-1 方案总体实现原理图 .....               | 98  |
| 图 7-1 离线表盘整体方案 .....                | 105 |
| 图 7-2 display_state_machine.c ..... | 106 |
| 图 7-3 model 设计 .....                | 124 |
| 图 7-4 Bin 文件的数据说明 .....             | 127 |
| 图 8-1 在线表盘样例 .....                  | 129 |
| 图 9-1 列表风格桌面 .....                  | 132 |
| 图 9-2 UIDesktopFragment 定义 .....    | 133 |
| 图 9-3 UIDesktopFragment 类图 .....    | 133 |
| 图 10-1 Graphic service 总体方案 .....   | 136 |

## 表格目录

|                                      |    |
|--------------------------------------|----|
| 表 2-1 UIView 常用函数说明.....             | 4  |
| 表 2-2 OnClickListener 回调函数.....      | 11 |
| 表 2-3 OnLongPressListener 回调函数 ..... | 11 |
| 表 2-4 OnDragListener 回调函数.....       | 12 |
| 表 2-5 OnTouchListener 回调函数.....      | 12 |
| 表 2-6 OnRotateListener 回调函数.....     | 13 |
| 表 2-7 UIViewGroup 核心函数.....          | 13 |
| 表 2-8 RootView 核心函数.....             | 15 |
| 表 2-9 OnKeyActListener 回调函数 .....    | 15 |
| 表 2-10 GridLayout 核心函数.....          | 16 |
| 表 2-11 FlexLayout 核心函数.....          | 16 |
| 表 2-12 ListLayout 核心函数.....          | 17 |
| 表 2-13 UIAbstractScroll 核心函数 .....   | 17 |
| 表 2-14 UIScrollView 核心函数.....        | 18 |
| 表 2-15 UISwipeView 核心函数.....         | 19 |
| 表 2-16 UISwipeViewNested 核心函数.....   | 20 |
| 表 2-17 UIList 核心函数 .....             | 21 |
| 表 2-18 UIListExt 核心函数.....           | 22 |
| 表 2-19 UIListNested 核心函数.....        | 22 |
| 表 2-20 UIAnalogClock 核心函数 .....      | 23 |

|                                      |    |
|--------------------------------------|----|
| 表 2-21 UISweepClock 核心函数.....        | 25 |
| 表 2-22 UIDigitalClock 核心函数.....      | 29 |
| 表 2-23 UIPickerView 核心函数.....        | 30 |
| 表 2-24 UITimePicker 核心函数.....        | 31 |
| 表 2-25 UIAxis 核心函数.....              | 32 |
| 表 2-26 UIXAxis 核心函数.....             | 32 |
| 表 2-27 UIYAxis 核心函数.....             | 32 |
| 表 2-28 UIChart 核心函数.....             | 33 |
| 表 2-29 UIChartPillar 核心函数.....       | 34 |
| 表 2-30 UIChartPolyline 核心函数.....     | 34 |
| 表 2-31 UICrossView 核心函数.....         | 35 |
| 表 2-32 UICardPage 核心函数.....          | 36 |
| 表 2-33 CardSwipe 核心函数.....           | 38 |
| 表 2-34 UIRollerView 核心函数.....        | 39 |
| 表 2-35 UICoverflowView 核心函数.....     | 41 |
| 表 2-36 UICoverflowView2 核心函数.....    | 43 |
| 表 2-37 UISphereView 核心参数.....        | 44 |
| 表 2-38 UISlipflowView 核心函数.....      | 46 |
| 表 2-39 UIIcosahedronView 核心函数.....   | 48 |
| 表 2-40 UITransformGroup 核心函数.....    | 49 |
| 表 2-41 UITransformList 核心函数.....     | 49 |
| 表 2-42 UIHexagonList 核心函数.....       | 50 |
| 表 2-43 UICustomHexagonList 核心函数..... | 51 |
| 表 2-44 UIWaterfallList 核心函数.....     | 51 |
| 表 2-45 UIParticleView 核心函数.....      | 53 |
| 表 2-46 ParticleCell 核心函数.....        | 54 |
| 表 2-47 UIMapView 核心函数.....           | 58 |

|                                      |    |
|--------------------------------------|----|
| 表 2-48 UIImageView 核心函数.....         | 59 |
| 表 2-49 UIImageViewAnimator 核心函数..... | 60 |
| 表 2-50 UILabel 核心函数.....             | 62 |
| 表 2-51 UILabelExt 核心函数.....          | 63 |
| 表 2-52 UIButton 核心函数.....            | 63 |
| 表 2-53 UILabelButtonExt 核心函数.....    | 64 |
| 表 2-54 UILabelButton 核心函数.....       | 64 |
| 表 2-55 UICheckBox 核心函数.....          | 65 |
| 表 2-56 UIRadioButton 核心函数.....       | 66 |
| 表 2-57 UIArcLabel 核心函数.....          | 67 |
| 表 2-58 UIAbstractProgress 核心函数.....  | 68 |
| 表 2-59 UIBoxProgress 核心函数.....       | 69 |
| 表 2-60 UICircleProgress 核心函数.....    | 69 |
| 表 2-61 UISlider 核心函数.....            | 70 |
| 表 2-62 UITextureMappre 核心函数.....     | 72 |
| 表 2-63 UIQrcode 核心函数.....            | 73 |
| 表 2-64 UIBarcode 核心函数函数介绍.....       | 74 |
| 表 2-65 UICanvas 核心函数.....            | 74 |
| 表 2-66 PaintExt 核心函数.....            | 76 |
| 表 2-67 UICanvasExt 核心函数.....         | 77 |
| 表 2-68 UILiteSurfaceView 核心函数.....   | 79 |
| 表 2-69 LiteSurface 核心函数.....         | 79 |
| 表 2-70 UIEditText 核心函数.....          | 82 |
| 表 2-71 UIKaleidoscopeView 核心函数.....  | 84 |
| 表 3-1 VibratorManager 核心函数.....      | 86 |
| 表 3-2 支持振动的控件.....                   | 86 |
| 表 4-1 FontGlobalManager 核心函数.....    | 88 |

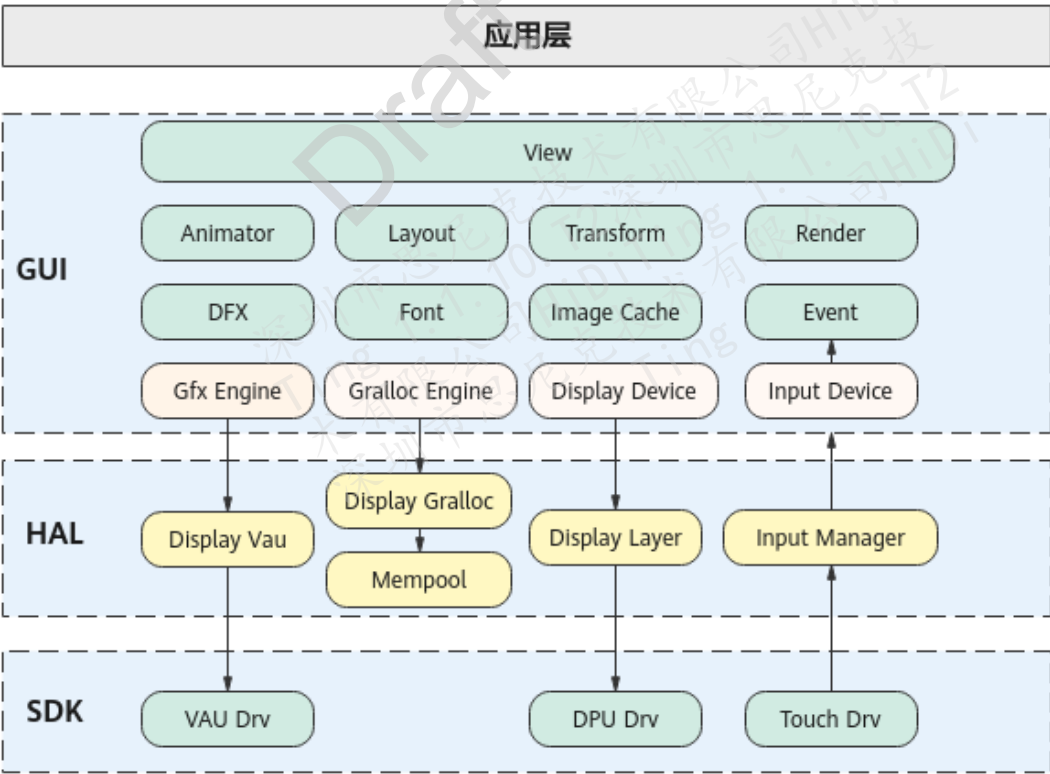
|                                      |     |
|--------------------------------------|-----|
| 表 4-2 UILabelButtonExt 核心函数 .....    | 90  |
| 表 4-3 多语言 xml 配置 .....               | 90  |
| 表 4-4 字符串元素配置.....                   | 91  |
| 表 5-1 关键对外接口 .....                   | 94  |
| 表 6-1 图片资源打包工具配置.....                | 99  |
| 表 6-2 ImageCacheManager 核心函数.....    | 101 |
| 表 7-1 AodView 成员函数 .....             | 108 |
| 表 7-2 xml 标签和属性说明.....               | 109 |
| 表 7-3 bind_data 范围定义 .....           | 123 |
| 表 7-4 DialViewGroup 主要函数 .....       | 125 |
| 表 7-5 DialBaseModel 主要函数.....        | 125 |
| 表 7-6 ModelDialDataFactory 主要函数..... | 126 |
| 表 9-1 UIDesktopFragment 关键接口说明.....  | 133 |
| 表 10-1 graphic service 对外 API .....  | 137 |

# 1 概述

图形子系统如图 1-1 所示，主要包括 UI 组件、布局、动画、字体、输入事件、窗口管理、渲染绘制等模块。

构建基于轻量 OS 的图形框架，满足硬件资源较小的物联网设备的系统应用开发。

图1-1 图形子系统架构图



- 各模块介绍

- View: 应用组件, 包括 UIView、UIViewGroup、UIButton、UILabel、UILabelButton、UICollection、UISlider 等。
- Animator: 动画模块, 开发者可以自定义动画。
- Layout: 布局控件, 包括 Flexlayout、GridLayout、ListLayout 等。
- Transform: 图形变换模块, 包括旋转、平移、缩放等。
- Event: 事件模块, 包括 click、press、drag、long press 等基础事件。
- Render: 渲染绘制模块。
- draw2D: 2D 绘制模块, 包括直线、矩形、圆、弧、图片、文字等绘制, 包括软件绘制和硬件加速能力对接。
- Multi-language: 多语言模块, 用于处理不同语言文字的换行、整形等。
- Input Manager: 输入事件管理模块。
- Display Layer: 图层相关操作的 HAL 层封装。
- Display Gralloc: 图形内存管理模块。
- Display Vau: 图形渲染、搬移、合成等硬件能力的 HAL 层封装。
- Input Device: 外部事件统一管理 (当前只支持 touch)。

## • 目录

open\_source/openharmony/foundation/graphic # uikit 原生

├─ ui # UI 模块, 包括 UI 控件、动画、字体等功能组件

└─ utils # 图形基础库及南向适配层

middleware/services/gui # uikit 自研

├─ hal

| └─ display # 显示相关功能的 HAL 层

| └─ input # 输入相关功能的 HAL 层

└─ uikit

├─ proprietary # 自研代码, 包括软件盘、扫描、硬件画布等控件, 以及 graphic service 服务

└─ sample # uikit 演示用例

# 2 组件介绍

## 2.1 概述

## 2.2 UIView 和 UIViewGroup

## 2.3 容器类组件

## 2.4 非容器类组件

## 2.1 概述

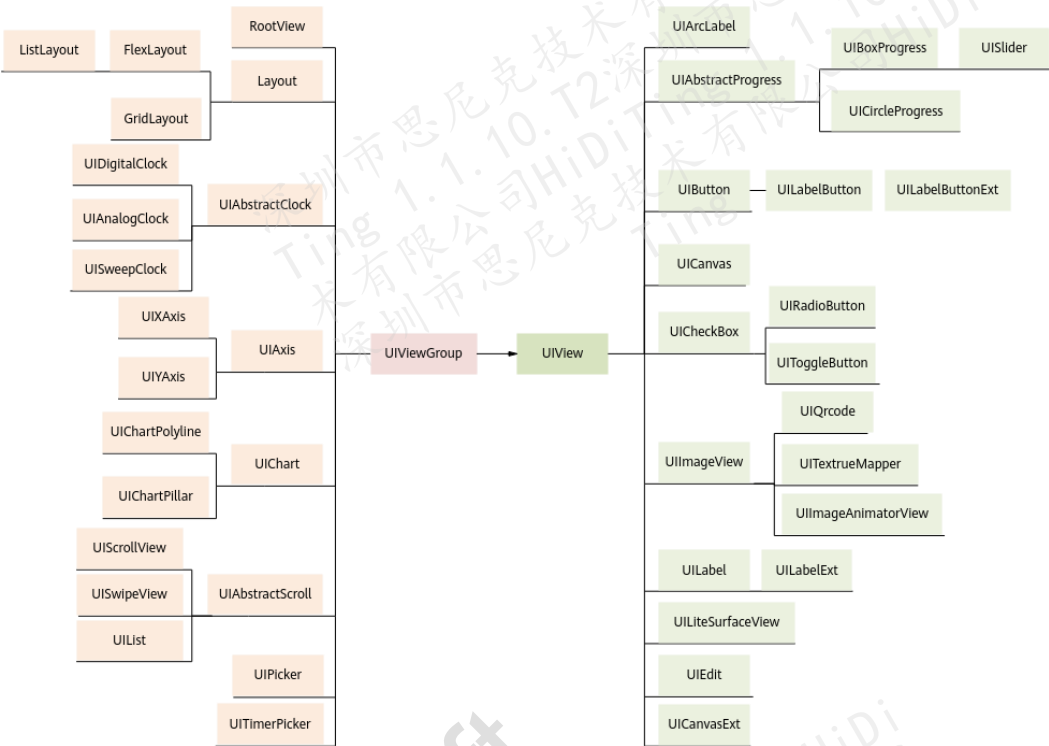
所有组件均继承于 UIView，组件分为基础组件和容器组件。

- 基础组件仅实现组件本身单一功能，例如 UIButton、UILabel、UIImageView 等。
- 容器类组件可以添加其他组件作为自身的子组件，例如 RootView、UICollection、UIPicker 等。

本章主要介绍各个组件的使用，并以 sample 的形式呈现，指导用户开发。



图2-1 组件继承关系图



## 2.2 UIView 和 UIViewGroup

UIView 为所有组件的父类，所有组件都继承自 UIView，实现自定义扩展。例如 UIViewGroup 继承于 UIView，并扩展其容器特性（允许添加其他组件）。  
下面分别介绍了 UIView 和 UIViewGroup 的常用函数，同时也是所有派生组件的公共函数。

### 2.2.1 UIView 函数介绍

UIView 常用函数如表 2-1 所示，用户可以结合实际情况使用。

表2-1 UIView 常用函数说明

| 函数                                                  | 介绍                                                  |
|-----------------------------------------------------|-----------------------------------------------------|
| virtual bool OnPreDraw(Rect& invalidatedArea) const | 在绘制视图之前调用。该函数用于检查失效区域是否可以被该视图完全覆盖，以优化绘制过程。主要用在自定义组件 |

| 函数                                                   | 介绍                                                                  |
|------------------------------------------------------|---------------------------------------------------------------------|
|                                                      | 的场景，自定义的 View 重写该函数。                                                |
| virtual void OnDraw(const Rect& invalidatedArea)     | 通过调用当前 OnDraw 函数来绘制，主要用在自定义组件的场景，自定义 View 重写该函数。                    |
| virtual void OnPostDraw(const Rect& invalidatedArea) | 调用 OnDraw 函数后回调该函数，主要用在自定义组件的场景，自定义 View 重写该函数。                     |
| virtual void ReMeasure()                             | 重新测量视图大小。                                                           |
| void Invalidate()                                    | 刷新视图的区域。                                                            |
| void InvalidateRect(const Rect& invalidatedArea)     | 刷新视图的指定区域。                                                          |
| void SetParent(UIView* parent)                       | 设置视图的父视图。                                                           |
| UIView* GetParent() const                            | 获取视图的父视图。                                                           |
| void SetNextSibling(UIView* sibling)                 | 设置视图的下一个同级视图。                                                       |
| UIView* GetNextSibling() const                       | 获取视图的下一个同级视图。                                                       |
| virtual void SetVisible(bool visible)                | 设置视图是否可见。                                                           |
| bool IsVisible() const                               | 获取视图是否可见。                                                           |
| void SetTouchable(bool touch)                        | 设置视图是否可触摸。                                                          |
| bool IsTouchable() const                             | 获取视图是否可触摸。                                                          |
| void SetDraggable(bool draggable)                    | 设置视图是否可拖动。<br><br>注意：可拖动的前提为可触摸，仅这两个属性均被设置为 true 时，才能支持拖动交互。        |
| bool IsDraggable() const                             | 获取视图是否可拖动。                                                          |
| void SetDragParentInstead(bool dragParentInstead)    | 设置拖动视图时是否将拖动事件传递给父视图进行处理。<br><br>注意：在嵌套场景中，如果父子控件都需要响应拖动事件，子控件该属性应当 |

| 函数                                                                                                                    | 介绍                                               |
|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
|                                                                                                                       | 确保已经被设置为 false，否则子控件的拖拽事件会被传递到父控件，导致子控件的拖拽事件不响应。 |
| <code>bool IsDragParentInstead() const</code>                                                                         | 获取是否拖动事件会传递给父视图进行处理。                             |
| <code>Rect GetRect() const</code>                                                                                     | 获取视图的绝对矩形区域，如经仿射变换，则为变换后的有效区域。                   |
| <code>Rect GetVisibleRect() const</code>                                                                              | 获取可见的绝对矩形区域。                                     |
| <code>Rect GetMaskedRect() const</code>                                                                               | 获取有效的绝对矩形区域。                                     |
| <code>Rect GetOrigRect() const</code>                                                                                 | 获取控件设置的绝对矩形区域。                                   |
| <code>virtual Rect GetContentRect()</code>                                                                            | 获取控件内容的绝对矩形区域，不包含 padding 和 border。              |
| <code>Rect GetRelativeRect() const</code>                                                                             | 获取相对于父视图的相对矩形区域。                                 |
| <code>void ResizeVisibleArea(int16_t x, int16_t y, int16_t width, int16_t height)</code>                              | 调整可见区域的大小。此操作可能会影响最终显示大小。                        |
| <code>virtual void SetWidth(int16_t width)</code>                                                                     | 设置视图的宽度。                                         |
| <code>virtual void SetHeight(int16_t height)</code>                                                                   | 设置视图的高度。                                         |
| <code>virtual void SetWidthPercent(float widthPercent)</code>                                                         | 按照相对于父视图的百分比来设置宽度。                               |
| <code>virtual void SetHeightPercent(float heightPercent)</code>                                                       | 按照相对于父视图的百分比来设置高度。                               |
| <code>virtual void Resize(int16_t width, int16_t height)</code>                                                       | 调整当前视图大小。                                        |
| <code>virtual void SetPosition(int16_t x, int16_t y)</code>                                                           | 设置当前视图的起始点坐标。                                    |
| <code>virtual void SetPosition(int16_t x, int16_t y, int16_t width, int16_t height)</code>                            | 设置视图的 X、Y 坐标以及宽高。                                |
| <code>virtual void SetPositionPercent(float xPercent, float yPercent, float widthPercent, float heightPercent)</code> | 按照父视图的百分比来设置 X、Y 坐标以及宽高。                         |

| 函数                                                                  | 介绍                                                                                                                                                                                               |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| virtual void ResizePercent(float widthPercent, float heightPercent) | 按照父视图的百分比来设置宽高。                                                                                                                                                                                  |
| virtual void SetX(int16_t x)                                        | 设置视图左上点的 X 坐标。                                                                                                                                                                                   |
| virtual void SetXPercent(float xPercent)                            | 按照相对于父视图的百分比来设置 X 坐标。                                                                                                                                                                            |
| int16_t GetX() const                                                | 获取视图的 X 坐标。                                                                                                                                                                                      |
| virtual void SetY(int16_t y)                                        | 设置视图左上点的 Y 坐标。                                                                                                                                                                                   |
| virtual void SetYPercent(float yPercent)                            | 按照相对于父视图的百分比来设置 Y 坐标。                                                                                                                                                                            |
| int16_t GetY() const                                                | 获取视图的 Y 坐标。                                                                                                                                                                                      |
| bool IsViewGroup() const                                            | 当前视图是否包含其他视图。                                                                                                                                                                                    |
| void SetIntercept(bool isIntercept)                                 | <p>设置是否拦截事件。仅在未注册对应事件监听器时生效：</p> <ul style="list-style-type: none"> <li>如果值为 true，则不会将事件传递到父视图。</li> <li>如果值为 false，则会将事件传递到父视图。</li> </ul> <p>已注册对应事件监听器时，则根据回调函数返回值判断（同上）是否已经消费事件而不传递给父视图。</p> |
| bool IsIntercept()                                                  | 获取是否拦截事件。                                                                                                                                                                                        |
| void SetTransformMap(const TransformMap& transMap)                  | 设置视图的仿射变换。                                                                                                                                                                                       |
| TransformMap& GetTransformMap()                                     | 获取视图的放射变换。                                                                                                                                                                                       |
| void SetViewId(const char* id)                                      | 给视图设置 ID。                                                                                                                                                                                        |
| const char* GetViewId() const                                       | 获取视图的 ID。                                                                                                                                                                                        |
| void SetViewIndex(int16_t index)                                    | 设置视图下标。                                                                                                                                                                                          |
| int16_t GetViewIndex() const                                        | 获取视图下标。                                                                                                                                                                                          |

| 函数                                                                                           | 介绍                                                                                                                              |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| virtual UIViewType GetViewType()<br>const                                                    | 获取视图类型。                                                                                                                         |
| void SetOpaScale(uint8_t opaScale)                                                           | 设置视图的不透明度，合法值为[0, 255]。                                                                                                         |
| uint8_t GetOpaScale() const                                                                  | 获取视图的不透明度。                                                                                                                      |
| ViewExtraMsg* GetExtraMsg()                                                                  | 获取视图的额外信息。                                                                                                                      |
| void SetExtraMsg(ViewExtraMsg* extraMsg)                                                     | 设置视图的额外信息。                                                                                                                      |
| void Rotate(int16_t angle, const Vector2<float>& pivot)                                      | 旋转该视图。 <ul style="list-style-type: none"> <li>参数 angle：旋转角度。</li> <li>参数 pivot：旋转中心点。</li> </ul>                                |
| void Rotate(int16_t angle, const Vector3<float>& pivotStart, const Vector3<float>& pivotEnd) | 旋转该视图。 <ul style="list-style-type: none"> <li>参数 angle：旋转角度。</li> <li>参数 pivotStart：旋转起点。</li> <li>参数 pivotEnd：旋转终点。</li> </ul> |
| void Scale(const Vector2<float>& scale, const Vector2<float>& pivot)                         | 缩放该视图。 <ul style="list-style-type: none"> <li>参数 scale：缩放系数 (x, y) 。</li> <li>参数 pivot：缩放中心点。</li> </ul>                        |
| void Scale(const Vector3<float>& scale, const Vector3<float>& pivot)                         | 缩放该视图。 <ul style="list-style-type: none"> <li>参数 scale：缩放系数 (x, y, z) 。</li> <li>参数 pivot：缩放中心点。</li> </ul>                     |
| void Translate(const Vector2<int16_t>& trans)                                                | 平移该视图。 <ul style="list-style-type: none"> <li>参数 trans：平移系数 (x, y) 。</li> </ul>                                                 |
| void Translate(const Vector3<int16_t>& trans)                                                | 平移该视图。 <ul style="list-style-type: none"> <li>参数 trans：平移系数 (x, y, z) 。</li> </ul>                                              |
| void SetCameraDistance(int16_t distance)                                                     | 设置视点距离（三维空间透视投影 z 轴方向）。 <ul style="list-style-type: none"> <li>参数 distance：视点距离。</li> </ul>                                     |

| 函数                                                                                                   | 介绍                                                                                                                                  |
|------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| void SetCameraPosition(const Vector2<float>& position)                                               | 设置视点位置（三维空间透视投影 x、y 平面位置） <ul style="list-style-type: none"> <li>参数 position：视点位置。</li> </ul>                                       |
| void Shear(const Vector2<float>& shearX, const Vector2<float>& shearY, const Vector2<float>& shearZ) | 切变该视图。 <ul style="list-style-type: none"> <li>参数 shearX：x 轴切变参数。</li> <li>参数 shearY：y 轴切变参数。</li> <li>参数 shearZ：z 轴切变参数。</li> </ul> |
| void SetOnDragListener(OnDragListener* onDragListener)                                               | 设置拖动事件监听。                                                                                                                           |
| void SetOnClickListener(OnClickListener* onClickListener)                                            | 设置点击事件监听。                                                                                                                           |
| void SetOnLongPressListener(OnLongPressListener* onLongPressListener)                                | 设置长按事件监听。                                                                                                                           |
| void SetOnRotateListener(OnRotateListener* onRotateListener)                                         | 设置旋转表冠事件监听。                                                                                                                         |
| void SetOnTouchListener(OnTouchListener* onTouchListener)                                            | 设置触摸事件监听。                                                                                                                           |
| virtual bool OnLongPressEvent(const LongPressEvent& event)                                           | 回调长按事件监听，通过返回值判断是否继续上报。                                                                                                             |
| virtual bool OnDragStartEvent(const DragEvent& event)                                                | 回调拖动开始事件监听，通过返回值判断是否继续上报。                                                                                                           |
| virtual bool OnDragEvent(const DragEvent& event)                                                     | 回调拖动中事件监听，通过返回值判断是否继续上报。                                                                                                            |
| virtual bool OnDragEndEvent(const DragEvent& event)                                                  | 回调拖动结束事件监听，通过返回值判断是否继续上报。                                                                                                           |
| virtual bool OnClickEvent(const ClickEvent& event)                                                   | 回调点击事件监听，通过返回值判断是否继续上报。                                                                                                             |
| virtual bool OnPressEvent(const PressEvent& event)                                                   | 回调触摸事件监听，通过返回值判断是否继续上报。                                                                                                             |

| 函数                                                        | 介绍                                                                                           |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------|
|                                                           | 否继续上报。                                                                                       |
| virtual bool OnCancelEvent(const CancelEvent& event)      | 回调触摸取消事件监听，通过返回值判断是否取消上报。                                                                    |
| virtual bool OnReleaseEvent(const ReleaseEvent& event)    | 回调触摸释放事件监听，通过返回值判断是否继续上报。                                                                    |
| virtual bool OnRotateStartEvent(const RotateEvent& event) | 回调旋转表冠开始事件监听，仅获焦视图可响应此事件。                                                                    |
| virtual bool OnRotateEvent(const RotateEvent& event)      | 回调旋转表冠事件监听，仅获焦视图可响应此事件。                                                                      |
| virtual bool OnRotateEndEvent(const RotateEvent& event)   | 回调旋转表冠结束事件监听，仅获焦视图可响应此事件。                                                                    |
| void SetBackgroundBlur(float radius)                      | 对最上层可见的静态控件设置背景高斯模糊。<br><br>参数 radius：模糊半径。内部调参值，并非指半径内的区域才会模糊，此值决定不同程度的高斯模糊效果，合法值为 [0, 32]。 |
| void ClearBackgroundBlur()                                | 清除控件设置的背景高斯模糊效果。                                                                             |
| void SetFullScreenBlur(bool isFull)                       | 设置高斯模糊为全屏高斯模糊（默认为局部高斯模糊）。                                                                    |
| bool IsFullScreenBlur() const                             | 获取高斯模糊状态。                                                                                    |
| virtual void RequestFocus()                               | 使控件获焦。                                                                                       |
| virtual void ClearFocus()                                 | 使控件失焦。                                                                                       |
| virtual void SetFocusable(bool focusable)                 | 设置控件能否获焦。                                                                                    |
| virtual bool IsFocusable() const                          | 获取控件能否获焦。                                                                                    |

说明

高斯模糊只支持 1 个控件持有。由于 BrandyK100 芯片的限制，如果高斯模糊在特定的模糊半径下的显示效果有方块感，建议添加蒙板来解决。

2.2.2 UIView 的监听器介绍

UIView 中定义了多种监听器，使用方式基本相同。用户需实例化监听器或者继承监听器，实现抽象函数，在执行具体操作时，会回调监听器中对应的回调函数。用户可以在回调函数中添加需要执行的逻辑。

下面分别介绍了 UIView 中定义的监听器。

- `OnClickListener`，该监听器主要用来响应处理点击事件。

表2-2 OnClickListener 回调函数

| 函数                                                                                           | 介绍                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>virtual bool<br/>OnClick(UIView&amp;<br/>view, const<br/>ClickEvent&amp; event)</code> | 当视图被点击时触发回调。 <ul style="list-style-type: none"><li>• 参数 <code>view</code>：点击的视图。</li><li>• 参数 <code>event</code>：点击事件。</li></ul> 返回值为 <code>bool</code> 类型： <ul style="list-style-type: none"><li>• 返回 <code>true</code> 表示事件已经处理。</li><li>• 返回 <code>false</code> 表示事件未处理，会将该事件继续传递到父视图。</li></ul> |

- `OnLongPressListener`，该监听器主要用来响应处理长按事件。

表2-3 OnLongPressListener 回调函数

| 函数                                                                                                        | 介绍                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>virtual bool<br/>OnLongPress(UIVie<br/>w&amp; view, const<br/>LongPressEvent&amp;<br/>event)</code> | 当视图被长按时触发回调。 <ul style="list-style-type: none"><li>• 参数 <code>view</code>：长按的视图。</li><li>• 参数 <code>event</code>：长按事件。</li></ul> 返回值为 <code>bool</code> 类型： <ul style="list-style-type: none"><li>• 返回 <code>true</code> 表示事件已经处理。</li><li>• 返回 <code>false</code> 表示事件未处理，会将该事件继续传递到</li></ul> |



| 函数 | 介绍   |
|----|------|
|    | 父视图。 |

- OnDragListener，该监听器主要用来响应处理拖动事件。

表2-4 OnDragListener 回调函数

| 函数                                                             | 介绍          |
|----------------------------------------------------------------|-------------|
| virtual bool OnDragStart(UIView& view, const DragEvent& event) | 当视图开始拖动时调用。 |
| virtual bool OnDrag(UIView& view, const DragEvent& event)      | 拖动视图过程中调用。  |
| virtual bool OnDragEnd(UIView& view, const DragEvent& event)   | 当视图停止拖动时调用。 |

以上函数中，

- 参数 view：接受拖动事件的视图。
- 参数 event：拖动事件。

返回值为 bool 类型：

- 返回 true 表示事件已经处理。
- 返回 false 表示事件未处理，会将该事件继续传递到父视图。

- OnTouchListener，该监听器主要用于响应处理触摸事件。

表2-5 OnTouchListener 回调函数

| 函数                                                              | 介绍            |
|-----------------------------------------------------------------|---------------|
| virtual bool OnPress(UIView& view, const PressEvent& event)     | 当按下视图时调用。     |
| virtual bool OnRelease(UIView& view, const ReleaseEvent& event) | 当释放视图时调用。     |
| virtual bool OnCancel(UIView& view, const CancelEvent& event)   | 当取消视图上报事件时调用。 |

以上函数中，

- 参数 view：接受触摸事件的视图。
- 参数 event：分别为按下/松开/取消事件。

返回值为 bool 类型：

- 返回 true 表示事件已经处理。
  - 返回 false 表示事件未处理，会将该事件继续传递到父视图。
- OnRotateListener，该监听器主要用于响应表冠事件。

表2-6 OnRotateListener 回调函数

| 函数                                                                 | 介绍          |
|--------------------------------------------------------------------|-------------|
| virtual bool OnRotateStart(UIView& view, const RotateEvent& event) | 当开始旋转表冠时调用。 |
| virtual bool OnRotate(UIView& view, const ReleaseEvent& event)     | 旋转表冠过程中调用。  |
| virtual bool OnRotateEnd(UIView& view, const CancelEvent& event)   | 当结束旋转表冠时调用。 |

以上函数中，

- 参数 view：接受表冠事件的视图。
- 参数 event：表示表冠事件。

返回值为 bool 类型：

- 返回 true 表示事件已经处理。
- 返回 false 表示事件未处理，会将该事件继续传递到父视图。

### 2.2.3 UIViewGroup 介绍

UIViewGroup 继承于 UIView，常用函数如表 2-7 所示。

表2-7 UIViewGroup 核心函数

| 函数                                                        | 介绍                          |
|-----------------------------------------------------------|-----------------------------|
| virtual void Add(UIView* view)                            | 添加一个子视图。如果入参视图已经有父视图，则无法加入。 |
| virtual void Insert(UIView* prevView, UIView* insertView) | 在 prevView 视图后插入一个子视图。      |

| 函数                                                                                         | 介绍                                                                         |
|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| virtual void Remove(UIView* view)                                                          | 从当前视图树中移除指定子视图，非释放子视图。                                                     |
| virtual void RemoveAll()                                                                   | 从当前视图树中移除所有子视图，非释放子视图。                                                     |
| virtual void GetTargetView(const Point& point, UIView** last) override                     | 获取可见的、最上层的、可以响应指定坐标点的触摸事件的目标子视图。                                           |
| virtual void GetTargetView(const Point& point, UIView** current, UIView** target) override | 获取一个可见的、最上层的、可以响应指定坐标点的触摸事件的目标子视图 (current) 。获取一个最上层的视图包含指定的坐标点 (target) 。 |
| virtual void MoveChildByOffset(int16_t x, int16_t y)                                       | 移动所有子视图。                                                                   |
| UIView* GetChildrenHead() const                                                            | 获取此视图组中的最前一个子视图。                                                           |
| UIView* GetChildrenTail() const                                                            | 获取此视图组中的最后一个子视图。                                                           |
| UIView* GetChildById(const char* id) const                                                 | 通过校验 id，获取对应子视图。                                                           |
| void SetDisallowIntercept(bool flag)                                                       | 设置是否拦截触摸事件至子视图。                                                            |
| virtual UIView* GetChildById(const char* id) const override                                | 根据指定的 id 获取子控件。                                                            |
| void SetAutoSize(bool state)                                                               | 设置该视图是否适应子控件的尺寸：自调整为能够包含所有子视图的尺寸。                                          |

## 2.3 容器类组件

容器类组件均继承于 UIViewGroup，包含添加、删除、移动子节点等功能。布局类组件也属于容器类组件。

### 2.3.1 RootView

RootView 类为单例模式，是 UIKit 组件视图树的根节点。

表2-8 RootView 核心函数

| 函数                                                                                               | 介绍                            |
|--------------------------------------------------------------------------------------------------|-------------------------------|
| void SetOnKeyActListener(OnKeyActListener* onKeyActListener)                                     | 设置物理事件上报时需调用的监听器。             |
| virtual void OnKeyEvent(const KeyEvent& event)                                                   | 回调物理键事件监听器回调。                 |
| void ClearOnKeyActListener()                                                                     | 清除物理按键监听器。                    |
| void SetOnVirtualDeviceEventListener(OnVirtualDeviceEventListener* onVirtualDeviceEventListener) | 设置监听器，用于监控虚拟设备的输入事件。          |
| void ClearOnVirtualDeviceEventListener()                                                         | 清除虚拟设备监听器。                    |
| static bool FindSubView(const UIView& parentView, const UIView* subView);                        | 检查目标视图是否为指定父视图的子视图之一。         |
| void Invalidate() override                                                                       | 刷新视图。                         |
| void AddSystemView(UIView* view)                                                                 | 系统层添加视图（主要用于消息、弹窗类视图，显示在最上层）。 |
| void RemoveSystemView(UIView* view)                                                              | 系统层删除视图。                      |

OnKeyActListener，该监听器主要用来响应处理按键事件。

表2-9 OnKeyActListener 回调函数

| 函数                                                         | 介绍                                                                                                                                                                                                                                          |
|------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| virtual bool OnKeyAct(UIView& view, const KeyEvent& event) | <p>当触发按键事件时调用。</p> <ul style="list-style-type: none"><li>参数 view：接受按键事件的视图。</li><li>参数 event：按键事件。</li></ul> <p>返回值为 bool 类型：</p> <ul style="list-style-type: none"><li>返回 true 表示事件已经处理。</li><li>返回 false 表示事件未处理，会将该事件继续传递到父视图。</li></ul> |

## 2.3.2 GridLayout

GridLayout 又称为网格布局，继承关系为 GridLayout→Layout→UIViewGroup。使用方法可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_layout/ui\\_test\\_basic\\_layout.h](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_layout/ui_test_basic_layout.h) 和 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_layout/ui\\_test\\_basic\\_layout.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_layout/ui_test_basic_layout.cpp)。对外开发接口如表 2-10 所示。

表2-10 GridLayout 核心函数

| 函数                                               | 介绍                                |
|--------------------------------------------------|-----------------------------------|
| void SetRows(const uint16_t& rows)               | 设置行数。<br>参数 rows：行数。              |
| void SetCols(const uint16_t& cols)               | 设置列数。<br>参数 cols：列数。              |
| void LayoutChildren(bool needInvalidate = false) | 布局子组件。<br>参数 needInvalidate：是否刷新。 |

## 2.3.3 FlexLayout

FlexLayout 又称为弹性布局，继承关系为 FlexLayout→Layout→UIViewGroup。

表2-11 FlexLayout 核心函数

| 函数                                                 | 介绍                                     |
|----------------------------------------------------|----------------------------------------|
| void SetMajorAxisAlign(const AlignType& align)     | 设置主方向上的布局方式。<br>参数 align：开始、居中、结尾等     |
| void SetSecondaryAxisAlign(const AlignType& align) | 设置副方向上的布局方式。<br>参数 align：开始、居中、结尾等。    |
| void SetFlexWrap(uint8_t wrap)                     | 设置是否支持换行 NOWRAP（默认不换行）。<br>参数 wrap：换行。 |
| void LayoutChildren(bool needInvalidate = false)   | 布局子组件。                                 |

| 函数 | 介绍                      |
|----|-------------------------|
|    | 参数 needInvalidate：是否刷新。 |

2.3.4 ListLayout

ListLayout 又称为线性布局，继承关系为 ListLayout→FlexLayout→Layout→UIViewGroup，和 Android 中的 LinearLayout 类似。

表2-12 ListLayout 核心函数

| 函数                                   | 介绍                                                                                                                     |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| void SetDirection(uint8_t direction) | 设置布局方向。<br><br>参数 direction：方向 <ul style="list-style-type: none"><li>• HORIZONTAL（横向）</li><li>• VERTICAL（纵向）</li></ul> |
| uint8_t GetDirection()               | 获取布局方向。                                                                                                                |

2.3.5 UIAbstractScroll

UIAbstractScroll 是所有支持滑动的容器组件的父类。

表2-13 UIAbstractScroll 核心函数

| 函数                                           | 介绍                      |
|----------------------------------------------|-------------------------|
| void SetScrollBlankSize(uint16_t size)       | 设置头结点或尾节点可滑动超出顶部或底部的距离。 |
| void SetMaxScrollDistance(uint16_t distance) | 设置单次滑动手势造成的滑动距离限制。      |
| uint16_t GetMaxScrollDistance()              | 获取单次滑动手势造成的滑动距离限制。      |
| void SetDragFunc(EasingFunc func)            | 设置拖动动效的函数。              |
| void SetThrowDrag(bool throwDrag)            | 设置结束拖动时，是否产生惯性滑动        |

| 函数                                           | 介绍                                                 |
|----------------------------------------------|----------------------------------------------------|
|                                              | 效。                                                 |
| void SetReboundSize(uint16_t size)           | 设置头结点或尾节点可滑动超出顶部或底部并回弹的距离，达到对应位置后，即可回弹至顶部或底部对齐的状态。 |
| void SetXScrollBarVisible(bool state)        | 设置 X 轴滚动条是否可见。                                     |
| void SetYScrollBarVisible(bool state)        | 设置 Y 轴滚动条是否可见。                                     |
| void SetScrollBarSide(uint8_t side)          | 设置滚动条显示方位。参数参考枚举 ScrollBarSide。                    |
| void SetScrollBarCenter(const Point& center) | 获取滚动条中心点。                                          |
| void SetDirection(uint8_t direction)         | 设置方向。                                              |
| uint8_t GetDirection()                       | 获取方向。                                              |
| void SetElastic(bool isElastic)              | 设置弹簧效果。                                            |
| void SetElasticTime(uint16_t elasticTime)    | 设置回弹时间（多长时间完成回弹）。                                  |

2.3.6 UIScrollView

UIScrollView 是一个容器类组件。主要作用是在屏幕大小有限制时，在控件范围内移动内容，展示更多内容。

继承关系为 UIScrollView→UIAbstractScroll→UIViewGroup。

UIScrollView 内部定义了滚动监听器 OnScrollListener，用来监听滚动状态。具体使用方法可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_ui\\_scroll\\_view/ui\\_test\\_ui\\_scroll\\_view.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_ui_scroll_view/ui_test_ui_scroll_view.cpp)。

表2-14 UIScrollView 核心函数

| 函数                                      | 介绍         |
|-----------------------------------------|------------|
| void SetVerticalScrollState(bool state) | 设置纵向是否可滚动。 |
| bool GetVerticalScrollState() const     | 获取纵向是否可滚动。 |

| 函数                                                               | 介绍                                                                                                                                                                   |
|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void SetHorizontalScrollState(bool state)</code>           | 设置横向是否可滚动。                                                                                                                                                           |
| <code>bool GetHorizontalScrollState() const</code>               | 获取横向是否可滚动。                                                                                                                                                           |
| <code>void ScrollBy(int16_t xDistance, int16_t yDistance)</code> | 设置滚动距离。<br>注意：坐标系基准点是左上为 (0, 0)，如果要向左下偏移，xDistance 和 yDistance 应为负值。 <ul style="list-style-type: none"><li>• xDistance: X 轴上偏移</li><li>• yDistance: Y 轴上偏移</li></ul> |

### 2.3.7 UISwipeView

UISwipeView 类似于 android 中的 ViewPager 的 Tab。继承于 UIAbstractScroll，实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_ui\\_swipe\\_view/ui\\_test\\_ui\\_swipe\\_view.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_ui_swipe_view/ui_test_ui_swipe_view.cpp)，常用函数如表 2-15 所示。

表2-15 UISwipeView 核心函数

| 函数                                                                          | 介绍                                |
|-----------------------------------------------------------------------------|-----------------------------------|
| <code>void SetDirection(uint8_t direction)</code>                           | 设置拖动方向。                           |
| <code>void Add(UIView* view) override</code>                                | 添加视图。                             |
| <code>void Insert(UIView* prevView, UIView* insertView) override</code>     | 向 UISwipeView 中的 prevView 后面插入视图。 |
| <code>virtual void Remove(UIView* view) override</code>                     | 移除视图。                             |
| <code>void SetCurrentPage(uint16_t index, bool needAnimator = false)</code> | 设置当前选项卡的索引（注意：需要保证已添加子元素）。        |
| <code>uint16_t GetCurrentPage() const</code>                                | 获取当前选项卡的索引。                       |
| <code>void SetBlankSize(uint16_t size)</code>                               | 设置空白大小，默认 DEFAULT_BLANK_SIZE。     |



| 函数                                                        | 介绍                            |
|-----------------------------------------------------------|-------------------------------|
| bool OnDragEvent(const DragEvent& event) override         | 拖动事件（内部回调，无需用户主动调用）。          |
| bool OnDragEndEvent(const DragEvent& event) override      | 拖动事件结束（内部回调，无需用户主动调用）。        |
| void SetAlignMode(AlignMode alignMode = ALIGN_CENTER)     | 设置 UISwipeView 子组件的对齐模式。      |
| void SetOnSwipeListener(OnSwipeListener* onSwipeListener) | 设置滑动事件上报时需调用的监听器。             |
| UIView* GetViewByIndex(uint16_t index) const              | 获取当前 View 在 UISwipeView 中的位置。 |
| void SetLoopState(bool loop)                              | 设置子视图循环显示。                    |
| void SetAnimatorTime(uint16_t time)                       | 设置页面动画的时间（ms）。                |

2.3.8 UISwipeViewNested

UISwipeViewNested 继承于 UISwipeView，用于解决滑动冲突。

表2-16 UISwipeViewNested 核心函数

| 函数                                                             | 介绍                  |
|----------------------------------------------------------------|---------------------|
| void SetBoundaryIntercept(uint8_t direction, bool isIntercept) | 设置当滑动超出边界时是否拦截滑动事件。 |
| bool IsBoundaryIntercept(uint8_t direction) const              | 查询当滑动超出边界时是否拦截滑动事件。 |

2.3.9 UICollection

UICollection 的作用是展示列表。继承关系为 UICollection→UIAbstractScroll→ViewGroup，具体使用请参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_ui\\_list/ui\\_test\\_ui\\_list.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_ui_list/ui_test_ui_list.cpp)。

表2-17 UICollection 核心函数

| 函数                                                   | 介绍                                                                                                                                                            |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UICollectionView()                                   | 构造函数，默认创建的是垂直方向上的 list。                                                                                                                                       |
| UICollectionView(uint8_t direction)                  | 构造函数，可以选择垂直或者水平方向。                                                                                                                                            |
| void SetDirection(uint8_t direction)                 | 设置方向。                                                                                                                                                         |
| uint8_t GetDirection() const                         | 获取方向。                                                                                                                                                         |
| void SetAdapter(AbstractAdapter* adapter)            | <p>为 UICollectionView 添加适配器，主要用于传递数据。sample 中使用的是 TextAdapter，还可以自定义适配器，继承于 AbstractAdapter，实现类似于微信朋友圈效果。</p> <p>注意，调用该接口前需要保证 UICollectionView 内部没有残余控件。</p> |
| void SetStartIndex(uint16_t index)                   | 设置当前显示列表从第几项开始展示。该函数仅在 SetAdapter 前调用才能生效。                                                                                                                    |
| void SetLoopState(bool state)                        | 设置是否循环展示。                                                                                                                                                     |
| UIView* GetSelectView()                              | 获取当前选择的项。                                                                                                                                                     |
| void RefreshList()                                   | 刷新此列表。                                                                                                                                                        |
| void SetSelectPosition(uint16_t position)            | <p>设置当前选择的项，可以在选择项上做放大效果操作。</p> <p>注意：该值设置后，需要调用 RefreshList 刷新列表，GetSelectView()才会生效。</p>                                                                    |
| void ScrollBy(int16_t distance)                      | <p>滚动列表。</p> <p>注意：向上滚动应为负值。</p>                                                                                                                              |
| void ScrollTo(uint16_t index)                        | 滚动到第 index 条开始展示。                                                                                                                                             |
| bool OnDragEvent(const DragEvent& event) override    | 处理拖动事件，并上报父类，不需要主动调用。一般不重写，除非自定义组件。                                                                                                                           |
| bool OnDragEndEvent(const DragEvent& event) override | 处理拖动事件，并上报父类，不需要主动调用。除非自定义组件，一般不重写。                                                                                                                           |

| 函数                                                  | 介绍                                  |
|-----------------------------------------------------|-------------------------------------|
| bool OnPressEvent(const PressEvent& event) override | 处理按下事件，并上报父类，不需要主动调用。除非自定义组件，一般不重写。 |

 说明

UList 控件进度条已开启，当前默认设为圆屏（即弧形进度条），若客户用的是方屏，请在启动阶段将屏幕类型设置为方屏（即矩形进度条）：`BaseGfxEngine::GetInstance()->SetScreenShape(RECTANGLE);`  
当前只需开启 scroll bar 即可看到对应的效果：`SetXScrollBarVisible(true);`

2.3.10 UListExt

UListExt 继承于 UList，支持列表缩放，最新版本推荐使用“[2.3.27 UITransformList](#)”组件。

表2-18 UListExt 核心函数

| 函数                                 | 介绍            |
|------------------------------------|---------------|
| void SetScaleFactor(int16_t value) | 设置缩放因子。       |
| void SetItemSpace(uint16_t value)  | 设置条目与条目之间的间隙。 |

2.3.11 UListNested

UListNested 继承于 UList，用于解决滑动冲突。

表2-19 UListNested 核心函数

| 函数                                                             | 介绍                  |
|----------------------------------------------------------------|---------------------|
| void SetBoundaryIntercept(uint8_t direction, bool isIntercept) | 设置当滑动超出边界时是否拦截滑动事件。 |
| bool IsBoundaryIntercept(uint8_t direction) const              | 查询当滑动超出边界时是否拦截滑动事件。 |

2.3.12 UIAnalogClock

UIAnalogClock 是显示时间的组件，继承关系为 UIAnalogClock→UIAbstractClock→ViewGroup。按照时针、分针、秒针的表盘模式来展示。具体创建流程和使用方法请参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_ui\\_analog\\_clock/ui\\_test\\_analog\\_clock.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_ui_analog_clock/ui_test_analog_clock.cpp)。

表2-20 UIAnalogClock 核心函数

| 函数                                                                                                                                   | 介绍                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| void SetHandImage(HandType type, const UllmageView& img, Point position, Point center)                                               | 设置用于绘制时钟指针的图像。 <ul style="list-style-type: none"><li>参数 type：指针类型（时针、分针、秒针）。</li><li>参数 img：指针类型对应的图片。</li><li>参数 position：图像的开始位置。</li><li>参数 center：此时钟指针的旋转中心。</li></ul>                                                                             |
| void SetHandLine(HandType type, Point position, Point center, ColorType color, uint16_t width, uint16_t height, OpacityType opacity) | 设置用于绘制时钟指针的线。 <ul style="list-style-type: none"><li>参数 type：指针类型（时针、分针、秒针）。</li><li>参数 position：线端点靠近旋转中心的位置。</li><li>参数 center：指示此时钟指针的旋转中心。</li><li>参数 color：线条颜色。</li><li>参数 width：线条宽度。</li><li>参数 height：线条长度。</li><li>参数 opacity：线条透明度。</li></ul> |
| Point GetHandRotateCenter(HandType type) const                                                                                       | 获取指定时钟指针的旋转中心。<br>参数 type：指针类型（时针、分针、秒针）。                                                                                                                                                                                                               |
| Point GetHandPosition(HandType type) const                                                                                           | 获取指定时钟指针的位置。<br>参数 type：指针类型（时针、分针、秒针）。                                                                                                                                                                                                                 |

| 函数                                                                             | 介绍                                          |
|--------------------------------------------------------------------------------|---------------------------------------------|
|                                                                                | 针)。                                         |
| uint16_t GetHandInitAngle(HandType type) const                                 | 获取指定时钟指针的初始旋转角度。<br>参数 type：指针类型（时针、分针、秒针）。 |
| uint16_t GetHandCurrentAngle(HandType type) const                              | 获取指定时钟指针的当前旋转角度。<br>参数 type：指针类型（时针、分针、秒针）。 |
| void SetInitTime24Hour(uint8_t hour, uint8_t minute, uint8_t second)           | 以 24 小时格式设置初始时间。                            |
| void SetInitTime12Hour(uint8_t hour, uint8_t minute, uint8_t second, bool am)  | 以 12 小时格式设置初始时间。                            |
| void SetPosition(int16_t x, int16_t y) override                                | 设置组件的 X、Y 位置。                               |
| void SetPosition(int16_t x, int16_t y, int16_t width, int16_t height) override | 设置组件的 X、Y、宽、高。                              |
| void SetWorkMode(WorkMode newMode) override                                    | 设置时钟工作模式。                                   |

2.3.13 UISweepClock

UISweepClock 是显示时间的组件，继承关系为 UISweepClock→UIAbstractClock→ViewGroup。基本功能与 UIAnalogClock 相似，但可额外支持到毫秒级的精度。按照时针、分针、秒针以及每根指针对应的圆圈的表盘模式来展示。

图2-2 UISweepClock 效果



表2-21 UISweepClock 核心函数

| 函数                                                                                                      | 介绍                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void SetHandImage(HandType type, const UIImageView&amp; img, Point position, Point center)</code> | <p>设置用于绘制时钟指针的图像。</p> <ul style="list-style-type: none"><li>• 参数 type：指针类型（时针、分针、秒针）。</li><li>• 参数 img：指针对应的图片。</li><li>• 参数 position：图像的开始位置。</li><li>• 参数 center：此时钟指针的旋转中心。</li></ul> |
| <code>void SetHandCircleImage(HandType type, const Image&amp; img, Point position)</code>               | <p>设置用于绘制时钟时指针圆圈的图像。</p> <ul style="list-style-type: none"><li>• 参数 type：对应指针的类型（时针、分针、秒针）。</li></ul>                                                                                    |

| 函数                                                                                                                                   | 介绍                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                      | <ul style="list-style-type: none"> <li>参数 img: 指针圆圈的图片。</li> <li>参数 position: 图像的开始位置。</li> </ul>                                                                                                                                                                      |
| void SetHandLine(HandType type, Point position, Point center, ColorType color, uint16_t width, uint16_t height, OpacityType opacity) | 设置用于绘制时钟指针的线。 <ul style="list-style-type: none"> <li>参数 type: 指针类型（时针、分针、秒针）。</li> <li>参数 position: 线端点靠近旋转中心的位置。</li> <li>参数 center: 指示此时钟指针的旋转中心。</li> <li>参数 color: 线条颜色。</li> <li>参数 width: 线条宽度。</li> <li>参数 height: 线条长度。</li> <li>参数 opacity: 线条透明度。</li> </ul> |
| Point GetHandRotateCenter(HandType type) const                                                                                       | 获取指定时钟指针的旋转中心。<br>参数 type: 指针类型（时针、分针、秒针）。                                                                                                                                                                                                                             |
| Point GetHandPosition(HandType type) const                                                                                           | 获取指定时钟指针的位置。<br>参数 type: 指针类型（时针、分针、秒针）。                                                                                                                                                                                                                               |
| uint16_t GetHandInitAngle(HandType type) const                                                                                       | 获取指定时钟指针的初始旋转角度。<br>参数 type: 指针类型（时针、分针、秒针）。                                                                                                                                                                                                                           |
| uint16_t GetHandCurrentAngle(HandType type) const                                                                                    | 获取指定时钟指针的当前旋转角度。<br>参数 type: 指针类型（时针、分针、秒针）。                                                                                                                                                                                                                           |
| void SetInitTime24Hour(uint8_t hour, uint8_t minute, uint8_t second)                                                                 | 以 24 小时格式设置初始时间。                                                                                                                                                                                                                                                       |
| void SetInitTime12Hour(uint8_t hour, uint8_t minute, uint8_t second, bool am)                                                        | 以 12 小时格式设置初始时间。                                                                                                                                                                                                                                                       |



| 函数                                                                                   | 介绍                                   |
|--------------------------------------------------------------------------------------|--------------------------------------|
| void SetSweepTime24Hour(uint8_t hour, uint8_t minute, uint16_t millisecond)          | 以 24 小时格式设置当前时间。此函数支持设置毫秒，由此可支持扫秒能力。 |
| void SetSweepTime12Hour(uint8_t hour, uint8_t minute, uint16_t millisecond, bool am) | 以 12 小时格式设置当前时间。此函数支持设置毫秒，由此可支持扫秒能力。 |
| void SetPosition(int16_t x, int16_t y) override                                      | 设置组件的 X、Y 位置。                        |
| void SetPosition(int16_t x, int16_t y, int16_t width, int16_t height) override       | 设置组件的 X、Y、宽、高。                       |
| void SetWorkMode(WorkMode newMode) override                                          | 设置时钟工作模式。                            |

表盘开发约束及应用举例

1. 表盘使用的指针图片请使用垂直指针图片如图 2-3 所示，方便后续位置计算。

图2-3 垂直指针图片



2. 以秒针为例，计算 SetHandImage 对应的 center 及 position 坐标：
- 秒针图片分辨率为 50×315，假定使用 454×454 圆形屏幕，且表盘显示在正中间，如效果图所示。
- a. center 选定为 (25, 211)，center 点位指的是在图片内的位置，与图片在屏幕上的位置无关，后续旋转也是以这个点为中心，大体位置如图 2-4 所示。



图2-4 center 位置示意图



#### 说明

center 的 y 值以实际展示效果为准，可以进行上下调整，此处仅以 sample 为例给出参考。

- b. position 选定为 (202, 16)，这个点代表图片在表盘中的位置，即图片左上角对应的位置。

这个点选取的原因如下：屏幕的中心点为 (227, 227)，想要以屏幕中心点为各指针的旋转中心，则需要将 center 移到 (227, 227) 位置，所以 position 值为  $(227, 227) - (25, 211) = (202, 16)$ 。

3. SetHandCircleImage 的 position 值确定。

sample 中，时、分、秒的 circle image 原图都是 50×50 的图片，图片的中心点为 (25, 25)，所以直接将此 center 移到屏幕正中心 (227, 227) 即可，所以 position 值为  $(227, 227) - (25, 25) = (202, 202)$ 。

#### 说明

上述 sample 指的是随 SDK 版本发的 uikit sample 中 “Sweep Clock” 这个测试用例。

代码位置：

middleware\services\gui\uiokit\sample\uitest\components\ui\_test\_clock\_dial\MainClockView.cpp

## 2.3.14 UIDigitalClock

UIDigitalClock 又称数字时钟，继承关系为 UIDigitalClock→UIAbstractClock。实例请参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-)

[LTS/test/uitest/test\\_digital\\_clock/ui\\_test\\_digital\\_clock.cpp](#)，常用函数参考如表 2-22 所示。

图2-5 UIDigitalClock 效果

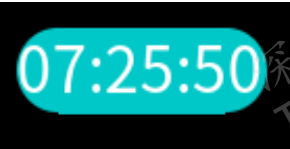


表2-22 UIDigitalClock 核心函数

| 函数                                               | 介绍                                                                                                                                                                                                                                                           |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| void SetFont(const char* name, uint8_t size)     | 设置此数字时钟的动态字体。                                                                                                                                                                                                                                                |
| void SetFontId(uint8_t fontId)                   | 设置此数字时钟的动态字体 ID。                                                                                                                                                                                                                                             |
| void SetColor(ColorType color)                   | 设置展示时间的字体颜色。                                                                                                                                                                                                                                                 |
| void SetDisplayMode(DisplayMode dm)              | 设置此数字时钟的显示模式。<br>参数 DisplayMode 有四个枚举： <ul style="list-style-type: none"><li>• DISPLAY_12_HOUR_NO_SECOND S: 12 小时制，不展示秒。</li><li>• DISPLAY_24_HOUR_NO_SECOND S: 24 小时制，不展示秒。</li><li>• DISPLAY_12_HOUR: 12 小时制。</li><li>• DISPLAY_24_HOUR: 24 小时制。</li></ul> |
| DisplayMode GetDisplayMode() const               | 获取此数字时钟的显示模式。                                                                                                                                                                                                                                                |
| void DisplayLeadingZero(bool displayLeadingZero) | 设置是否在此数字时钟的小时之前添加 '0' 。                                                                                                                                                                                                                                      |
| void SetOpacity(uint8_t opacity)                 | 设置此数字时钟的不透明度。                                                                                                                                                                                                                                                |

2.3.15 UIPicker

UIPicker 是滑动选择器，主要使用场景是选择 Item，使用实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_picker/ui\\_test\\_ui\\_picker.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_picker/ui_test_ui_picker.cpp)。常用函数介绍如表 2-23 所示。

表2-23 UIPickerView 核心函数

| 函数                                                                     | 介绍                    |
|------------------------------------------------------------------------|-----------------------|
| virtual bool SetValues(const char* value[], uint16_t count)            | 使用字符串数组在选取器中设置动态文本数据。 |
| virtual bool SetValues(int16_t start, int16_t end)                     | 使用给定的数字范围设置选取器中的数字数据。 |
| virtual void ClearValues()                                             | 清除选取器中的所有值。           |
| void SetFontId(uint8_t backgroundFontId, uint8_t highlightFontId)      | 设置动态文本的字体 ID。         |
| void SetBackgroundFont(const char* name, uint8_t size)                 | 设置背景文本的字体名称和大小。       |
| void SetHighlightFont(const char* name, uint8_t size)                  | 设置高亮显示文本的字体名称和大小。     |
| void SetTextColor(ColorType backgroundColor, ColorType highlightColor) | 设置文本颜色。               |
| bool SetSelected(uint16_t index)                                       | 设置当前选定项目的索引。          |
| void SetItemHeight(int16_t height)                                     | 设置每个项目的高度。            |
| void SetWidth(int16_t width) override                                  | 设置当前组件的宽度。            |
| void SetHeight(int16_t height) override                                | 设置当前组件的高度。            |
| void SetLoopState(bool state)                                          | 设置当前组件是否可以循环滑动。       |
| void RegisterSelectedListener(SelectedListener* pickerListener)        | 为选定事件注册侦听器。           |
| void SetDirect(UITextLanguageDirect direct)                            | 设置文本方向。               |
| void SetTextFormatter(TextFormatter* formatter)                        | 设置文本格式器。              |
| uint16_t GetSelected()                                                 | 获取当前选中值的下标。           |

2.3.16 UTimePicker

UTimePicker 又称作时间选择器。主要用于时间选择。使用实例可参考[https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_picker/ui\\_test\\_ui\\_picker.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_picker/ui_test_ui_picker.cpp)。常用函数介绍如表 2-24 所示。

表2-24 UTimePicker 核心函数

| 函数                                                                                                                        | 介绍                         |
|---------------------------------------------------------------------------------------------------------------------------|----------------------------|
| bool SetSelected(const char* value)                                                                                       | 设置 UTimePicker 中当前选定的时间。   |
| const char* GetSelectValue()                                                                                              | 获取 UTimePicker 中当前选择的时间。   |
| const char* GetSelectHour() const                                                                                         | 获取 UTimePicker 中当前选定的小时。   |
| const char* GetSelectMinute() const                                                                                       | 获取 UTimePicker 中当前选定的分钟。   |
| const char* GetSelectSecond() const                                                                                       | 获取 UTimePicker 中当前选定的秒钟。   |
| void SetItemHeight(int16_t height)                                                                                        | 设置每一项 Item 的高度。            |
| void EnableSecond(bool state)                                                                                             | 设置是否在时间选择器中启用秒。            |
| void SetTextStyle(uint8_t backgroundFontId, uint8_t highlightFontId, ColorType backgroundColor, ColorType highlightColor) | 设置时间选择器中的文本格式，包括字体 ID 和颜色。 |
| void SetTextColor(ColorType backgroundColor, ColorType highlightColor)                                                    | 设置时间选择器中的文本颜色。             |
| void SetBackgroundFont(const char* name, uint8_t size)                                                                    | 设置背景文本的字体名称和大小。            |
| void SetHighlightFont(const char* name, uint8_t size)                                                                     | 设置高亮显示文本的字体名称和大小。          |
| void RegisterSelectedListener(SelectedListener* timePickerListener)                                                       | 为选定事件注册监听器。                |

2.3.17 UIAxis

UIAxis 是用来表示坐标轴的组件。该组件继承于 UIViewGroup，常以子类 UIXAxis 和 UIYAxis 来展示坐标轴。使用实例可参考

[https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_chart/ui\\_test\\_chart\\_pillar.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_chart/ui_test_chart_pillar.cpp)。

UIAxis

表2-25 UIAxis 核心函数

| 函数                                                        | 介绍          |
|-----------------------------------------------------------|-------------|
| virtual bool SetDataRange(uint16_t min, uint16_t max) = 0 | 设置坐标轴的取值范围。 |
| virtual void SetMarkNum(uint16_t count) = 0               | 设置坐标轴上的刻度数。 |
| void SetLineColor(const ColorType& color)                 | 设置线条颜色。     |

UIXAxis

表2-26 UIXAxis 核心函数

| 函数                                                     | 介绍              |
|--------------------------------------------------------|-----------------|
| void TranslateToPixel(int16_t& value) override         | 将数据转换为像素的 X 坐标。 |
| bool SetDataRange(uint16_t min, uint16_t max) override | 设置 X 轴的取值范围。    |
| void SetMarkNum(uint16_t count) override               | 设置 X 轴上的刻度数。    |

UIYAxis

表2-27 UIYAxis 核心函数

| 函数                                                     | 介绍              |
|--------------------------------------------------------|-----------------|
| void TranslateToPixel(int16_t& value) override         | 将数据转换为像素的 Y 坐标。 |
| bool SetDataRange(uint16_t min, uint16_t max) override | 设置 Y 轴的取值范围。    |
| void SetMarkNum(uint16_t count) override               | 设置 Y 轴上的刻度数。    |

| 函数                                     | 介绍               |
|----------------------------------------|------------------|
| void SetAxisAlign(AxisAlign axisAlign) | 设置 Y 轴位置：左边或者右边。 |

2.3.18 UIChart

UIChart 是图表类，提供添加、删除数据集等功能，以显示图表。常以子类 UIChartPolyline（线条形）和 UIChartPillar（柱状图）形式提供。具体参考实例 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_chart/ui\\_test\\_chart\\_polyline.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_chart/ui_test_chart_polyline.cpp) 和 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_chart/ui\\_test\\_chart\\_pillar.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_chart/ui_test_chart_pillar.cpp)，如下分别介绍其常用函数。

UIChart

表2-28 UIChart 核心函数

| 函数                                                                                         | 介绍               |
|--------------------------------------------------------------------------------------------|------------------|
| void SetHeight(int16_t height) override                                                    | 设置当前组件的高度。       |
| void SetWidth(int16_t width) override                                                      | 设置当前组件的宽度。       |
| virtual bool AddDataSerial(UIChartDataSerial* dataSerial)                                  | 添加数据集。           |
| virtual bool DeleteDataSerial(UIChartDataSerial* dataSerial)                               | 删除数据集。           |
| virtual void ClearDataSerial()                                                             | 清理所有数据。          |
| virtual void RefreshChart() = 0                                                            | 刷新图表并重新绘制脏区域。    |
| UIXAxis& GetXAxis()                                                                        | 获取 X 轴实例。        |
| UIYAxis& GetYAxis()                                                                        | 获取 Y 轴实例。        |
| void EnableReverse(bool enable)                                                            | 启用图表反转。          |
| bool SetGradient(const LinearGradient& linear, const ColorStop* stops, uint16_t stopCount) | 设置 chart 线性梯度渐变。 |

UIChartPillar

图2-6 UIChartPillar 效果

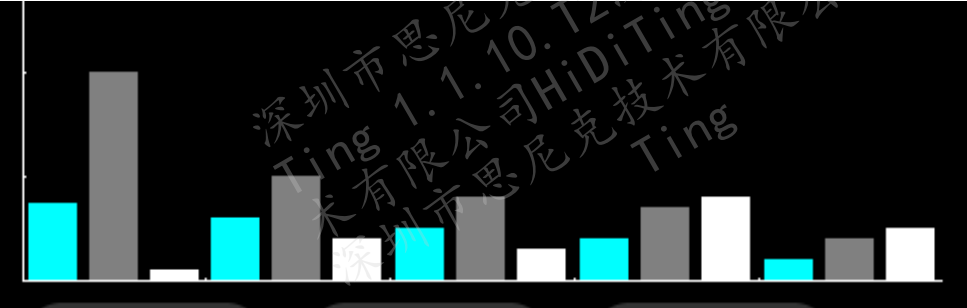


表2-29 UIChartPillar 核心函数

| 函数                           | 介绍             |
|------------------------------|----------------|
| void RefreshChart() override | 刷新柱状图并重新绘制脏区域。 |

UIChartPolyline

图2-7 UIChartPolyline 效果

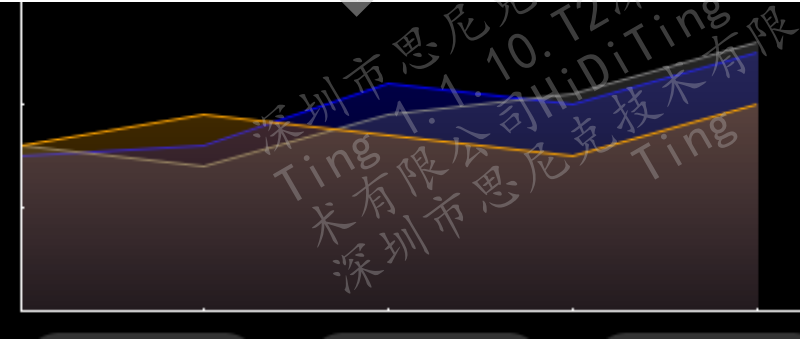


表2-30 UIChartPolyline 核心函数

| 函数                                                      | 介绍                  |
|---------------------------------------------------------|---------------------|
| void RefreshChart() override                            | 刷新折线图并重新绘制脏区域。      |
| void SetGradientOpacity(uint8_t minOpa, uint8_t maxOpa) | 设置填充颜色渐变的不透明度范围。    |
| void SetGradientBottom(uint16_t                         | 设置填充颜色范围的底边与 x 轴之间的 |



| 函数      | 介绍  |
|---------|-----|
| bottom) | 距离。 |

2.3.19 UICrossView

UICrossView 是用于管理主界面的卡片的容器组件，该组件支持的功能如下：

- 支持卡片添加、插入、删除。水平添加卡片数无限制，上下卡片各只支持一个。
- 支持上下卡片的局部和全局模糊。
- 支持监听滑动开始、滑动结束、覆盖开始、覆盖结束事件。
- 支持资源的预加载与卸载，可预加载当前显示卡片的上下左右页面。
- 支持卡片切换动效配置。目前 GUI 提供缩放、翻转、翻页动效，其他动效用户可以自定义。
- 水平方向至少需要插入一个页面，如果需要在某个界面支持上下拉，需要将这个页面的 coverable 设置为 true（参考 UICardPage）。

表2-31 UICrossView 核心函数

| 函数                                                               | 介绍                                                            |
|------------------------------------------------------------------|---------------------------------------------------------------|
| void HorAdd(UICardPage* view)                                    | 添加水平卡片。                                                       |
| void HorInsert(UICardPage* prevView, UICardPage* insertView)     | 插入水平卡片。                                                       |
| void VerAdd(UICardPage* view, VPage page)                        | 添加垂直卡片。                                                       |
| void Remove(UIView* view, uint8_t direction)                     | 删除卡片。                                                         |
| void SetHorCurrentPage(uint16_t index, bool loadAdjacent = true) | 设置当前水平显示卡片，并配置是否加载相邻的两张卡片。                                    |
| void SetSwipeACCLLevel(uint8_t value, uint8_t direction = 0)     | 设置快速滑动后松手的加速权值。<br>direction 为 0 代表水平方向，direction 为 1 代表垂直方向。 |
| uint16_t GetCurrentPageId(uint8_t direction = 0)                 | 获取目标方向的当前卡片索引。                                                |
| UICardPage* GetCurrentPageView(uint8_t direction = 0)            | 获取目标方向的当前卡片对象。                                                |



| 函数                                                                     | 介绍                                                                                     |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 0)                                                                     |                                                                                        |
| void SetAnimatorTime(uint16_t time)                                    | 设置松手动画时间。                                                                              |
| void SetLoopState(bool loop)                                           | 设置水平卡片循环模式。                                                                            |
| void EnableScreenCap(bool enable)                                      | 设置切换卡片时，是否使能截图模式。                                                                      |
| bool IsEnableScreenCap(void)                                           | 判断当前切换卡片模式。                                                                            |
| void RegisterSwipeCallback(CardSwipe* callback)                        | 注册一个拖拽动效回调函数。                                                                          |
| void SetOnSwipeListener(UISwipeView::OnSwipeListener* onSwipeListener) | 注册一个滑动动画结束监听器。                                                                         |
| UIViewType GetViewType()                                               | 获取控件类型。                                                                                |
| void SetBlurRadius(float radius)                                       | 设置上下界面的高斯模糊半径。<br><br>参数 radius：模糊半径。内部调参值，并非指半径内的区域才会模糊，此值决定不同程度的高斯模糊效果，合法值为 [0, 32]。 |

## UICardPage

表2-32 UICardPage 核心函数

| 函数                         | 介绍                                                                                         |
|----------------------------|--------------------------------------------------------------------------------------------|
| virtual void PreLoad(void) | 卡片资源加载回调函数：在水平方向卡片切换 (SetHorCurrentPage) 或者滑动结束时，如果当前卡片未加载并且位于屏幕中间或者紧邻的左/右任一位置时，则会调用该函数。   |
| virtual void UnLoad(void)  | 卡片资源卸载回调函数：在水平方向卡片切换 (SetHorCurrentPage) 或者滑动结束时，如果当前卡片已加载并且远离屏幕（不在屏幕中间以及紧邻左右位置）时，则会调用该函数。 |
| virtual void               | 卡片滑动开始回调函数：在水平滑动开始时，当前水平                                                                   |

| 函数                                      | 介绍                                                                                                                               |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| ScrollBegin(bool isActive)              | <p>方向显示的卡片以及左右相邻卡片将调用该函数。向上滑动时，下界面调用该函数；向下滑动时，上界面调用该函数。</p> <p>isActive 参数：无论是水平滑动，还是垂直滑动，滑动开始显示的卡片为 true，其它调用该函数的卡片为 false。</p> |
| virtual void ScrollEnd(bool isActive)   | <p>卡片滑动结束回调函数：在水平/垂直滑动动画结束时，滑动开始时已经调用了 ScrollBegin 函数的卡片，都会接收到结束事件。</p> <p>isActive 参数：滑动结束时当前显示的卡片为 true，其他为 false。</p>         |
| virtual void CoverBegin(bool isCovered) | <p>卡片覆盖开始回调函数：在垂直滑动开始时，水平方向当前显示的卡片调用该函数。</p> <p>isCovered 参数：水平卡片被覆盖时为 true，否则为 false。</p>                                       |
| virtual void CoverEnd(bool isCovered)   | <p>卡片覆盖结束回调函数：在垂直滑动动画结束时，水平方向当前显示的水平卡片调用该函数。</p> <p>isCovered 参数：水平界面被覆盖时为 true，否则为 false。</p>                                   |
| virtual void OnActive(void)             | <p>水平方向卡片切换（SetHorCurrentPage）回调函数：当水平方向卡片切换后，当前显示的卡片调用该函数。</p> <p>Slice 状态变化回调函数：当 Slice 切换到 Resume 状态时，水平方向显示的卡片调用该函数。</p>     |
| virtual void OnPause(void)              | <p>Slice 状态变化回调函数：当 Slice 切换到 Pause 状态时，水平方向显示的卡片调用该函数。</p>                                                                      |
| virtual void OnInactive(void)           | <p>水平方向卡片切换（SetHorCurrentPage）回调函数：当水平方向显示卡片切换后，切换前显示的卡片调用该函数。</p>                                                               |
| virtual void OnCovered(void)            | <p>垂直方向卡片切换（SetVerCurrentPage）回调函数：在垂直方向卡片切换结束后，水平方向显示的卡片由被覆盖切换到可见状态时，调用该函数。</p>                                                 |

| 函数                                | 介绍                                                                        |
|-----------------------------------|---------------------------------------------------------------------------|
| virtual void OnUncovered(void)    | 垂直方向卡片切换（SetVerCurrentPage）回调函数：在垂直方向卡片切换结束后，水平方向显示的卡片由可见切换到被覆盖状态时，调用该函数。 |
| void SetCoverable(bool coverable) | 设置当前水平卡片是否能被垂直卡片覆盖。                                                       |
| bool IsCoverable(void)            | 获取当前卡片覆盖状态。                                                               |
| UIViewType GetViewType()          | 获取控件类型。                                                                   |

表2-33 CardSwipe 核心函数

| 函数                                                                                      | 介绍                            |
|-----------------------------------------------------------------------------------------|-------------------------------|
| virtual void CardSwipeAlg(UICardPage* leftCard, UICardPage* rightCard, int16_t xOffset) | 拖动卡片时的滑动算法，输入为 UICardPage 类型。 |
| virtual void CardSwipeAlg(UImageView* leftCard, UImageView* rightCard, int16_t xOffset) | 拖动卡片时的滑动算法，输入为 UImageView 类型。 |

2.3.20 UIRollerView

UIRollerView 将图片按照滚筒的形式排布并支持滚动交互，目前仅支持水平方向。

图2-8 UIRollerView Display 模式效果



表2-34 UIRollerView 核心函数

| 函数                                                   | 介绍                          |
|------------------------------------------------------|-----------------------------|
| void SetItemSize(uint16_t width, uint16_t height)    | 设置滚筒元素的原始宽高。                |
| void SetItemPadding(uint16_t padding)                | 设置滚筒元素的间隔距离。                |
| void SetMirrorOpacity(uint8_t opa)                   | 设置滚筒元素镜像图的透明度。              |
| void SetSensitivityFactor(float sensitivity)         | 设置滚筒灵敏度。                    |
| float GetSensitivityFactor()                         | 获取滚筒灵敏度。                    |
| bool AddItem(std::string name, const ImageInfo* src) | 添加滚筒元素。                     |
| void RemoveItem(std::string name)                    | 移除滚筒元素。                     |
| void ClearAll()                                      | 清除所有滚筒元素。                   |
| std::string GetCurrentItemName()                     | 获取当前对齐的元素名称。                |
| UIViewGroup* GetCurrentItem()                        | 获取当前对齐的元素组，包含原始元素以及该元素的镜像图。 |

| 函数                                                                                    | 介绍                                                                                                                                                                                       |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void SetCurrentItem(std::string name, bool enableAnimator = false)</code>       | 设置当前对齐的元素，并配置是否进行切换动画。                                                                                                                                                                   |
| <code>void SetMode(RollerViewMode mode, bool enableAnimator = false)</code>           | 设置元素陈列模式，并配置是否进行切换动画。<br>陈列模式分为以下三种： <ul style="list-style-type: none"><li>• DISPLAY：将元素按照圆筒的形式排布。</li><li>• COVER：将当前对齐的元素平铺满控件。</li><li>• TRANSITION：特指上述两个模式切换过程中的状态，仅内部使用。</li></ul> |
| <code>RollerViewMode GetMode()</code>                                                 | 获取元素陈列模式。                                                                                                                                                                                |
| <code>void SetOnModeChangeListener(OnModeChangeListener* onModeChangeListener)</code> | 设置元素陈列模式变化监听器。                                                                                                                                                                           |
| <code>OnModeChangeListener*&amp; GetOnModeChangeListener()</code>                     | 获取元素陈列模式变化监听器。                                                                                                                                                                           |
| <code>void SetOnScrollListener(OnScrollListener* onScrollListener)</code>             | 设置滚动监听器。                                                                                                                                                                                 |
| <code>OnScrollListener*&amp; GetOnScrollListener()</code>                             | 获取滚动监听器。                                                                                                                                                                                 |
| <code>void SetCamDistance(int16_t distance)</code>                                    | 设置相机距离。                                                                                                                                                                                  |
| <code>void SetCamY(int16_t y)</code>                                                  | 设置相机 Y 坐标。                                                                                                                                                                               |

2.3.21 UICoverflowView

UICoverflowView 用于添加图片，使图片之间存在覆盖流效果。仅支持水平方向，效果如图 2-9 所示。

## 2 kaleidoscope



### 表2-35 UICoverflowView 核心函数

| 函数                                                | 介绍                                |
|---------------------------------------------------|-----------------------------------|
| void Add(const char* id, const ImageInfo* src);   | 添加图片资源，以及对应的 ID 值，ID 为了用例进行删除时使用。 |
| void Remove(const char* id);                      | 按照添加时的 ID 进行移除。                   |
| void SetImgSize(uint16_t width, uint16_t height); | 设置图片的大小。                          |
| void SetImgPadding(uint16_t padding);             | 设置图片之间的间距。                        |
| void SetRotateAngle(float angle);                 | 设左右图片的最大旋转角度。                     |
| void                                              | 设计滑动监听器，能监听该控件的滑动                 |



| 函数                                                                     | 介绍               |
|------------------------------------------------------------------------|------------------|
| SetOnCoverflowListener(OnCoverflowScrollListener* onCoverflowListener) | 事件，实现一些跟随滑动进行操作。 |

说明

建议使用 UICoverflowView2 控件，具体请参见“2.3.22 UICoverflowView2”。  
UICoverflowView2 能够覆盖 UICoverflowView 功能，而且适用于更多的 UI 场景。

2.3.22 UICoverflowView2

适用于 UICoverflowView 子控件的内容为多种图片场景。实例如图 2-10 所示。

图2-10 UICoverflowView 实例显示效果



表2-36 UICoverflowView2 核心函数

| 函数                                                                          | 介绍                                |
|-----------------------------------------------------------------------------|-----------------------------------|
| void Add(UiView* view);                                                     | 添加一个 UITransformGroup 类型的子控件。     |
| void Insert(UiView* prevView, UiView* insertView)                           | 插入一个 UITransformGroup 类型的子控件。     |
| void Remove(UiView* view);                                                  | 删除一个已添加的子控件。                      |
| void SetPagePadding(uint16_t padding)                                       | 设置每项之间的间距。                        |
| void SetChildMaxRotateAngle(float angle)                                    | 设置项的最大旋转角度。                       |
| void SetOnCoverflowListener(OnCoverflowScrollListener* onCoverflowListener) | 设计滑动监听器，能监听该控件的滑动事件，实现一些跟随滑动进行操作。 |

须知

不要添加 UILabel UILabelExt 到 UITransformGroup 类型的子控件中，会导致位置显示异常。

2.3.23 UISphereView

该控件可以实现 2.5D 球形菜单界面。具体如图 2-11 所示。



图2-11 球形效果

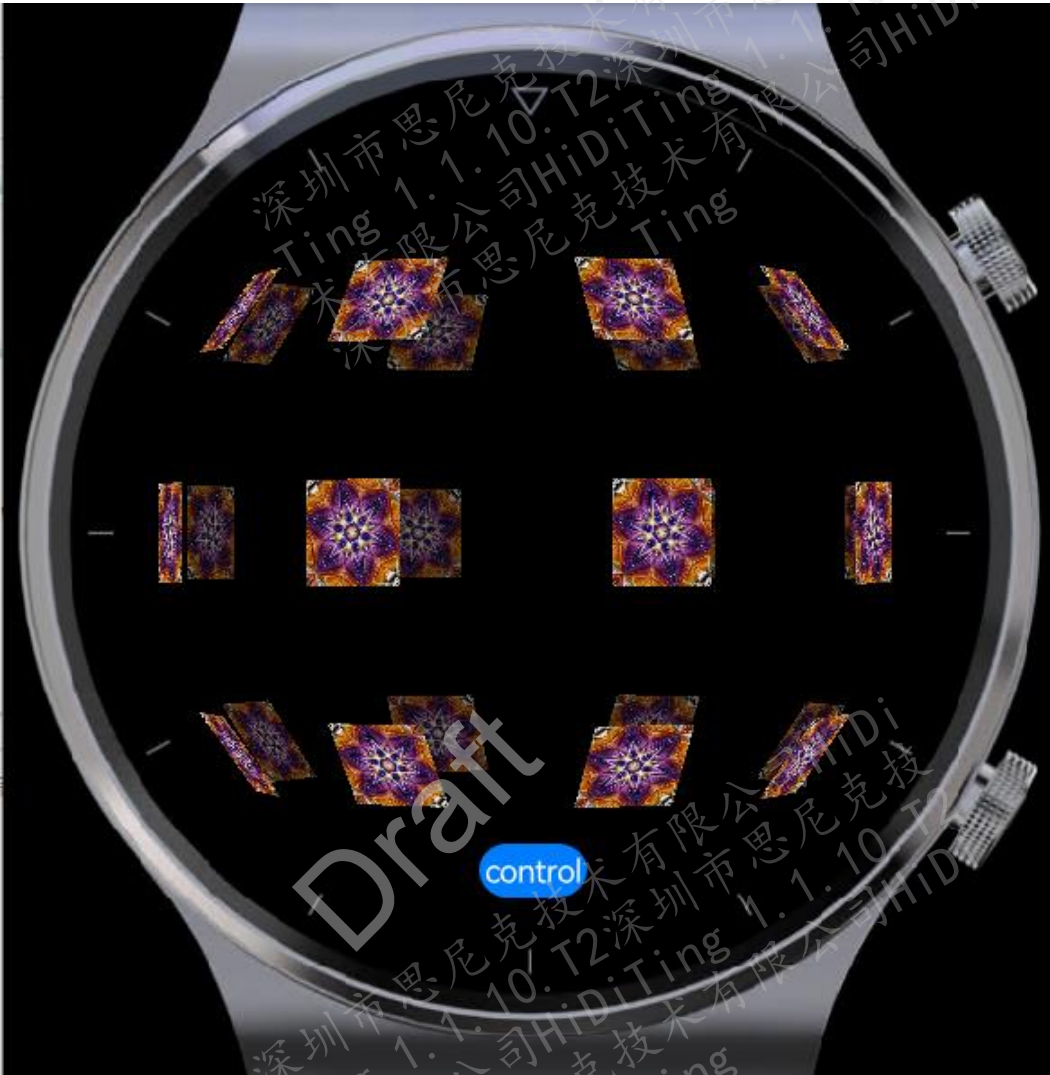


表2-37 UISphereView 核心参数

| 函数                                                          | 介绍                               |
|-------------------------------------------------------------|----------------------------------|
| UISphereView(uint8_t numberOfRows);                         | 构造函数，需要输入球形的一行最多显示数量，超过则换行。      |
| void Add(UIView* view) override;                            | 添加子控件，只支持 UITransformGroup 类型控件。 |
| void Insert(UIView* prevView, UIView* insertView) override; | 插入子控件，只支持 UITransformGroup 类型控件。 |
| void Remove(UIView* view) override;                         | 删除子控件，需要输入 add 或者 insert         |

| 函数                                            | 介绍             |
|-----------------------------------------------|----------------|
|                                               | 的控件对象。         |
| void SetNumberOfRow(uint8_t numberOfRow);     | 设置球形每行最多显示控件数。 |
| uint8_t GetNumberOfRow();                     | 获取当前球形每行的数量。   |
| void UpdateItemAngle(float angle);            | 旋转当前球行。        |
| void SetSensitivityFactor(float sensitivity); | 设置滚筒灵敏度。       |
| float GetSensitivityFactor();                 | 获取滚筒灵敏度。       |

须知

球形控件的宽高需要设置一致。

以下是球型菜单控件的实例代码：

```
UISphereView sphereView = new UISphereView(8); // 创建一个每行显示8个控件的球形菜单
sphereView->SetPosition(0, 0, Screen::GetInstance().GetWidth(),
Screen::GetInstance().GetHeight()); // 设置球形菜单大小为屏幕大小
sphereView->SetThrowDrag(true);
sphereView->RequestFocus(); // 申请焦点，支持表冠滚动。

for (uint8_t i = 0; i < 24; i++) { // 添加24的子控件到球形菜单

    UITransformGroup* group = new UITransformGroup(); // 创建UITransformGroup控件。
    group->SetPosition(0, 0, 60, 60);
    group->SetStyle(STYLE_BACKGROUND_OPA, OPA_TRANSPARENT);
    UIImageView* image = new UIImageView(); // 创建需要显示的图片控件。
    image->SetPosition(0, 0, 60, 60);
    image->SetAutoEnable(false);
    image->SetResizeMode(UIImageView::ImageResizeMode::FILL);
    ImageInfo* info = ImageCacheManager::GetInstance().LoadSingleRes("[图片资源路径]");
    image->SetSrc(info);
    group->Add(image);
    sphereView->Add(group);
}
```

2.3.24 UISlipflowView

图2-12 滑动流效果



表2-38 UISlipflowView 核心函数

| 函数                                                      | 介绍                                 |
|---------------------------------------------------------|------------------------------------|
| void Add(UIView* view);                                 | 添加子控件。                             |
| void Remove(UIView* view);                              | 删除子控件。                             |
| void SetPageSize(uint16_t width, uint16_t height);      | 设置子页的大小。                           |
| void SetCurrentPage(uint16_t index, bool needAnimator); | 设置当前页。                             |
| void SetAnimatorTime(uint16_t time);                    | 设置页面切换的动画时间。                       |
| void SetMoveCoefficient(uint16_t coefficient);          | 设置当前页面及右侧页面相对于其他页面的移动系数，及当前页面及右侧页面 |

| 函数                                                                              | 介绍                                |
|---------------------------------------------------------------------------------|-----------------------------------|
|                                                                                 | 的移动速度是其他页的 coefficient 倍。         |
| void<br>SetOnSlipflowListener(OnSlipflowScroll<br>Listener* onSlipflowListener) | 设计滑动监听器，能监听该控件的滑动事件，实现一些跟随滑动进行操作。 |

2.3.25 UllcosahedronView

UllcosahedronView 二十面体容器控件，可以实现足球效果。如图 2-13 所示。

图2-13 足球效果



该控件实际是一个由 20 个三角面组成的容器控件，所以如果可以实现足球效果，对传入的各个面的图片资源存在限制。该图片是一个背景透明的正六边的图片，如图 2-14 所示，方向也需要特别注意，要与实例一致。



图2-14 足球面的图片要求



表2-39 UIIcosahedronView 核心函数

| 函数                                                                                                  | 介绍                              |
|-----------------------------------------------------------------------------------------------------|---------------------------------|
| void SetSideLength(uint16_t sideLen);                                                               | 设置二十面体的棱长，通过该接口可以控制球体的大小。       |
| void Add(UIView* view) override;                                                                    | 添加 UIImageView 控件。              |
| void RefreshIcosahedron();                                                                          | 刷新二十面体容器，在更新棱长或旋转的角度后使用。        |
| bool RotatePlanes(int16_t angle, const Vector3<float>& pivotStart, const Vector3<float>& pivotEnd); | 旋转二十面体容器。                       |
| void SetDefaultImage(ImageInfo* src)                                                                | 设置默认图片。在添加子控件不满二十面体时，使用该图片进行显示。 |
| void SetLuminanceFactor(float value)                                                                | 设置惯性性的补偿系数。系数越大，惯性越大。           |

2.3.26 UITransformGroup

UITransformGroup 容器支持整体变换。对容器设置的变换将应用到子控件上。支持子控件为 UIImageView、UILabel、UILabelExt。

须知

因为 UILabel UILabelExt 控件不支持透视变换，如果 UITransformGroup 添加了以上控件，请不要进行透视变换。

表2-40 UITransformGroup 核心函数

| 函数                                                       | 介绍                                           |
|----------------------------------------------------------|----------------------------------------------|
| virtual void Add(UIView* view) override                  | 添加子控件。                                       |
| void Insert(UIView* prevView, UIView* insertView)        | 插入子控件。                                       |
| void Remove(UIView* view) override                       | 移除子控件。                                       |
| void RemoveAll() override                                | 移除所有子控件。                                     |
| void TransformChild(UIView* view, TransformMap transmap) | 对内部特定子控件配置变换。内部计算时会根据实时位置调用 SetTransMapRect。 |
| void SetStyle(uint8_t key, int64_t value) override       | 对内部所有控件（一级子控件）和本身设置 style。                   |

2.3.27 UITransformList

UITransformList 容器继承于 UICollection, 使用需要配置 Adapter。与 UICollectionView 有所不同，支持自定义滚动时各条目的变换动画。

表2-41 UITransformList 核心函数

| 函数                                                                  | 介绍                        |
|---------------------------------------------------------------------|---------------------------|
| void SetItemSpace(uint16_t value)                                   | 设置条目与条目之间的间隙。             |
| using TransformFunc = bool (*)(UITransformList* list, UIView* item) | 滑动时，回调函数类型。               |
| void SetItemTransformFunc(TransformFunc func)                       | 设置滑动时的回调函数，栏目将按照回调事件进行变换。 |

2.3.28 UIHexagonList

UIHexagonList 是呈蜂窝状排布子节点的容器控件，常用于陈列菜单界面的各应用图标。该控件仅支持添加 UIImageView 和 UITransformGroup。

表2-42 UIHexagonList 核心函数

| 函数                                                       | 介绍                                                                                                                                                                                                        |
|----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| void SetImageDistance(uint16_t distance)                 | 设置图标间距。                                                                                                                                                                                                   |
| void SetImageSizeInCenter(uint16_t size)                 | 设置图标居中显示时的尺寸。                                                                                                                                                                                             |
| void SetScaleFactorByDistance(float factor)              | 设置图标缩放系数。                                                                                                                                                                                                 |
| float GetScaleFactorByDistance()                         | 获取图标缩放系数。                                                                                                                                                                                                 |
| void SetScaleThesholdOfZeroFactor(float threshold)       | 设置图标缩放阈值。                                                                                                                                                                                                 |
| float GetScaleThesholdOfZeroFactor()                     | 获取图标缩放阈值。                                                                                                                                                                                                 |
| void Add(UiView* view)                                   | 添加控件，支持控件类型为 UIImageView 和 UITransformGroup。                                                                                                                                                              |
| void Insert(UiView* prevView, UiView* insertView)        | 插入控件，支持控件类型为 UIImageView 和 UITransformGroup。                                                                                                                                                              |
| void LayoutChildren(bool needInvalidate = false)         | 按照蜂窝状布局控件。                                                                                                                                                                                                |
| void MoveChildByOffset(int16_t offsetX, int16_t offsetY) | 移动控件。                                                                                                                                                                                                     |
| void Scale(float scale, uint8_t centerIndex)             | 以索引值为 centerIndex 的控件为中心点缩放指定倍数。                                                                                                                                                                          |
| float GetGlobalScale()                                   | 获取全局缩放倍数。                                                                                                                                                                                                 |
| UiView* GetViewByIndex(uint8_t index)                    | 获取指定索引值关联的控件。                                                                                                                                                                                             |
| void SetReboundMode(ReboundMode mode)                    | 设置回弹模式，支持模式为： <ul style="list-style-type: none"> <li>NEAREST_IMG_ON_SCR_CENTER：离屏幕中心点最近的图标将回弹至屏幕中间。</li> <li>CENTER_IMG_ON_SCR_CENTER：初始位于中心的图标将回弹至屏幕中间。</li> <li>NORMAL：与 UIScrollView 相同的模式。</li> </ul> |

| 函数                                                    | 介绍              |
|-------------------------------------------------------|-----------------|
| int8_t GetFocusedImgIndex()                           | 获取位于屏幕中间的图标索引。  |
| Vector2<int16_t><br>GetCurrentPosition(uint8_t index) | 获取指定索引值关联的控件位置。 |

### 2.3.29 UICustomHexagonsList

UICustomHexagonList 继承自 UIHexagonList，额外支持表冠响应，可通过 SetOnRotateListener 实现表冠转动监听。

表2-43 UICustomHexagonList 核心函数

| 函数                                                                                                               | 介绍                                         |
|------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| void StartRotateAnimator(uint8_t center, int16_t xDistance, int16_t yDistance, float startScale, float endScale) | 以索引值为 center 的控件为中心，启动一段动画完成指定距离偏移和指定倍数缩放。 |
| void StopRotateAnimator()                                                                                        | 停止动画。                                      |

### 2.3.30 UIWaterfallList

UIWaterfallList 是以每行 3 列或 4 列交叉排布子控件的容器控件，常用于陈列菜单界面的各应用图标。

表2-44 UIWaterfallList 核心函数

| 函数                                                 | 介绍            |
|----------------------------------------------------|---------------|
| void SetImageDistance(uint16_t distance)           | 设置图标的间距。      |
| void SetImageSizeInCenter(uint16_t size)           | 设置图标居中显示时的尺寸。 |
| void SetScaleFactorByDistance(float factor)        | 设置图标缩放系数。     |
| float GetScaleFactorByDistance()                   | 获取图标缩放系数。     |
| void SetScaleThesholdOfZeroFactor(float threshold) | 设置图标缩放阈值。     |



| 函数                                                       | 介绍                                           |
|----------------------------------------------------------|----------------------------------------------|
| float GetScaleThesholdOfZeroFactor()                     | 获取图标缩放阈值。                                    |
| void Add(UIView* view)                                   | 添加控件，支持控件类型为 UIImageView 和 UITransformGroup。 |
| void Insert(UIView* prevView, UIView* insertView)        | 插入控件，支持控件类型为 UIImageView 和 UITransformGroup。 |
| void LayoutChildren(bool needInvalidate = false)         | 按照瀑布状布局控件。                                   |
| void MoveChildByOffset(int16_t offsetX, int16_t offsetY) | 移动控件。                                        |
| void Scale(float scale, uint8_t centerIndex)             | 以索引值为 centerIndex 的控件为中心点缩放指定倍数。             |
| float GetGlobalScale()                                   | 获取全局缩放倍数。                                    |
| UIView* GetViewByIndex(uint8_t index)                    | 获取指定索引值关联的控件。                                |
| int8_t GetFocusedImgIndex()                              | 获取位于屏幕中间的图标索引。                               |
| Vector2<int16_t> GetCurrentPosition(uint8_t index)       | 获取指定索引值关联的控件位置。                              |
| int16_t GetInitRow(uint8_t index)                        | 获取指定索引值关联的控件初始行数。                            |
| int16_t GetInitCol(uint8_t index)                        | 获取指定索引值关联的控件初始列数。                            |
| Vector2<int16_t> GetInitPosition(uint8_t index)          | 获取指定索引值关联的控件初始位置。                            |
| void EnterMove(float curTime, float totalTime)           | 启动入场动画。                                      |
| Vector2<int16_t> GetReboundDistance()                    | 获取回弹距离。                                      |

### 2.3.31 UIParticleView

UIParticleView 是用于生成粒子效果的容器控件。UIParticleView 通过 ParticleCell 来定义一组粒子的样式和行为，同时也提供适用于所有粒子的全局属性配置。通过调整属性，可以创造丰富多样的粒子效果。

图2-15 粒子效果

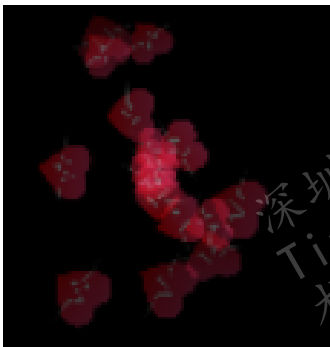


表2-45 UIParticleView 核心函数

| 函数                                         | 介绍                                               |
|--------------------------------------------|--------------------------------------------------|
| UIParticleView()                           | 粒子容器控件构造函数。粒子动画将不间断运行。                           |
| UIParticleView(uint32_t time, bool repeat) | 粒子容器控件构造函数。可指定粒子动画运行周期以及是否重复运行。                  |
| virtual ~UIParticleView() override         | 粒子容器控件析构函数。停止粒子动画，并销毁所有 ParticleCell 及粒子。        |
| ParticleCell* CreateParticleCell()         | 创建并返回 ParticleCell。ParticleCell 用于定义一组粒子的样式和行为。  |
| void SetBirthRateRatio(float ratio)        | 设置粒子生产率系数，将与 ParticleCell 定义的粒子生产率相乘，默认值为 1.0。   |
| void SetLifetimeRatio(float ratio)         | 设置粒子生命周期系数，将与 ParticleCell 定义的粒子生命周期相乘，默认值为 1.0。 |
| void SetScaleRatio(float ratio)            | 设置粒子缩放系数，将与 ParticleCell 定义的粒子缩放倍数相乘，默认值为 1.0。   |
| void SetVelocityRatio(float ratio)         | 设置粒子速度系数，将与 ParticleCell 定义的粒子速度相乘，默认值为 1.0。     |
| float GetBirthRateRatio()                  | 获取粒子生产率系数。                                       |
| float GetLifetimeRatio()                   | 获取粒子生命周期系数。                                      |

| 函数                          | 介绍                             |
|-----------------------------|--------------------------------|
| float GetScaleRatio()       | 获取粒子缩放系数。                      |
| float GetVelocityRatio()    | 获取粒子速度系数。                      |
| void SetTime(uint32_t time) | 设置粒子动画运行周期。                    |
| uint32_t GetTime()          | 获取粒子动画运行周期。                    |
| bool IsRepeat()             | 获取粒子动画是否重复运行。                  |
| void Start()                | 启动粒子动画。                        |
| void Pause()                | 暂停粒子动画。                        |
| void Resume()               | 继续粒子动画。                        |
| void Stop()                 | 停止粒子动画。将销毁所有 ParticleCell 及粒子。 |
| uint8_t GetState()          | 获取粒子动画状态。                      |
| void ClearParticles()       | 销毁所有 ParticleCell 及粒子。         |

表2-46 ParticleCell 核心函数

| 函数                                               | 介绍                    |
|--------------------------------------------------|-----------------------|
| void SetSrc(const ImageInfo* src)                | 设置粒子图源。               |
| void SetSize(int16_t width, int16_t height)      | 设置粒子初始宽高。             |
| void SetSizeRange(int16_t range)                 | 设置粒子初始宽高上下浮动范围。       |
| void SetPos(int16_t x, int16_t y)                | 设置粒子初始位置。             |
| void SetPosRange(int16_t xRange, int16_t yRange) | 设置粒子初始位置上下浮动范围。       |
| void SetLifetime(int16_t lifetime)               | 设置粒子生命周期。             |
| void SetLifetimeRange(int16_t range)             | 设置粒子生命周期上下浮动范围。       |
| void SetBirthRate(int16_t birthRate)             | 设置粒子生产率：每个动画周期生产多少粒子。 |

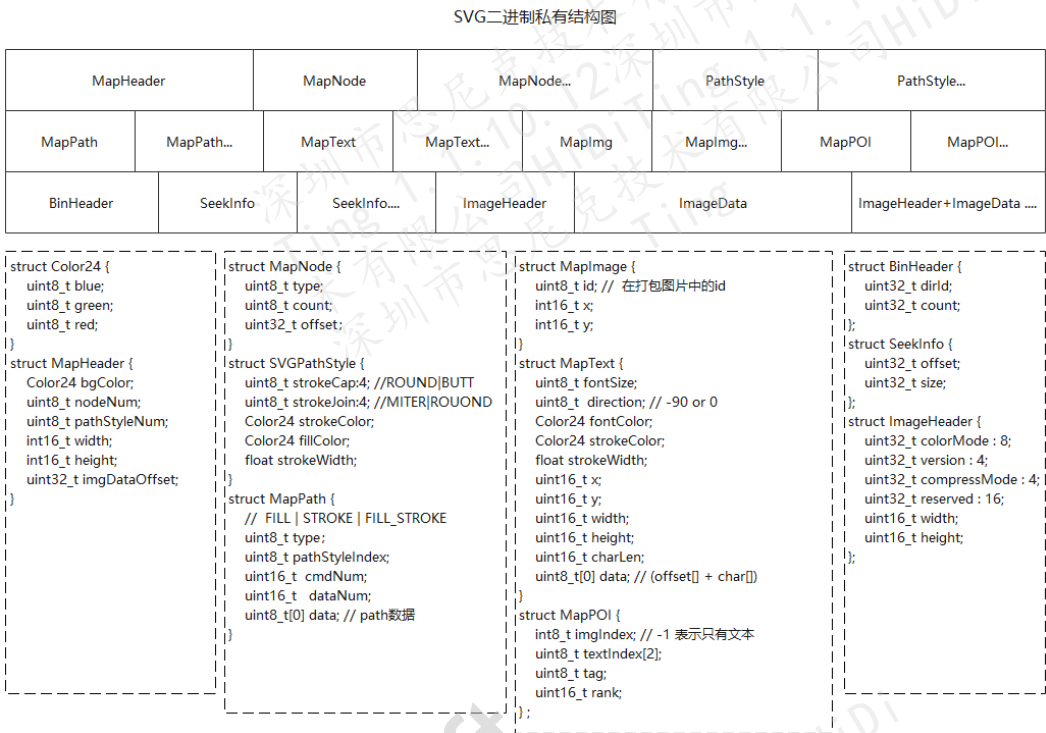
| 函数                                                                                                  | 介绍                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void SetVelocity(float velocity)</code>                                                       | 设置粒子速度。                                                                                                                                         |
| <code>void SetVelocityRange(float range)</code>                                                     | 设置粒子速度上下浮动范围。                                                                                                                                   |
| <code>void SetAcceleration(float x, float y)</code>                                                 | 设置粒子加速度：每个动画周期 x 和 y 方向的速度变化。                                                                                                                   |
| <code>void SetEmissionAngle(float angle)</code>                                                     | 设置粒子发射角度，有效范围为[0, 360]。定义 3 点钟方向为 0 度，顺时针增加度数。                                                                                                  |
| <code>void SetEmissionRange(float range)</code>                                                     | 设置粒子发射角度上下浮动范围。                                                                                                                                 |
| <code>void SetScale(float scaleX, float scaleY)</code>                                              | 设置粒子最终缩放倍数。                                                                                                                                     |
| <code>void SetScaleRange(float range)</code>                                                        | 设置粒子最终缩放倍数上下浮动范围。                                                                                                                               |
| <code>void SetScaleFunc(EasingFunc func)</code>                                                     | 设置粒子缩放变化的数学函数。                                                                                                                                  |
| <code>void SetAlpha(int16_t startAlpha, int16_t finalAlpha)</code>                                  | 设置粒子起始和最终透明度，有效范围为[0, 255]。                                                                                                                     |
| <code>void SetAlphaRange(int16_t range)</code>                                                      | 设置粒子起始和最终透明度上下浮动范围。                                                                                                                             |
| <code>void SetAlphaFunc(EasingFunc func)</code>                                                     | 设置粒子透明度变化的数学函数。                                                                                                                                 |
| <code>void SetSpin(float spin, ParticleCell::SpinMode mode = ParticleCell::SpinMode::CENTER)</code> | 设置粒子周期性旋转度数和旋转模式，默认为绕中线旋转。<br><br>SpinMode::CENTER：绕中线旋转。<br><br>SpinMode::DIAGONAL_LR：绕左上至右下的对角线旋转。<br><br>SpinMode::DIAGONAL_RL：绕右上至左下的对角线旋转。 |
| <code>void SetSpinRange(float range)</code>                                                         | 设置粒子周期性旋转度数上下浮动范围。                                                                                                                              |
| <code>bool HasParticles()</code>                                                                    | 获取当前 ParticleCell 是否包含粒子。                                                                                                                       |
| <code>void SetTotalPeriod(int16_t period)</code>                                                    | 设置 ParticleCell 活跃周期。仅在活跃周                                                                                                                      |

| 函数                         | 介绍                                                      |
|----------------------------|---------------------------------------------------------|
|                            | 期内的 ParticleCell 可以生产粒子。<br>PERIOD_FOREVER：代表永远活跃的特殊周期。 |
| int16_t GetTotalPeriod()   | 获取 ParticleCell 活跃周期。                                   |
| int16_t GetCurrentPeriod() | 获取 ParticleCell 当前周期。                                   |
| const ImageInfo* GetSrc()  | 获取粒子图源。                                                 |
| void ClearParticles()      | 销毁 ParticleCell 所生产的所有粒子。                               |

2.3.32 UIMapView

UIMapView 为定制化的地图显示控件。该控件只针对私有数据进行显示。私有格式如图 2-16 所示。

图2-16 私有格式结构图



简化结构体，将相同类型的MapNode集中到一起，减少fseek

uint16\_t dirId  
脚本中支持指定起始DirId;  
系统预置： 指定起始ID  
客户新增： 指定起始ID  
0xfe0 ~: 预留，不给外部使用  
表盘： 0xfef 一次性只能加载一个表盘  
地图： 0xff0 -> 0xffff // 用于地图svg的dirId，一次性最多加载16个，文件中默认为0xff0,加载到内存后，循环增长，保证唯一性

地图显示效果如图 2-17 所示。

图2-17 显示效果

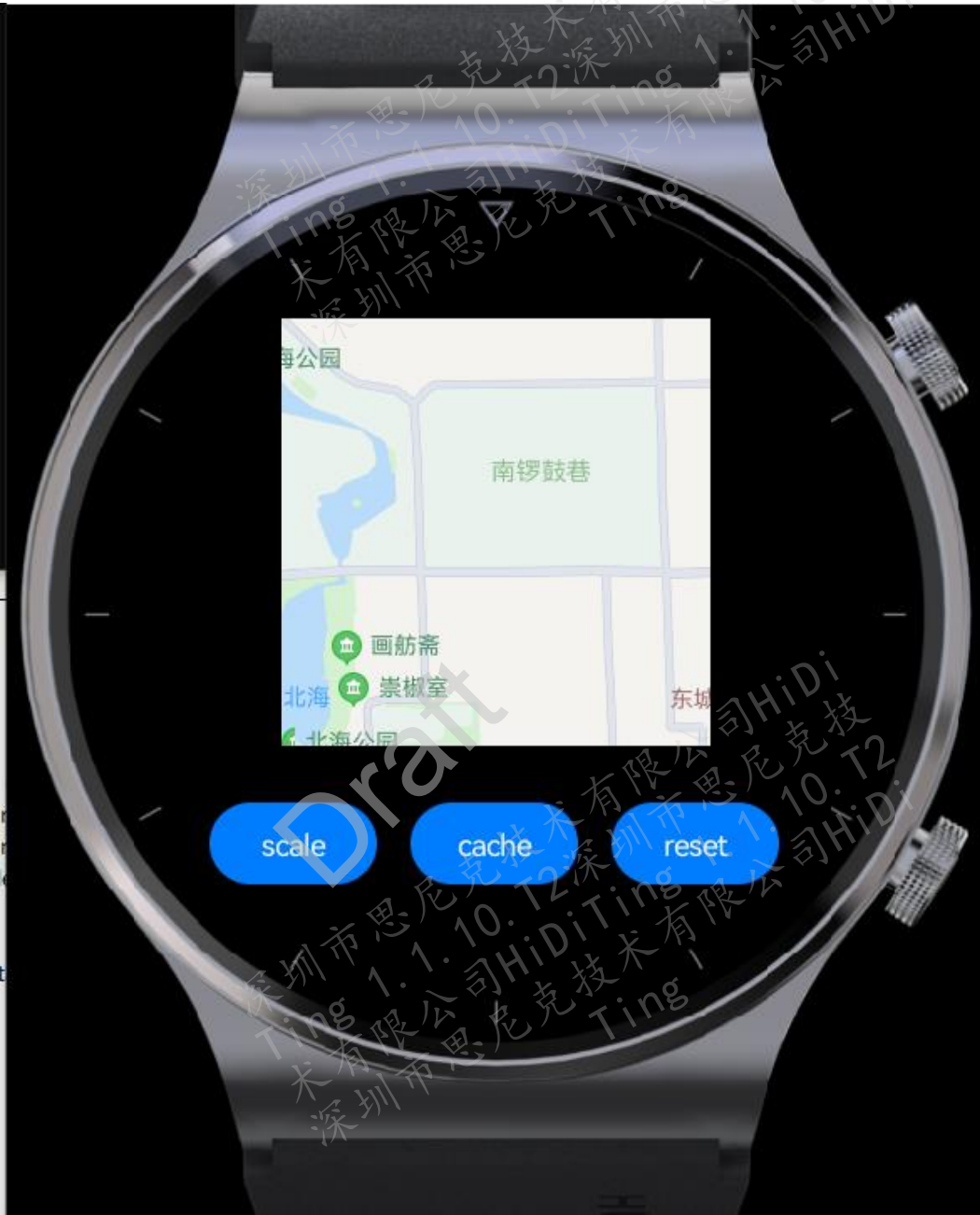


表2-47 UIMapView 核心函数

| 函数                                           | 描述             |
|----------------------------------------------|----------------|
| bool SetSVG(const char* path);               | 使用文件路径设置 SVG。  |
| bool SetSVG(uint8_t* data, uint32_t length); | 使用文件缓冲区设置 SVG。 |

| 函数                                                        | 描述                                                                                         |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------|
| void SetOnPOIClickListener(OnPOIClickListener* listener); | 设置 POI 的监听器。                                                                               |
| void EnableCache(bool enable);                            | 启用缓存。当 enable 为 true 时，它将立即缓存这个图块，并立即释放缓存。我们需要在设置 SVG 的时候开启缓存，或者在缩放地图完成之后开启缓存，在缩放地图之前关闭缓存。 |
| void ScaleMap(float scale, const Vector2<float>& pivot);  | 用于以目标点为基准缩放地图。                                                                             |
| void HidePOI(uint16_t tag, bool isHide = true);           | 通过标签隐藏 POI。                                                                                |

## 2.4 非容器类组件

### 2.4.1 UIImageView

UIImageView 用于展示图片。

#### 须知

- 使用时需注意 AutoEnable 和 ResizeMode 属性设置顺序，建议 AutoEnable 在 ResizeMode 之前进行设置。
- 使用 UIImageView 背景不透明度建议调整为 0。

表2-48 UIImageView 核心函数

| 函数                                 | 介绍                |
|------------------------------------|-------------------|
| void SetSrc(const char* src)       | 通过路径来设置图片。        |
| void SetSrc(const ImageInfo* src)  | 设置组件图片信息。         |
| void SetAutoEnable(bool enable)    | 设置组件自适应图片。        |
| void SetBlurLevel(BlurLevel level) | 设置旋转或缩放此图像时的模糊级别。 |



| 函数                                       | 介绍                                                                                                                                                                              |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| void SetResizeMode(ImageResizeMode mode) | 设置图片缩放模式（平铺、填充、保持宽高比）。 <ul style="list-style-type: none"> <li>在 AutoEnable 为 true 时，SetResizeMode()不生效。</li> <li>在 AutoEnable 为 false 时，需要先设置宽高，再设置 SetResizeMode()。</li> </ul> |
| void SetHeight(int16_t height) override  | 设置组件的高度。                                                                                                                                                                        |
| void SetWidth(int16_t width) override    | 设置组件的宽度。                                                                                                                                                                        |

## 2.4.2 UIImageAnimator

UIImageAnimator 组件继承自 UIImageView，主要用于播放序列帧。

表2-49 UIImageAnimator 核心函数

| 函数                                                                                                    | 介绍                                                                 |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| void SetImageAnimatorSrc(const ImageAnimatorInfo imageSrc[], uint8_t imageNum, uint16_t timeOfUpdate) | 设置序列帧图源信息以及更新时间，单位：ms。<br>注意：调用该接口时，需要将对应参数内部数据填充完整，禁止先传递地址，再填充数据。 |
| void SetImageAnimatorSrc(const ImageAnimatorInfo imageSrc[], uint8_t imageNum)                        | 设置序列帧图源信息。<br>注意：调用该接口时，需要将对应参数内部数据填充完整，禁止先传递地址，再填充数据。             |
| void SetTimeOfUpdate(uint16_t timeOfUpdate)                                                           | 设置更新时间，单位：ms。<br>该值将会被除以<br>DEFAULT_TASK_PERIOD，从而转换为更新周期。         |
| uint16_t GetTimeOfUpdate() const                                                                      | 获取更新时间。                                                            |
| const ImageAnimatorInfo*                                                                              | 获取序列帧图源信息。                                                         |

| 函数                                                                        | 介绍                                      |
|---------------------------------------------------------------------------|-----------------------------------------|
| <code>GetImageAnimatorSrc() const</code>                                  |                                         |
| <code>uint8_t GetImageAnimatorImageNum() const</code>                     | 获取序列帧图源数量。                              |
| <code>void SetSizeFixed(bool fixed)</code>                                | 设置序列帧图片大小是否保持一致。如果不一致，则每张图片按照各自设置的宽高显示。 |
| <code>bool IsSizeFixed() const</code>                                     | 获取序列帧图片大小是否保持一致。                        |
| <code>void SetRepeat(bool repeat)</code>                                  | 设置是否重复播放。                               |
| <code>bool IsRepeat() const</code>                                        | 获取是否重复播放。                               |
| <code>void SetRepeatTimes(uint32_t times)</code>                          | 设置重复播放次数。                               |
| <code>uint32_t GetRepeatTimes() const</code>                              | 获取重复播放次数。                               |
| <code>uint8_t GetState() const</code>                                     | 获取播放状态。                                 |
| <code>void SetReverse(bool reverse)</code>                                | 设置是否倒放。                                 |
| <code>bool IsReverse() const</code>                                       | 获取是否倒放。                                 |
| <code>void SetFillMode(bool fillMode)</code>                              | 设置图片填充模式。                               |
| <code>bool GetFillMode() const</code>                                     | 获取图片填充模式。                               |
| <code>void Start()</code>                                                 | 启动序列帧动画。                                |
| <code>void Stop()</code>                                                  | 停止序列帧动画。                                |
| <code>void Pause()</code>                                                 | 暂停序列帧动画。                                |
| <code>void Resume()</code>                                                | 继续序列帧动画。                                |
| <code>void SetAnimatorStopListener(AnimatorStopListener* listener)</code> | 设置序列帧停止监听。                              |

#### 说明

父控件为可滑动控件时，必需调用 `SetSizeFixed (True)`。

2.4.3 UILabel

UILabel 组件主要用于展示文字。

须知

该控件不支持透视变换变换，UILabelExt 也不支持。

表2-50 UILabel 核心函数

| 函数                                                                                                                 | 介绍                                                                       |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| void SetText(const char* text)                                                                                     | 设置展示文本内容。<br>参数：要展示的字符串                                                  |
| void SetLineBreakMode(const uint8_t lineBreakMode)                                                                 | 设置展示模式。例如是否包裹内容展示，是否换行等，具体请参考 UILabel::LineBreakMode。                    |
| void SetTextColor(ColorType color)                                                                                 | 设置文字颜色。                                                                  |
| void SetAlign(UITextLanguageAlignment horizontalAlign, UITextLanguageAlignment verticalAlign = TEXT_ALIGNMENT_TOP) | 设置对齐方式。                                                                  |
| void SetDirect(UITextLanguageDirect direct)                                                                        | 设置文本展示方向。                                                                |
| void SetFontId(uint8_t fontId)                                                                                     | 设置点阵字库 id。                                                               |
| void SetFont(const char* name, uint8_t size)                                                                       | 设置矢量字体和大小。<br>支持仅设置字体大小，字库 name 设置为空；常用于多语言场景，文本通过子类提供的接口 SetTextId 来配置。 |
| void SetRollStartPos(int16_t pos)                                                                                  | 设置字体滚动位置。                                                                |
| void SetWidth(int16_t width) override                                                                              | 设置当前组件的宽度。                                                               |
| void SetHeight(int16_t height) override                                                                            | 设置当前组件的高度。                                                               |

2.4.4 UILabelExt

UILabelExt 继承于 UILabel，支持文本语言切换以及复杂语系的文本整形。

表2-51 UILabelExt 核心函数

| 函数             | 描述                                                                                               |
|----------------|--------------------------------------------------------------------------------------------------|
| SetTextId      | 设置 TextId。 <ul style="list-style-type: none"><li>参数：uint16_t textId</li><li>返回值：void</li></ul>   |
| ForceResetText | 强制根据控件内部的 TextId 设置文本内容。 <ul style="list-style-type: none"><li>参数：N/A</li><li>返回值：void</li></ul> |

2.4.5 UIButton

UIButton 是可点击的按钮，用户可以通过函数设置图片以此改变默认展示效果。

表2-52 UIButton 核心函数

| 函数                                                                                 | 介绍                                                                            |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| void SetImageSrc(const char* defaultImgSrc, const char* triggeredImgSrc)           | 设置 UIButton 默认展示的图片和点击后展示的图片。<br>参数：图片文件路径。                                   |
| void SetImageSrc(const ImageInfo* defaultImgSrc, const ImageInfo* triggeredImgSrc) | 设置 UIButton 默认展示的图片 and 点击后展示的图片，ImageInfo 主要用于加载系统内置资源。<br>参数：传入的 ImageInfo。 |
| void SetImagePosition(const int16_t x, const int16_t y)                            | 设置资源图片相对于控件的位置。                                                               |
| void Disable()                                                                     | UIButton 不可点击。                                                                |
| void Enable()                                                                      | 恢复 UIButton 可点击。                                                              |
| void SetStateForStyle(ButtonState state)                                           | 设置样式状态。非当前状态，而是即将                                                             |

| 函数                                                                   | 介绍                  |
|----------------------------------------------------------------------|---------------------|
|                                                                      | 要设置的样式对应的状态。        |
| void SetStyle(uint8_t key, int64_t value) override                   | 设置当前 style 对应属性的样式。 |
| void SetStyleForState(uint8_t key, int64_t value, ButtonState state) | 定制对应状态的样式。          |
| void SetHeight(int16_t height)                                       | 设置组件高度。             |
| void SetWidth(int16_t width)                                         | 设置组件宽度。             |

## 2.4.6 UILabelButtonExt

UILabelButtonExt 继承于 UILabelButton，支持文本语言切换以及整形。

表2-53 UILabelButtonExt 核心函数

| 函数             | 描述                                                                                               |
|----------------|--------------------------------------------------------------------------------------------------|
| SetTextId      | 设置 TextId。 <ul style="list-style-type: none"><li>参数：uint16_t textId</li><li>返回值：void</li></ul>   |
| ForceResetText | 强制根据控件内部的 TextId 设置文本内容。 <ul style="list-style-type: none"><li>参数：N/A</li><li>返回值：void</li></ul> |

## 2.4.7 UILabelButton

UILabelButton 是展示文字的按钮。该组件继承于 UIButton，事件监听可以使用 UIButton 的函数设置监听器。

表2-54 UILabelButton 核心函数

| 函数                             | 介绍           |
|--------------------------------|--------------|
| void SetText(const char* text) | 为组件设置文字。     |
| const char* GetText() const    | 获取组件当前设置的文字。 |

| 函数                                                                        | 介绍             |
|---------------------------------------------------------------------------|----------------|
| void SetLabelPosition(int16_t x, int16_t y)                               | 设置文字在组件位置。     |
| Point GetLabelPosition() const                                            | 获取文字在组件位置。     |
| void SetAlign(UITextLanguageAlignment align)                              | 设置文字在组件中的对齐方式。 |
| void SetDirect(UITextLanguageDirect direct)                               | 设置文字在组件中的方向。   |
| void SetTextColor(ColorType color)                                        | 设置文字颜色。        |
| void SetFont(const char* name, uint8_t size)                              | 设置字体和字体大小。     |
| void SetLabelStyleForState(uint8_t key, int64_t value, ButtonState state) | 设置组件各个状态样式。    |

## 2.4.8 UICheckBox

UICheckBox 常用来表示状态开关，包括选中和非选择状态。

表2-55 UICheckBox 核心函数

| 函数                                                                                             | 介绍                                           |
|------------------------------------------------------------------------------------------------|----------------------------------------------|
| Void SetOnChangeListener(OnChangeListener* onStateChangeListener)                              | 为 UICheckBox 设置监听器，在状态改变时回调监听器的 OnChange 函数。 |
| bool OnClickEvent(const ClickEvent& event) override                                            | 执行点击事件。                                      |
| virtual void SetImages(const char* selectedImageSrc, const char* unselectedImageSrc)           | 通过图片路径设置选中和不选择状态的图片。                         |
| virtual void SetImages(const ImageInfo* selectedImageSrc, const ImageInfo* unselectedImageSrc) | 通过 ImageInfo 设置选中和不选择状态的图片。                  |
| UICheckBoxState GetState() const                                                               | 获取当前状态。                                      |
| void SetState(UICheckBoxState state, bool needAnimator = false)                                | 设置状态以及状态改变是否触发动效。                            |
| void SetSelectedStateColor(ColorType color)                                                    | 设置选中状态颜色。                                    |

| 函数                                         | 介绍        |
|--------------------------------------------|-----------|
| ColorType GetSelectedStateColor()<br>const | 获取选中状态颜色。 |

## 2.4.9 UIRadioButton

UIRadioButton 又称为单选按钮，继承于 UICheckBox，可以复用父类的函数。常将多个 UIRadioButton 添加到一个 ViewGroup 中作为一组。实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_button/ui\\_test\\_button.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_button/ui_test_button.cpp)。

表2-56 UIRadioButton 核心函数

| 函数                             | 介绍    |
|--------------------------------|-------|
| void SetName(const char* name) | 设置名称。 |

## 2.4.10 UIToggleButton

UIToggleButton 继承于 UICheckBox，其展示方式类似于拨码开关。没有自定义扩展的函数，使用方法参考 UICheckBox。

## 2.4.11 UIArcLabel

UIArcLabel 是以圆弧形态展示字体的组件，可以在圆形表盘的圆周按照指定的圆心、半径和角度展示文字。

图2-18 UIArcLabel 效果



实例请参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_arc\\_label/ui\\_test\\_arc\\_label.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_arc_label/ui_test_arc_label.cpp)，常用函数如表 2-57 所示。

表2-57 UIArcLabel 核心函数

| 函数                                                         | 介绍                 |
|------------------------------------------------------------|--------------------|
| void SetText(const char* text)                             | 设置展示的文字。           |
| void SetAlign(UILanguageAlignment horizontalAlign)         | 设置此文本的对齐模式。        |
| UILanguageAlignment GetHorAlign()                          | 获取水平对齐方式。          |
| UILanguageDirect GetDirect()                               | 获取此文本的方向。          |
| void SetFont(const char* name, uint8_t size)               | 设置此圆弧标签的字体及字体大小。   |
| void SetArcTextCenter(int16_t x, int16_t y)                | 设置此圆弧文本的中心位置。      |
| void SetArcTextRadius(uint16_t radius)                     | 设置此圆弧文本的半径。        |
| uint16_t GetArcTextRadius() const                          | 获取此圆弧文本的半径。        |
| void SetArcTextAngle(int16_t startAngle, int16_t endAngle) | 设置此圆弧文本的开始角度和结束角度。 |



| 函数                                                      | 介绍                                               |
|---------------------------------------------------------|--------------------------------------------------|
| int16_t GetArcTextStartAngle() const                    | 获取此圆弧文本的起始角度。                                    |
| int16_t GetArcTextEndAngle() const                      | 获取此圆弧文本的结束角度。                                    |
| void SetArcTextOrientation(TextOrientation orientation) | 设置此圆弧文本的方向。<br>参数 orientation (INSIDE 或 OUTSIDE) |

2.4.12 UIBoxProgress

UIBoxProgress 是以条状形式表示进度的组件，例如播放视频时的进度条。继承关系为 UIBoxProgress→UIAbstractProgress。父类 UIAbstractProgress 的常用函数如表 2-58 所示。

表2-58 UIAbstractProgress 核心函数

| 函数                                                                                          | 介绍                         |
|---------------------------------------------------------------------------------------------|----------------------------|
| void EnableBackground(bool enable)                                                          | 设置进度条背景是否可见。对 UISlider 无效。 |
| void SetValue(int32_t value)                                                                | 设置此进度条的当前值。                |
| int32_t GetValue() const                                                                    | 获取此进度条的当前值。                |
| void SetRange(int32_t rangeMax, int32_t rangeMin)                                           | 设置此进度条的范围。                 |
| int32_t GetRangeMin() const                                                                 | 获取进度条最小值。                  |
| int32_t GetRangeMax() const                                                                 | 获取进度条最大值。                  |
| void SetCapType(CapType cap)                                                                | 设置端点风格。                    |
| void SetImage(const char* foregroundImage, const char* backgroundImage = nullptr)           | 设置此进度条的图像。                 |
| void SetImage(const ImageInfo* foregroundImage, const ImageInfo* backgroundImage = nullptr) | 设置此进度条的图像。                 |
| void SetStep(uint32_t step)                                                                 | 设置此进度条的步长。                 |
| uint32_t GetStep() const                                                                    | 获取此进度条的步长。                 |

UIBoxProgress 实例可参考  
[https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_progress\\_bar/ui\\_test\\_box\\_progress.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_progress_bar/ui_test_box_progress.cpp), 常用函数如表 2-59 所示。

表2-59 UIBoxProgress 核心函数

| 函数                                            | 介绍         |
|-----------------------------------------------|------------|
| void SetDirection(const Direction& direction) | 设置此进度条的方向。 |
| void SetValidWidth(int16_t width)             | 设置实际进度条宽度。 |
| void SetValidHeight(int16_t height)           | 设置实际进度条高度。 |
| int16_t GetValidWidth() const                 | 获取实际进度条宽度。 |
| int16_t GetValidHeight() const                | 获取实际进度条高度。 |

2.4.13 UICircleProgress

UICircleProgress 组件是以圆形来展示进度的组件。该组件继承关系为  
UICircleProgress→UIAbstractProgress。

当前介绍 UICircleProgress 特有的函数，具体使用可以参考 sample 代码  
[https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_progress\\_bar/ui\\_test\\_circle\\_progress.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_progress_bar/ui_test_circle_progress.cpp)。常用函数如表 2-60 所示

表2-60 UICircleProgress 核心函数

| 函数                                           | 介绍                     |
|----------------------------------------------|------------------------|
| void SetCenterPosition(int16_t x, int16_t y) | 设置此圆形进度条的中心点坐标。        |
| void SetRadius(uint16_t radius)              | 设置此圆形进度条的外半径。          |
| void SetStartAngle(int16_t startAngle)       | 设置开始角度（进度条初始值）。        |
| void SetEndAngle(int16_t endAngle)           | 设置结束角度（进度条 100%时加载位置）。 |

| 函数                                                    | 介绍                   |
|-------------------------------------------------------|----------------------|
| void SetCenterPosition(int16_t x, int16_t y)          | 设置进度条相对于控件的中心点位置。    |
| Point GetCenterPosition() const                       | 获取进度条相对于控件的中心点位置。    |
| void SetProgressImagePosition(int16_t x, int16_t y)   | 设置此进度条的前景图像相对于视图的坐标。 |
| void SetBackgroundImagePosition(int16_t x, int16_t y) | 设置此进度条的背景图像相对于视图的坐标。 |
| void SetLineColor(ColorType color)                    | 设置此进度条的前景色。          |

2.4.14 UISlider

UISlider 继承于 UIBoxProgress，也是以进度条的方式展示进度，常用于音量条。相较于 UIBoxProgress，该组件额外支持拖动以及点击事件。实例可参考[https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_slider/ui\\_test\\_slider.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_slider/ui_test_slider.cpp)。

图2-19 UISlider 效果

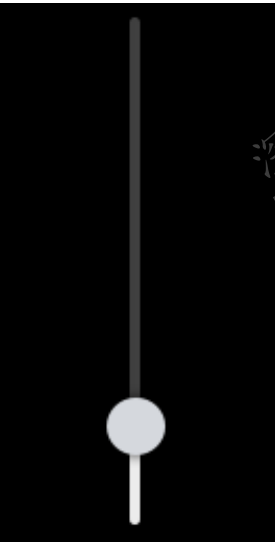


表2-61 UISlider 核心函数

| 函数 | 介绍 |
|----|----|
|----|----|

| 函数                                                                                                 | 介绍                      |
|----------------------------------------------------------------------------------------------------|-------------------------|
| <code>void SetKnobWidth(int16_t width)</code>                                                      | 设置此旋钮的宽度。               |
| <code>int16_t GetKnobWidth()</code>                                                                | 获取此旋钮的宽度。               |
| <code>void SetImage(const ImageInfo* backgroundImage, const ImageInfo* foregroundImage)</code>     | 将图像设置为此滑块的像素贴图，包括背景、前景。 |
| <code>void SetImage(const char* backgroundImage, const char* foregroundImage)</code>               | 设置此滑块的图像，包括背景、前景。       |
| <code>void SetSliderColor(const ColorType backgroundColor, const ColorType foregroundColor)</code> | 设置此滑块的颜色，包括背景、前景。       |
| <code>void SetSliderRadius(int16_t backgroundRadius, int16_t foregroundRadius)</code>              | 设置此滑块的半径，包括背景、前景。       |
| <code>void SetKnobImage(const ImageInfo* knobImage)</code>                                         | 设置旋钮图片。                 |
| <code>void SetKnobImage(const char* knobImage)</code>                                              | 设置旋钮图片。                 |
| <code>void SetKnobColor(const ColorType knobColor)</code>                                          | 设置旋钮颜色。                 |
| <code>void SetKnobRadius(int16_t knobRadius)</code>                                                | 设置旋钮半径。                 |
| <code>void SetKnobStyle(const Style&amp; style)</code>                                             | 设置旋钮样式。                 |
| <code>void SetKnobStyle(uint8_t key, int64_t value)</code>                                         | 设置旋钮样式。                 |
| <code>int64_t GetKnobStyle(uint8_t key) const</code>                                               | 获取旋钮样式。                 |
| <code>void SetSliderEventListener(UISliderEventListener* listener)</code>                          | 为该组件设置监听器，使用参考 sample。  |

## 2.4.15 UITextureMapper

UITextureMapper 继承于 UIImageView，主要用于做动画效果：缩放和旋转，实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_texture\\_mapper/ui\\_test\\_texture\\_mapper.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_texture_mapper/ui_test_texture_mapper.cpp)。常用函数如表 2-62 所示。

表2-62 UITextureMappre 核心函数

| 函数                                                           | 介绍                  |
|--------------------------------------------------------------|---------------------|
| void Start()                                                 | 启动动画。               |
| void SetRotateStart(int16_t start)                           | 设置此图像的起始旋转角度。       |
| void SetRotateEnd(int16_t end)                               | 设置此图像的旋转结束角。        |
| void SetScaleStart(float start)                              | 设置缩放此图像的起始比率。       |
| void SetScaleEnd(float end)                                  | 设置缩放此图像的结束比率。       |
| void SetDurationTime(uint16_t durationTime)                  | 设置此动画器的持续时间，以毫秒为单位。 |
| void SetDelayTime(uint16_t delayTime)                        | 设置此动画器的延迟时间，以毫秒为单位。 |
| void SetEasingFunc(EasingFunc easingFunc)                    | 设置此动画器的函数。          |
| void Cancel()                                                | 取消此动画。              |
| void Reset()                                                 | 显示原始图像。             |
| void SetPivot(float x, float y)                              | 设置此图像的旋转和缩放轴的变换中心点。 |
| void SetAnimatorStopListener(AnimatorStopListener* listener) | 设置此动画器停止的监听器。       |

2.4.16 UIQrcode

UIQrcode 继承于 UIImageView，主要是用来生成二维码。

图2-20 UIQrcode 效果



实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test\\_qrcode/ui\\_test\\_qrcode.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uitest/test_qrcode/ui_test_qrcode.cpp)。常用函数介绍如表 2-63 所示。

表2-63 UIQrcode 核心函数

| 函数                                                                                                                                                | 介绍                                                                                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>void SetQrcodeInfo(const char* val, ColorType backgroundColor = Color::White(), ColorType qrColor = Color::Black())</code>                  | 设置二维码信息。 <ul style="list-style-type: none"><li>参数 <code>val</code>: 生成二维码内容的字符串。</li><li>参数 <code>backgroundColor</code>: 二维码的背景色，默认为白色。</li><li>参数 <code>qrColor</code>: 二维码颜色，默认为黑色。</li></ul>                                            |
| <code>void SetQrcodeInfo(const uint8_t* val, uint16_t len, ColorType backgroundColor = Color::White(), ColorType qrColor = Color::Black())</code> | 设置二维码信息。参数说明： <ul style="list-style-type: none"><li><code>val</code>: 生成二维码内容的 binary 地址。</li><li><code>len</code>: binary 数据流长度。</li><li><code>backgroundColor</code>: 二维码的背景色，默认为白色。</li><li><code>qrColor</code>: 二维码颜色，默认为黑色。</li></ul> |
| <code>void SetWidth(int16_t width) override</code>                                                                                                | 设置生成二维码图片的宽度。                                                                                                                                                                                                                               |
| <code>void SetHeight(int16_t height) override</code>                                                                                              | 设置生成二维码图片的高度。                                                                                                                                                                                                                               |
| <code>void SetECCLevel(QrCode::Ecc level)</code>                                                                                                  | 设置二维码纠错等级。                                                                                                                                                                                                                                  |

2.4.17 UIBarcode

UIBarcode 继承于 UIImageView，主要是用来生成条形码。

图2-21 UIBarcode 效果



UIBarcode 的使用方式与 UIQrcode 类似，不再赘述。

表2-64 UIBarcode 核心函数函数介绍

| 函数                                      | 介绍                                     |
|-----------------------------------------|----------------------------------------|
| void SetBarcodeInfo(const char* srcVal) | 设置条形码信息。<br><br>参数 srcVal：生成条形码内容的字符串。 |
| void SetWidth(int16_t width) override   | 设置生成条形码图片的宽度。                          |
| void SetHeight(int16_t height) override | 设置生成条形码码图片的高度。                         |

2.4.18 UICanvas

UICanvas 继承于 UIView，只用于绘制各个形状图案，如直线，圆，弧线等，具体实例可参考 [https://gitee.com/openharmony/graphic\\_ui/tree/OpenHarmony-v3.0.3-LTS/test/uittest/test\\_canvas/ui\\_test\\_canvas.cpp](https://gitee.com/openharmony/graphic_ui/tree/OpenHarmony-v3.0.3-LTS/test/uittest/test_canvas/ui_test_canvas.cpp)，常用函数介绍如表 2-65 所示。

表2-65 UICanvas 核心函数

| 函数                                                       | 介绍                                                  |
|----------------------------------------------------------|-----------------------------------------------------|
| void SetStartPosition(const Point& startPoint)           | 设置绘制的起点坐标。                                          |
| void DrawLine(const Point& endPoint, const Paint& paint) | 绘制一条直线，如果没有通过 SetStartPosition 设置起点，默认走上一次结束位置开始绘制。 |



| 函数                                                                                                                               | 介绍                |
|----------------------------------------------------------------------------------------------------------------------------------|-------------------|
| void DrawLine(const Point& startPoint, const Point& endPoint, const Paint& paint)                                                | 从起点的坐标绘制直线，到终点结束。 |
| void DrawCurve(const Point& startPoint, const Point& control1, const Point& control2, const Point& endPoint, const Paint& paint) | 从起点坐标绘制三次贝塞尔曲线。   |
| void DrawCurve(const Point& control1, const Point& control2, const Point& endPoint, const Paint& paint)                          | 绘制立方贝塞尔曲线。        |
| void DrawRect(const Point& startPoint, int16_t height, int16_t width, const Paint& paint)                                        | 绘制矩形。             |
| void DrawCircle(const Point& center, uint16_t radius, const Paint& paint)                                                        | 绘制圆形。             |
| void DrawSector(const Point& center, uint16_t radius, int16_t startAngle, int16_t endAngle, const Paint& paint)                  | 绘制扇区。             |
| void DrawArc(const Point& center, uint16_t radius, int16_t startAngle, int16_t endAngle, const Paint& paint)                     | 绘制圆弧。             |
| void DrawImage(const Point& startPoint, const char* image, const Paint& paint)                                                   | 绘制图像。             |
| void DrawLabel(const Point& startPoint, const char* text, uint16_t maxWidth, const FontStyle& fontStyle, const Paint& paint)     | 绘制文本。             |
| void BeginPath()                                                                                                                 | 创建路径。             |
| void MoveTo(const Point& point)                                                                                                  | 将此路径的起点移动到指定的点。   |
| void LineTo(const Point& point)                                                                                                  | 创建从此路径的端点到指定点的直线。 |
| void ArcTo(const Point& center, uint16_t radius, int16_t startAngle, int16_t endAngle)                                           | 创建圆弧路径。           |
| void AddRect(const Point& point, int16_t height, int16_t width)                                                                  | 创建矩形路径。           |
| void ClosePath()                                                                                                                 | 关闭此路径。            |
| void DrawPath(const Paint& paint)                                                                                                | 绘制此路径。            |



## 2.4.19 UICanvasExt

UICanvasExt 继承于 UIView，与 UICanvas 功能相似，是基于硬件绘制能力定制的一个画布控件。绘制属性通过继承于 Paint 的 PaintExt 设置。常用函数介绍如表 2-66、表 2-67 所示。

表2-66 PaintExt 核心函数

| 函数                                                                                               | 介绍                                                                                                                                |
|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| void SetLinearGradient(const LinearGradient& linear, const ColorStop* stops, uint16_t stopCount) | 设置线性渐变。                                                                                                                           |
| void SetRadialGradient(const RadialGradient& radial, const ColorStop* stops, uint16_t stopCount) | 设置放射渐变。当前芯片不支持渐变中心点为负坐标。                                                                                                          |
| void SetSweepGradient(const SweepGradient& sweep, const ColorStop* stops, uint16_t stopCount)    | 设置扫描渐变，默认 12 点钟方向为 0 度位置，若传入值为负数，则按 0 处理。当前芯片不支持渐变中心点为负坐标。<br><br>注意旧版扫描渐变是以 3 点钟为起始方向，弧形绘制是以 12 点钟为起始方向；新版扫描渐变和弧形绘制都是以 12 点钟为方向。 |
| void SetFillRule(const FillRule& fillRule)                                                       | 设置填充规则。仅对 Fill 有效。                                                                                                                |
| void SetGradientSpread (const GradientSpread& gradientSpread)                                    | 设置超出渐变区域的渲染模式。                                                                                                                    |
| void SetCapType(const CapType& capType)                                                          | 设置线帽样式。仅对 Stroke 类型有效。<br><br>注：如果设置线帽类型为 round，会在原有路径两端加上以线宽为直径的半圆。                                                              |
| void SetJoinType (const JoinType& joinType)                                                      | 设置线段连接样式。仅对 Stroke 类型有效。                                                                                                          |
| void SetMiterLimit (float miterLimit)                                                            | 设置线端连接时内接点和外接点的长度限制。仅对 Stroke 类型且线段连接样式为 JOIN_MITER 有效。                                                                           |

| 函数                                     | 介绍       |
|----------------------------------------|----------|
| void SetAntialiased (bool antialiased) | 设置是否抗锯齿。 |
| void SetClipRect(Rect clipRect)        | 设置剪切矩阵。  |

### 说明

渐变指的是多色的一个渐变效果，设置单色会提示错误。

表2-67 UICanvasExt 核心函数

| 函数                                                                                         | 介绍                                                                                                              |
|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| int16_t BeginPath(PaintExt paint)                                                          | 开始一段路径。<br><br>若已经调用过且后续未调用 SetPathData 进行数据设置，或是数据设置后未调用 DrawPath，再次调用则无效。<br><br>返回此 path 对象对应的 index。        |
| void SetPathData(uint32_t cmdNum, const uint8_t* cmd, uint32_t dataNum, const float* data) | 设置 Path 数据。<br><br>仅在开始一段路径后调用。针对单个路径，仅能设置一次。                                                                   |
| void SetMatrix(Matrix3<float> matrix, int16_t index = -1)                                  | 设置内部 path 对象对应的变换矩阵。如不设置，则使用单位矩阵。<br><br>index 默认参数为-1，表示最后一个创建的 path 对象。用户可使用其他接口返回的 path index 指定具体的 path 对象。 |
| void DrawPath()                                                                            | 设置路径绘制标志位为 true。<br><br>若未调用此函数，创建的 Path 将无法显示在画布上。后续也无法创建新路径。                                                  |
| void Clear()                                                                               | 清除 Canvas 画布，清除所有内部 Path 对象以及其相关的数据。                                                                            |
| int16_t DrawLine(const Point& startPoint, const Point& endPoint, const PaintExt& paint)    | 绘制直线。<br><br>返回此 path 对象对应的 index。                                                                              |

| 函数                                                                                                                                                                      | 介绍                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <code>int16_t DrawCurve(const Point&amp; startPoint, const Point&amp; control1, const Point&amp; control2, const Point&amp; endPoint, const PaintExt&amp; paint)</code> | 绘制三阶贝塞尔曲线。<br>返回此 path 对象对应的 index。                            |
| <code>int16_t DrawRect(const Point&amp; startPoint, int16_t height, int16_t width, const PaintExt&amp; paint)</code>                                                    | 绘制矩形。<br>返回此 path 对象对应的 index。                                 |
| <code>int16_t DrawArc(const Point&amp; center, uint16_t radius, int16_t startAngle, int16_t endAngle, const PaintExt&amp; paint)</code>                                 | 绘制弧线。注：画跨 0 度区间且为上半部分的弧线，需要分 2 段拼接实现。<br>返回此 path 对象对应的 index。 |
| <code>int16_t DrawSvg(const char* path)</code>                                                                                                                          | 绘制 svg 文件通过工具转换出的 bin。                                         |
| <code>void SetSvgMatrix(Matrix3&lt;float&gt; matrix, int16_t index)</code>                                                                                              | 设置内部 svg 对象对应的变换矩阵。如不设置，则使用单位矩阵。                               |

#### 说明

DrawSvg API 入参为 svg\_to\_bin 工具转换出的二进制文件路径。

工具路径：customer\tools\DebugKits\graphic\_tools\svg\_to\_path

- Linux 环境：
  - 前置条件：需要安装 python 3.8.10 以上版本。
  - 使用命令：python3 main.py -f [FILE] -output [output]  
\* [FILE]：svg 格式文件，文件名中不能包含空格。  
\* [output]：输出路径，非必要。
  - 使用示例：python3 main.py -f ./run.svg
- Windows 环境：
  - 前置条件：需要安装 python 3.11.2 以上版本，并将添加到计算机环境变量内。
  - 使用命令：python main.py -f [FILE] -output [output]  
\* [FILE]：svg 格式文件，文件名中不能包含空格。  
\* [output]：输出路径，非必要。
  - 使用示例：python main.py -f ./run.svg

## 2.4.20 UILiteSurfaceView

UILiteSurfaceView 是用于视频预览场景的组件，继承于 UIView。

表2-68 UILiteSurfaceView 核心函数

| 函数                                                                              | 介绍                                          |
|---------------------------------------------------------------------------------|---------------------------------------------|
| Surface* GetSurface() const;                                                    | 获取 Surface。                                 |
| void SetPosition(int16_t x, int16_t y, int16_t width, int16_t height) override; | 设置当前视图的位置以及宽高。                              |
| void Resize(int16_t width, int16_t height) override;                            | 调整此视图的大小。                                   |
| void SetX(int16_t x) override;                                                  | 设置此视图的 X 坐标位置。                              |
| void SetY(int16_t y) override;                                                  | 设置此视图的 Y 坐标位置。                              |
| void SetWidth(int16_t width) override;                                          | 设置此视图的宽度。                                   |
| void SetHeight(int16_t height) override;                                        | 设置此视图的高度。                                   |
| void SetSurfaceColorkey(ColorType color)                                        | 设置 Colorkey 值。图形层最终的像素颜色与这个颜色值一致时，将透出视频层内容。 |

LiteSurface 是由 UILiteSurfaceView 中通过 GetSurface 获取的 Surface，用于管理媒体和图形的共享内存。

表2-69 LiteSurface 核心函数

| 函数                                                                | 介绍                                            |
|-------------------------------------------------------------------|-----------------------------------------------|
| void SetQueueSize(uint8_t queueSize) override;                    | 设置 buffer 队列大小（最大值为 3，建议设置为 3）。               |
| uint8_t GetQueueSize() override;                                  | 获取 buffer 队列大小。                               |
| void SetWidthAndHeight(uint32_t width, uint32_t height) override; | 设置 buffer 宽高。                                 |
| uint32_t GetWidth() override;                                     | 获取 buffer 宽度。                                 |
| uint32_t GetHeight() override;                                    | 获取 buffer 高度。                                 |
| void SetFormat(uint32_t format) override;                         | 设置 buffer 像素格式（目前仅支持 PIXEL_FMT_YCBCR_420_SP）。 |
| uint32_t GetFormat() override;                                    | 获取 buffer 像素格式。                               |

| 函数                                                                         | 介绍                                                     |
|----------------------------------------------------------------------------|--------------------------------------------------------|
| void RegisterConsumerListener(IBufferConsumerListener& listener) override; | 注册消费者回调。当生产者刷新一个 buffer 至脏队列即通知消费者。                    |
| void UnregisterConsumerListener() override;                                | 去注册消费者回调。                                              |
| void SetStrideAlignment(uint32_t strideAlignment) override;                | 设置 stride 对齐。                                          |
| uint32_t GetStrideAlignment() override;                                    | 获取 stride 对齐（目前必须传入 64）。                               |
| uint32_t GetStride() override;                                             | 获取 stride。                                             |
| void SetSize(uint32_t size) override;                                      | 设置 buffer 大小。                                          |
| uint32_t GetSize() override;                                               | 获取 buffer 大小。                                          |
| SurfaceBuffer* RequestBuffer(uint8_t wait = 0) override;                   | 生产者从空闲队列中获取 buffer 用于填写数据。                             |
| int32_t FlushBuffer(SurfaceBuffer* buffer) override;                       | 生产者在 RequestBuffer 后且完成填写数据动作后，刷新 buffer 至脏队列，等待消费者处理。 |
| void CancelBuffer(SurfaceBuffer* buffer) override;                         | 生产者释放 buffer 至空闲队列。                                    |
| SurfaceBuffer* AcquireBuffer() override;                                   | 消费者从脏队列中获取被生产者刷新的 buffer 用于送显。                         |
| bool ReleaseBuffer(SurfaceBuffer* buffer) override;                        | 消费者在 AcquireBuffer 后且完成送显动作后，释放 buffer 至空闲队列。          |

## 2.4.21 UIEditText

UIEditText 是支持输入法编辑的文本类组件，继承于 UIView。

支持两种以下模式：

- 明码模式：显示文本具体内容。
- 密码模式：不显示具体输入的内容，以符合 “” 替代。

图2-22 UICollectionCell 两种模式显示效果图

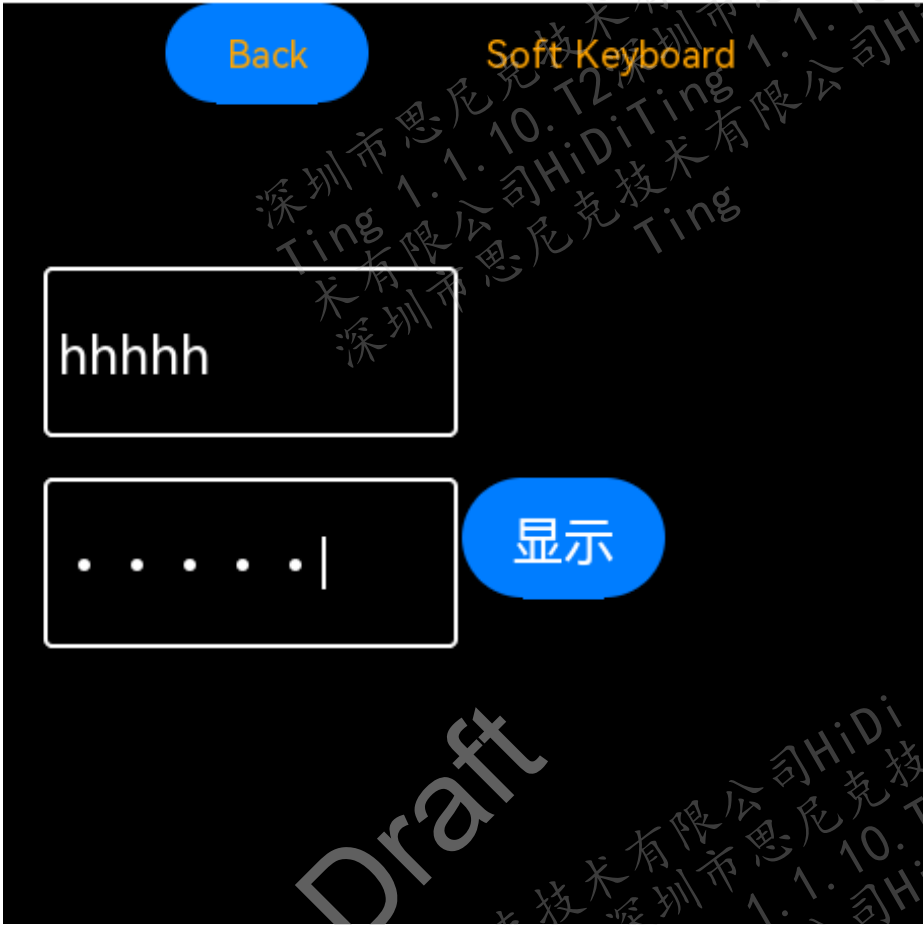
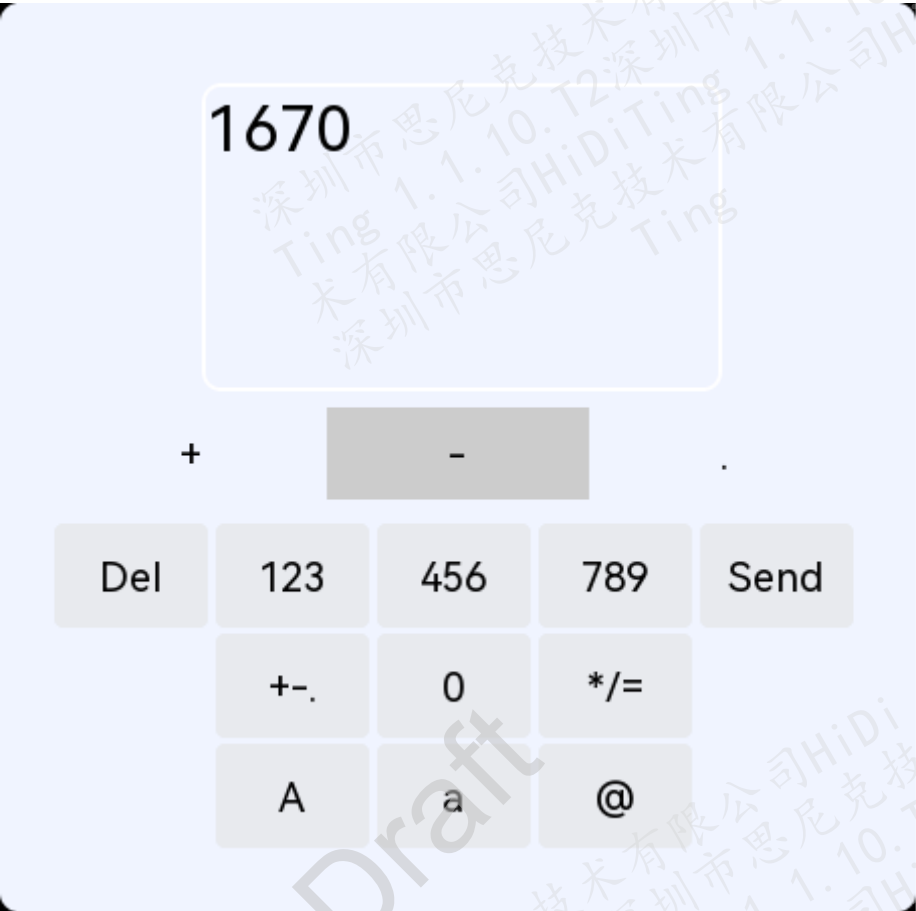


图2-23 软键盘效果图



常用函数介绍如表 2-70 所示。

表2-70 UITextView 核心函数

| 函数                                           | 介绍          |
|----------------------------------------------|-------------|
| void SetTextColor(ColorType color)           | 设置文本内容显示颜色。 |
| ColorType GetTextColor()                     | 获取文本内容显示颜色。 |
| void SetInputType(InputType type)            | 设置文本内容显示模式。 |
| InputType GetInputType()                     | 获取文本内容显示模式。 |
| void SetText(const char* text)               | 设置当前文本内容。   |
| const char* GetText()                        | 获取当前文本内容。   |
| void SetPlaceholder(const char* placeholder) | 设置提示文本内容。   |



| 函数                                           | 介绍            |
|----------------------------------------------|---------------|
| const char* GetPlaceholder()                 | 获取提示文本内容。     |
| void SetMaxLength(uint16_t maxLength)        | 设置最大文本内容长度。   |
| uint16_t GetMaxLength()                      | 获取最大文本内容长度。   |
| void SetPlaceholderColor(ColorType color)    | 设置提示文本内容显示颜色。 |
| ColorType GetPlaceholderColor()              | 获取提示文本内容显示颜色。 |
| void SetCursorColor(ColorType color)         | 设置光标颜色。       |
| ColorType GetCursorColor()                   | 获取光标颜色。       |
| void SetFontId(uint16_t fontId)              | 设置字体。         |
| uint8_t GetFontId()                          | 获取字体。         |
| void SetFont(const char* name, uint8_t size) | 设置字体。         |
| uint16_t GetTextWidth()                      | 获取字体宽度。       |
| uint16_t GetTextHeight()                     | 获取字体高度。       |
| void InsertText(std::string text)            | 在末尾插入文本。      |
| void DeleteBackward(uint32_t length)         | 删除末尾文本。       |

 说明

软键盘功能为客户参考代码，适配的屏幕为 454×454 屏，客户可按需进行布局调整，包括布局效果调整、适配不同屏幕尺寸等。重新适配需要修改文件：  
middleware\services\gui\UIKit\proprietary\src\ui\frameworks\components\lui\_soft\_keyboard.cpp  
中的控件位置、大小等布局信息。

2.4.22 UIKaleidoscopeView

UIKaleidoscopeView 是基于原始图片素材生成万花筒效果的控件。该控件将直接读写送显 Buffer，因此需要设置控件起始位置为[0, 0]，并保持其尺寸与送显 Buffer 尺寸一致。



图2-24 万花筒效果示意图

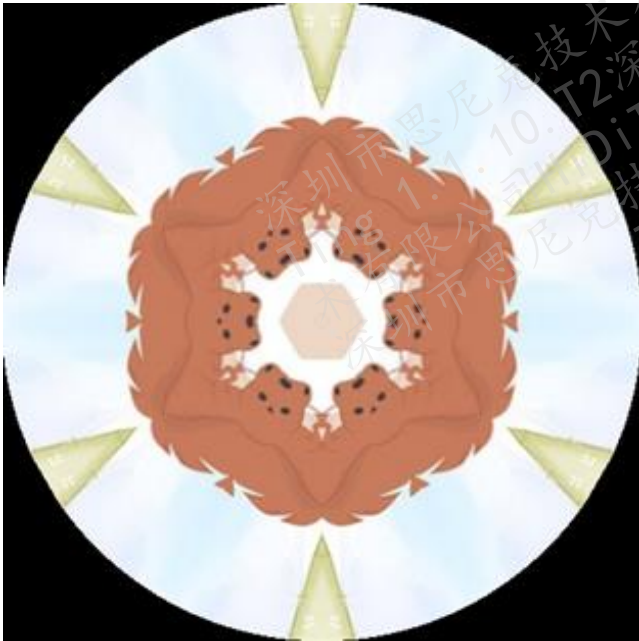


表2-71 UIKaleidoscopeView 核心函数

| 函数                                        | 描述                    |
|-------------------------------------------|-----------------------|
| bool SetSrc(const ImageInfo* src)         | 设置万花筒效果的源素材。          |
| void Rotate(float rotate)                 | 根据设定的旋转刻度，生成新的万花筒效果图。 |
| void SetRotateEventFactor(float factor)   | 设置万花筒响应表冠旋转事件的因子。     |
| void SetRotateAnimatorValue(float rotate) | 设置万花筒动画使用的旋转刻度。       |
| void StartRotateAnimator()                | 启动万花筒动画。              |
| void StopRotateAnimator()                 | 停止万花筒动画。              |

# 3 支持振动

## 3.1 功能介绍

## 3.2 振动类型

## 3.3 对外接口

### 3.1 功能介绍

滑动或旋转表冠时，某些控件会振动（请参考[表 3-2](#)），具体振动行为由外部通过 `VibratorManager` 注册。

#### 📖 说明

如果要启动振动，需要提前打开宏定义 `ENABLE_VIBRATOR` 并注册振动函数。

### 3.2 振动类型

振动类型说明如下：

```
enum class VibratorType {  
    VIBRATOR_TYPE_ONE, // 振动1次  
  
    VIBRATOR_TYPE_TWO, // 振动2次  
  
    VIBRATOR_TYPE_THREE // 振动3次  
};
```

### 3.3 对外接口

表3-1 VibratorManager 核心函数

| 函数                                                   | 说明                                                                                             |
|------------------------------------------------------|------------------------------------------------------------------------------------------------|
| void RegisterVibratorFunc(VibratorFunc vibratorFunc) | 注册振动函数，类型 VibratorFunc 的定义为<br><br>typedef<br>void(*VibratorFunc)(VibratorType vibratorType);。 |
| VibratorFunc GetVibratorFunc()                       | 获取当前振动函数。                                                                                      |

表3-2 支持振动的控件

| 支持振动的控件      |
|--------------|
| UIRollerView |
| UICollection |
| UIScrollView |
| UISlider     |
| UISwipeView  |

# 4 多语言切换

## 4.1 功能介绍

## 4.2 对外接口

## 4.3 XML APIs

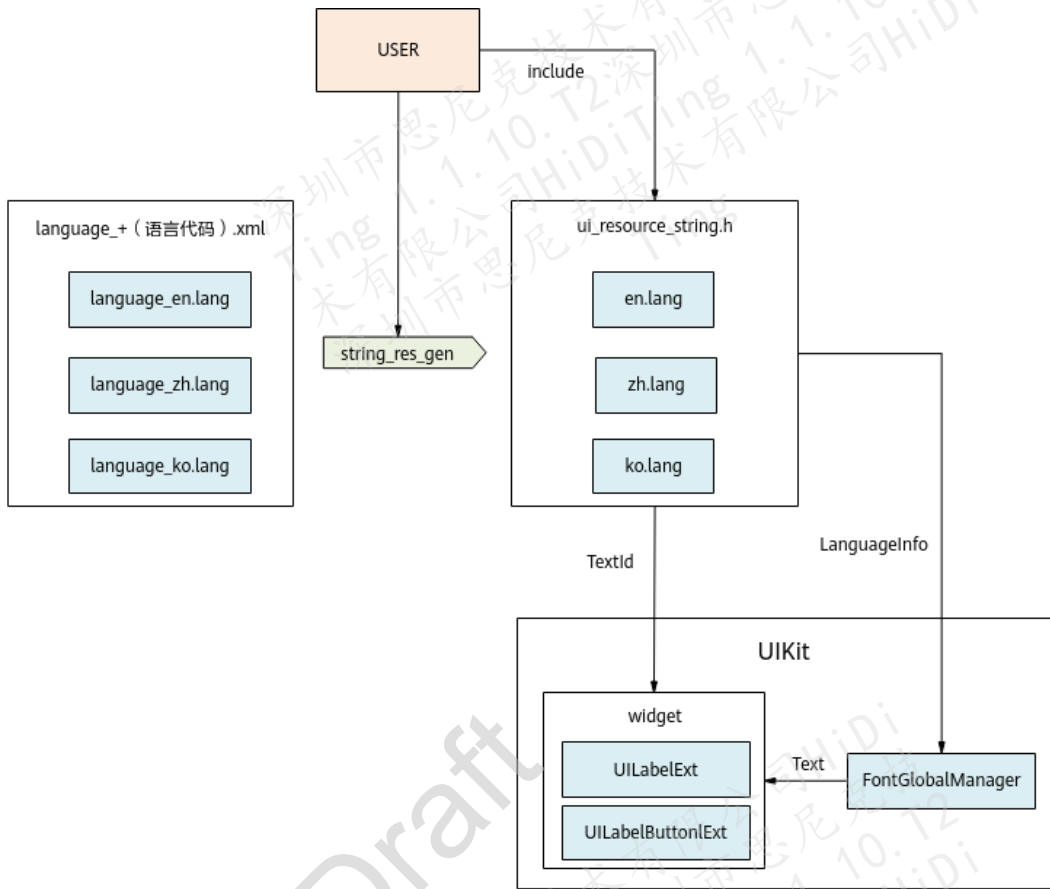
## 4.1 功能介绍

多语言切换主要提供的是字符串自动翻译功能，通过字符串 ID 自动查找对应语言的字符串，并提供了一个工具将 xml 转换为 bin 文件和对应的 C++头文件。

语言代码参考网站：<http://www.lingoes.cn/zh/translator/langcode.htm>

转换工具及使用请参考 Debugkits 指导文档《SpartaHiDiTingV100 DebugKits 工具 使用指南》的“多语言脚本调用”章节。

图4-1 原理框图



## 4.2 对外接口

### 4.2.1 FontGlobalManager

FontGlobalManager 为全球化文字管理模块，用户需要将多语言信息注册到该模块，通过 SetCurrentLangId 函数进行语言切换，其他接口为内部模块使用，外部模块可以忽略。

表4-1 FontGlobalManager 核心函数

| 函数                                            | 说明                                                                                                 |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------|
| static<br>FontGlobalManager*<br>GetInstance() | FontGlobalManager 是单例，该接口用于获取文字全球化管理的实例。<br><ul style="list-style-type: none"><li>参数：N/A</li></ul> |

| 函数                                                                                                   | 说明                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                      | <ul style="list-style-type: none"> <li>返回值类型: FontGlobalManager*</li> </ul>                                                                                                                                                                                                                                                                                                              |
| bool<br>RegisterLanguageInfo(const char** resFile,<br>uint8_t totalLangId,<br>uint8_t defaultLangId) | 注册语言信息。 <ul style="list-style-type: none"> <li>参数 1: const char** resFile, 字符串二进制文件数组首地址, 每个二进制文件对应一种语言。</li> <li>参数 2: uint8_t totalLangId, 文件最大个数 (最大值参考 ui_resource_string.h, 且不可超出实际注册语言个数)。</li> <li>参数 3: uint8_t defaultLangId, 默认的语言 ID (ID 值参考 ui_resource_string.h)。</li> </ul> 返回值类型: bool。 <ul style="list-style-type: none"> <li>true 代表成功。</li> <li>false 代表失败。</li> </ul> |
| void<br>UnRegisterLanguageInfo()                                                                     | 卸载所有的多语言字符串信息。 <ul style="list-style-type: none"> <li>传参: N/A</li> <li>返回值类型: void</li> </ul>                                                                                                                                                                                                                                                                                            |
| void<br>SetCurrentLangId(uint8_t langId)                                                             | 设置当前的语言 <ul style="list-style-type: none"> <li>传参: uint8_t langId (ID 值参考 ui_resource_string.h)</li> <li>返回值类型: void</li> </ul>                                                                                                                                                                                                                                                          |
| FontParam<br>GetFontParam(uint16_t textId)                                                           | 获取当前的语言。 <ul style="list-style-type: none"> <li>传参: uint8_t textId, 文本 ID, (ID 值参考 ui_resource_string.h)</li> <li>返回值类型: FontParam (fontSize, direct, fontName)</li> </ul>                                                                                                                                                                                                               |
| uint8_t GetTotalLangId()                                                                             | 获取支持的多语言数量。 <ul style="list-style-type: none"> <li>参数: N/A</li> <li>返回值类型: uint8_t 最大的语言 ID。</li> </ul>                                                                                                                                                                                                                                                                                  |

### 4.2.2 UILabelExt 和 UILabelButtonExt

多语言切换功能是以 UIKit 扩展方式实现的，为了尽量减少对原始 UIKit 的修改，扩展了两个组件，分别为 UILabelExt 和 UILabelButtonExt。如果控件需要支持多语言切换，需要将 UILabel 和 UILabelButton 切换到 UILabelExt 和 UILabelButtonExt，并通过 SetTextId 来设置静态文本。ForceResetText 在切换语言时由 FontGlobalManager 调用，用户无需调用该接口。

表4-2 UILabelButtonExt 核心函数

| 函数             | 描述                                                                                               |
|----------------|--------------------------------------------------------------------------------------------------|
| SetTextId      | 设置 TextId。 <ul style="list-style-type: none"><li>参数：uint16_t textId</li><li>返回值：void</li></ul>   |
| ForceResetText | 强制根据控件内部的 TextId 设置文本内容。 <ul style="list-style-type: none"><li>参数：N/A</li><li>返回值：void</li></ul> |

## 4.3 XML APIs

### 4.3.1 根元素

XML 必须以 language 为根元素并支持以下属性，language 元素中的属性对所有的子元素都生效，并且作为子元素的默认值，如果子元素有定义相同的属性，以子元素设置的生效。

表4-3 多语言 xml 配置

| 属性名          | 描述                          | 取值                                                | 必选 |
|--------------|-----------------------------|---------------------------------------------------|----|
| languageinfo | 语言信息，使用语言代码 (ISO 639) 标识语言。 | ISO 639 标准定义的语言代码                                 | 是  |
| direct       | 文本方向布局。                     | ltr (left to right) , rtl (right to left) , mixed | 否  |

| 属性名      | 描述                                                   | 取值      | 必选 |
|----------|------------------------------------------------------|---------|----|
|          |                                                      | (多语言混排) |    |
| fontName | 设置矢量字库名，路径需要在 graphic_config.h 中配置宏 VECTOR_FONT_DIR。 | ttf 文件名 | 否  |

### 4.3.2 字符串元素

字符串元素以 lanstr 为命名，定义字符串的相关信息，包括字符串取值、文本大小、字库名等信息。如果根元素也定义的相同的属性，会以字符串元素中定义的值为准，如果都没有定义，取 UIKit 中设置的默认值。

表4-4 字符串元素配置

| 属性名      | 描述                                                                                                                                          | 取值                                                        | 必选 |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|----|
| id       | 字符串唯一标识，需要保证唯一性。                                                                                                                            | 统一以 STR_ 为前缀                                              | 是  |
| value    | 字符串内容。                                                                                                                                      | 任意字符串                                                     | 是  |
| direct   | 文本方向布局。                                                                                                                                     | ltr (left to right) , rtl (right to left) , mixed (多语言混排) | 否  |
| size     | 文本大小，如果同时有在代码中通过接口 SetFont 进行设置，会按照和 SetText 以及 ForceResetText 先后顺序为准。                                                                      | 0 ~ 255                                                   | 否  |
| fontName | 设置矢量字库名，路径需要在 graphic_config.h 中配置宏 VECTOR_FONT_DIR。<br><br>注意：UILabelExt 和 UILabelButtonExt 不建议显式调用 SetFont 配置字库，而应该通过 fontName 字段配置；如果要配置 | ttf 文件名                                                   | 是  |



| 属性名 | 描述                                                                   | 取值 | 必选 |
|-----|----------------------------------------------------------------------|----|----|
|     | font 的大小，可以将 void SetFont(const char* name, uint8_t size);第一个参数配置为空。 |    |    |

# 5 多语言文本显示

## 5.1 功能介绍

## 5.2 关键对外接口

## 5.3 开发建议

## 5.1 功能介绍

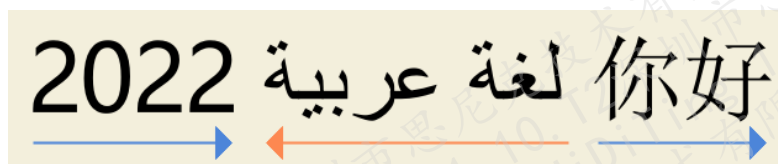
为了支持多语言文本的显示，UIKit 提供了如下文本显示功能：

- 支持注册多字库以及设置备选字库。
- 支持复杂语系文本的整形。
- 支持多语言文本（无论是否是复杂语系）的混排。
- 支持文本的 RTL、LTR 以及 MIXED（多语言文本混排场景）显示。
- 支持 ICU 换行规则。

以上功能限定于定制控件 UILabelExt 和 UILabelButtonExt，并且需确保 graphic\_config.h 中下述宏的值均为 1（默认均为 1）。

- ENABLE\_SHAPING：支持整形，依赖于 ENABLE\_ICU。
- ENABLE\_ICU：支持 ICU。
- ENABLE\_MULTI\_FONT：支持多字库。

图5-1 多语言文本混排效果



## 5.2 关键对外接口

UIFont 提供接口支持注册多字库，UIFontManager 提供接口支持配置字库的搜索顺序，两者配合即可实现基于多字库显示多语言文本的功能。ICU 换行、整形以及混排等功能需要使用 UILabelExt 和 UILabelButtonExt 两个控件，由系统内部处理，开发者并不可见。

表5-1 关键对外接口

| 类                    | 接口                                                                                            | 说明                                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UIFont               | <code>uint8_t RegisterFontInfo(const char* ttfName, uint8_t shaping = 0)</code>               | 注册系统字库。 <ul style="list-style-type: none"><li>参数 ttfName：字库名。</li><li>参数 shaping：当前字库是否包含整形的字形数据。</li></ul> 返回注册字库的 ID。<br>注意：向系统依次注册要使用的字库文件，但第一个需为不包含整形字形数据的字库。需将返回的 fontId 保存，为后续设置字库搜查表所用。 |
|                      | <code>uint8_t RegisterFontInfo(const UIFontLanguageFontParam* fontsTable, uint8_t num)</code> | 一次性注册多个系统字库。 <ul style="list-style-type: none"><li>参数 fontsTable：指向多个字库结构体数组的指针，字库 ID 保存在字库结构体中。</li><li>参数 num：字库结构体数组的个数。</li></ul>                                                          |
| UILabel/<br>UILabelB | <code>void SetDirect(UIFontLanguageDirect</code>                                              | 设置文本的方向。                                                                                                                                                                                       |

| 类                  | 接口                                                                           | 说明                                                                                                                                                                                                                                                       |
|--------------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| utton              | direct)                                                                      | <ul style="list-style-type: none"><li>参数 direct： 文本方向，TEXT_DIRECT_LTR / RTL / MIXED。</li></ul> <p>注意：对于混排并且方向不一致的多语言文本，开发者需要将文本方向设置为 TEXT_DIRECT_MIXED</p>                                                                                               |
|                    | void SetFont(const char* name, uint8_t size)                                 | <p>设置当前 Label 使用的字库。</p> <ul style="list-style-type: none"><li>参数 name： 字库名，如果为 nullptr，则只刷新字体大小。</li><li>参数 direct： 字体大小。</li></ul>                                                                                                                     |
|                    | void SetFontId(uint8_t fontId)                                               | <p>设置当前字库。</p> <ul style="list-style-type: none"><li>参数 fontId： 注册字库时返回的字库 ID。</li></ul>                                                                                                                                                                 |
| UIMultiFontManager | int8_t SetSearchFontList(uint8_t fontListId, uint8_t *fontIds, uint8_t size) | <p>配置备选字库。如果当前字库 id 为 fontListId，当出现不识别的字符时，则依次在 fontIds 所对应的备选字库中查找字符。</p> <ul style="list-style-type: none"><li>参数 fontListId： 字库 ID 值，保存为搜索表的 key 值。</li><li>参数 fontIds： 待搜索的字库 ID 值数组，保存在搜索表的 Value 值中。</li><li>参数 size： 待搜索的字库 ID 值数组的长度。</li></ul> |

## 5.3 开发建议

### 字库的注册

- 字库注册可以依次一个一个地注册多个字库，也可以一次性注册多个字库。
- 无论是哪种注册方式，支持整形的字库均不能第一个注册。
- 注册支持整形的字库，字库名必须包含并且仅包含一个匹配字段：Arabic、Thai、Myanmar、Devanagari、Hebrew、Bengali，用于快速基于 unicode 找到所在的整形字库。

### 文本控件的使用

- 文本控件建议优先使用扩展控件 UILabelExt、UILabelButtonExt，以便支持多语言切换、整形、ICU 换行等功能。
- 文本控件如果使用多语言切换功能（SetTextId），系统会自动配置匹配的字库，字体大小通过接口 SetFont(nullptr, size)指定。
- 文本控件如果不使用多语言切换功能，必须显式指定匹配的字库；否则会导致文本显示不出来，或者由于备选字库搜索时间过长导致显示卡顿。
- 文本控件如果包含有方向不一致的多语言文本，那么必须将文本方向设置为 TEXT\_DIRECT\_MIXED。

### 备选字库的使用

- UIFontMultiFontManager::SetSearchFontList 可以动态指定特定字库的备选字库：若当前字库不包含待处理的字符，则依次在其备选字库中查找。
- 备选字库根据混排文本的内容来确定，如果备选字库配置的不合理，则会导致混排文本中的非主文本显示不出来，或者因为备选字库搜索时间过长导致文本显示卡顿。

# 6 图片资源管理

- 6.1 方案目的
- 6.2 整体功能介绍
- 6.3 服务器要求
- 6.4 图片资源打包
- 6.5 资源使用

## 6.1 方案目的

在 IOT 的产品中，从 eMMC（一种嵌入式、非易失存储系统）中直接读取图片资源的速度比较慢。如果在图片绘制的过程中，每次都从 eMMC 中读取图片资源，那么 UI 的显示就会比较卡顿。因此，为了提升图片读取效率，则需要将图片资源提前缓存到 PSRAM（伪静态随机存取器）指定区域中，后续的读取可直接从 PSRAM 中读取，从而提升图片加载效率。

另外，PSRAM 由于成本问题，其内存的大小是严格受限的（目前可实现存储容量有：32M、64M、256M）。因此，需要将图片资源进行离线压缩，在 PSRAM 内存大小不变的情形下，可以缓存更多的图片资源。与此同时，还需要支持硬件对图片进行解压，以便减小图片压缩对显示性能的影响。

总之，该方案是为了在 PSRAM 受限的情形下，以更高的效率缓存和显示图片资源。

## 6.2 整体功能介绍

图片资源管理的总体方案主要分为两部分：

- PC 端对图片资源的压缩打包。（请参考《SpartaHiDiTingV100 DebugKits 工具使用指南》）
- 板端解析图片资源，并将其缓存到内存池中。

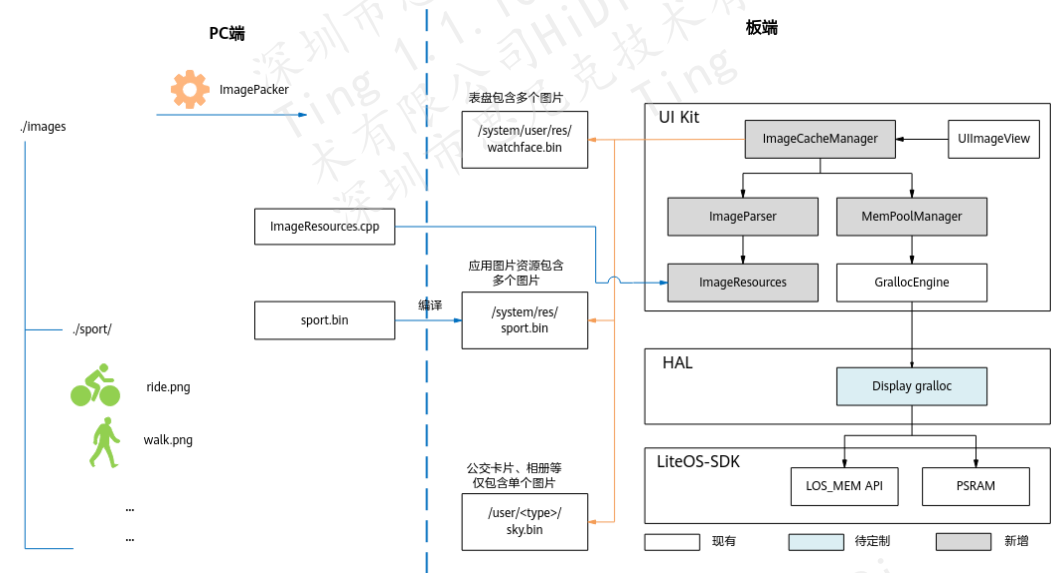
PC 端对图片资源的压缩打包，是指使用项目（工具组）提供的资源打包工具依据 xml 文件配置的压缩算法以及像素格式信息，将指定目录下的每一个子目录都打包成一个资源文件，并生成资源索引文件 ui\_resource\_image.h；或者将指定目录下的每一个图片文件都打包成一个资源文件。

ui\_resource\_image.h 存储了每一个被处理过的图片的 index 值，以便板端可以快速从打包的文件中定位到图片内容，具体可以参考后续的设计描述。

在板端，如果是单个目录打包成一个资源文件，ImageCacheManager 需要基于 ui\_resource\_image.h 中保存的图片索引信息从资源文件中解析出图片内容，否则 ImageCacheManager 可以直接从资源文件中解析出图片内容。然后，ImageCacheManager 利用系统提供的内存池机制对图片内容进行缓存。为了屏蔽芯片和系统的差异，我们使用 MemPoolManager + Gralloc + LOS\_MEM 的结构对内存池进行管理。另外，ImageCacheManager 也同时提供移除图片资源的接口，以便 PSRAM 内存资源得到充分利用。

注意，图片的压缩算法以及解压方案不是本方案的重点（UIKit 属于使用方），本文不再赘述。

图6-1 方案总体实现原理图



## 6.3 服务器要求

由于图片资源转换成 bin 格式文件、图片压缩、解压等操作是通过 Python 工具完成的。想要使用此功能，需要对服务器安装这两个工具。

```
pip install opencv-python
pip install Pillow
```

## 6.4 图片资源打包

图片资源打包工具的功能如下：依据 xml 文件配置的压缩算法以及像素格式信息，将指定目录下的每一个子目录都打包成一个资源文件，并生成资源索引文件 ui\_resource\_image.h；或者将指定目录下的每一个图片文件都打包成一个资源文件。

### 6.4.1 配置文件说明

```
<!-- 注意，compress和format必须同时配置；align可选，默认为1 -->
<!-- 注意，如果定制了文件的配置，其父目录也必须配置 -->
<!-- 指定默认的压缩算法、像素格式以及非压缩的宽度对齐值 -->
<!-- 以CS方式压缩：tile压缩单元宽度、alpha压缩模式、rgb压缩模式、ver压缩方式输入为1-->
<!-- CS、ES打包都应保证输入文件夹有图片可以压缩-->
<!-- 三种压缩模式的选择可以通过xml文件进行配置，但RGB565除外需要手动配置
format="RGB565"-->
<imageRes compress="HFBC" tile="6" alpha="1" rgb="1" autocmpmode="0" highqualitymode="0"
highcmpratiomode="0" ver="1">
</imageRes>
<!-- 以ES方式压缩：ver压缩方式输入为0-->
<imageRes compress="HFBC" autocmpmode="0" highqualitymode="0" highcmpratiomode="0"
ver="0">
</imageRes-->
```

表6-1 图片资源打包工具配置

| 标签       | 属性       | 说明             |
|----------|----------|----------------|
| imageRes | -        | 根标签。           |
|          | compress | 指定默认压缩算法，取值可为： |



| 标签        | 属性       | 说明                                                   |
|-----------|----------|------------------------------------------------------|
|           |          | HFBC HFBC_ABYPASS。                                   |
|           | format   | 指定默认像素格式，取值可为：<br>RGB565 RGB888 ARGB8888。            |
| imageDir  | -        | imageRes 的子标签，用于标识存放图片的文件夹。                          |
|           | name     | 文件夹名，可以指定为 app 的名字，或者场景名，不与 APP 强绑定。                 |
|           | compress | 指定该文件夹下的图片文件的默认压缩算法，优先级高于 imageRes 指定的 compress 属性值。 |
|           | format   | 指定该文件夹下的图片文件的默认像素格式，优先级高于 imageRes 指定的 format 属性值。   |
| imageFile | -        | imageDir 的子标签，用于标识具体某一个文件。                           |
|           | name     | 文件名。                                                 |
|           | compress | 指定特定图片文件的压缩算法，优先级高于 imageDir 指定的 compress 属性值。       |
|           | format   | 指定特定图片文件的像素格式，优先级高于 imageDir 指定的 format 属性值。         |

6.4.2 工具输入要求

- 工具建议命名为：images\_tool.py
- 命令格式：

```
python images_tool.py -p [dir|file] -c config.xml -i <Input Dir> -o <Output Dir> -t <Dir>
```

说明：

- -p: 指定打包方式。
- file: 以每个图片为单位进行打包。
- dir: 将图片进行打包后，再以子文件夹为单位将其包含的已打包的图片再一起打包。
- -c: 配置文件，指定打包的压缩算法、像素格式以及宽度对齐值。

- 压缩算法可选：HFBC|HFBC\_ABYPASS
- 像素格式可选：RGB565|RGB888|ARGB8888
- -i: 指定待处理图片的根目录，图片目录下的图片可以为.jpg、.png、.bmp 格式。
- -o: 指定输出的文件夹。
- -t: 指定头文件 ui\_resource\_image.h 生成的目录，仅在 dir 打包方式下生效。
- -e: 指定需要解压的文件路径，文件类型为.bin。
- -f: 指定解压后图片输出的路径。

输入限制：子文件夹名以及文件名必须为数字以及字母的组合。

6.4.3 打包使用

- 多图片按照目录打包命令如下：

```
python images_tool.py -p dir -c config.xml -i <Input Dir> -o <Output Dir>
```

- 单图片打包命令如下：

```
python images_tool.py -p file -c config.xml -i <Input Dir> -o <Output Dir>
```

6.5 资源使用

ImageCacheManager 是对外的唯一接口类。按照资源文件类型，可分为多资源文件以及单资源文件，通过对外接口可加载和卸载文件中的资源数据。

须知

如果用户想自己填充 ImageInfo，填充之前请使用 void\* ImageCacheMalloc(ImageInfo& info)接口申请 ImageInfo 所需的内存，然后再进行数据填充，不建议使用其他系统接口申请内存（没有适配）。

表6-2 ImageCacheManager 核心函数

| 函数                                                                                                               | 介绍                             |
|------------------------------------------------------------------------------------------------------------------|--------------------------------|
| bool LoadAllInMultiRes(const std::string& file, FILE* userFP = nullptr, bool isLongTerm = false, int offset = 0) | 一次性加载多资源文件中的所有资源，并为这些资源分配整块内存。 |

| 函数                                                                                                                                     | 介绍                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                        | <ul style="list-style-type: none"> <li>file: 图片 bin 文件的路径。</li> <li>userFP: 文件指针（在外部打开），如果 userFP 不为 nullptr，将首先使用它。</li> <li>isLongTerm: 图片是否应该长期保存。</li> <li>offset: 多图起始点的偏移量，主要与 userFP（非 nullptr）配合使用。</li> </ul> |
| bool LoadAllInMultiRes(uint8_t* buf)                                                                                                   | <p>一次从输入缓冲区加载全部图片资源。之后使用 LoadOneInMultiRes 来获取实际的图像。</p> <p>buf: 图片 bin 文件在内存中的地址。</p>                                                                                                                                 |
| ImageInfo* LoadOneInMultiRes(uint32_t resId, const std::string& file, FILE* userFP = nullptr, bool isLongTerm = false, int offset = 0) | <p>加载多资源文件中的单个指定资源，并为其分配内存。如果之前已经加载过镜像，那么它将直接返回缓冲区。</p> <ul style="list-style-type: none"> <li>resId: 要加载的图片的资源 ID。</li> <li>其他: 意义与 LoadAllInMultiRes 相同。</li> </ul>                                                  |
| ImageInfo* LoadOneInMultiRes(uint8_t* buf, uint32_t resId);                                                                            | <p>加载多资源文件中的单个指定资源，并为其分配内存。如果之前已经加载过镜像，那么它将直接返回缓冲区。</p> <ul style="list-style-type: none"> <li>buf: 图片 bin 文件在内存中的地址。</li> <li>resId: 要加载的图片的资源 ID。</li> </ul>                                                         |
| bool UnloadOneInMultiRes(uint32_t resId, const std::string& file)                                                                      | <p>卸载通过 LoadOneInMultiRes 的方式加载的单个资源。</p> <ul style="list-style-type: none"> <li>resId: 要卸载的图片的资源 ID。</li> <li>file: 图片 bin 文件的路径。</li> </ul>                                                                          |
| bool UnloadOneInMultiRes(uint8_t* buf, uint32_t resId)                                                                                 | <p>卸载通过 LoadOneInMultiRes 的方式加载的单个资源。</p> <ul style="list-style-type: none"> <li>buf: 图片 bin 文件在内存中的地址。</li> </ul>                                                                                                     |

| 函数                                                                         | 介绍                                                                                                               |
|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
|                                                                            | <ul style="list-style-type: none"><li>resId: 要卸载的图片的资源 ID。</li></ul>                                             |
| bool UnloadAllInMultiRes(const std::string& file)                          | 卸载多资源文件中已加载的资源。<br>file: 图片 bin 文件的路径。                                                                           |
| bool UnloadAllInMultiRes(uint8_t* buf)                                     | 卸载多资源文件中已加载的资源。<br>buf: 图片 bin 文件在内存中的地址。                                                                        |
| ImageInfo* LoadSingleRes(const std::string& file, bool isLongTerm = false) | 加载单资源文件中的资源。 <ul style="list-style-type: none"><li>file: 图片 bin 文件的路径。</li><li>isLongTerm: 图片是否应该长期保存。</li></ul> |
| bool UnloadSingleRes(const std::string& file)                              | 卸载单资源文件中的资源。<br>file: 图片 bin 文件的路径。                                                                              |

# 7

## 离线表盘

### 7.1 方案介绍

### 7.2 AOD 离线表盘

### 7.3 表盘制作

### 7.4 数据类型 model

### 7.5 相关类介绍

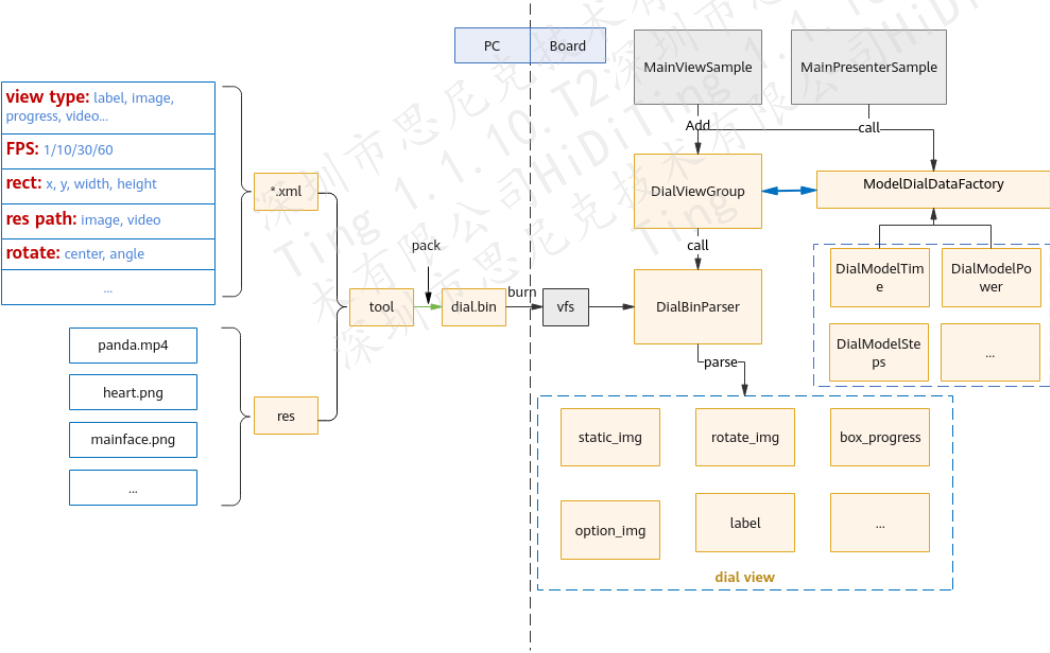
### 7.6 协议说明

## 7.1 方案介绍

离线表盘总体方案：

1. 将不同显示元素，拆分成不同的基础控件单元。
2. 定义不同的数据类型，用于数据绑定及数据更新。
3. 编写一个 xml 文件，里面写明控件类型（控件单元）、输入参数、周期配置、以及数据类型绑定。
4. 提供一套 python 打包脚本，将 xml 中的信息、以及对应的资源打包成 bin。
5. native 提供 DialBinParser 解析控件，用于解析 bin。
6. 提供 DialViewGroup，用于管理控件的添加、周期管理、数据刷新，并继承 cardpage，完成页面相关的管理机制。此控件即为最终的表盘页面。
7. 提供一套 model 管理机制，用于更新数据。可参考“7.4 数据类型 model”小节。

图7-1 离线表盘整体方案

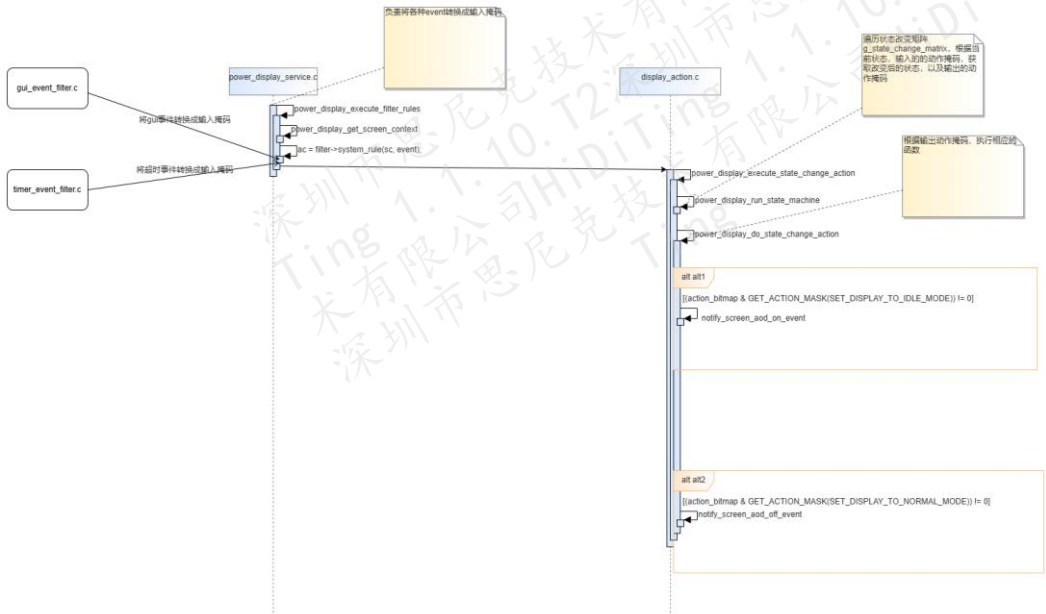


## 7.2 AOD 离线表盘

### 7.2.1 状态机

在文件“display\_state\_machine.c”中定义了状态机的状态转移表g\_state\_change\_matrix。g\_state\_change\_matrix 每个 item 分为：当前状态、输入掩码、输出状态、输出掩码。

图7-2 display\_state\_machine.c



如图 7-2 所示，状态机的工作流程如下：

- 步骤 1 “power\_display\_service.c” 负责将各种表盘事件转换成输入掩码。
- 步骤 2 “display\_action.c” 根据输入掩码和当前状态转换成输出状态和输出掩码，并将输出掩码转换成自定义函数。

----结束

可修改点：

- 用户可以参考现存的状态转移表 **g\_state\_change\_matrix** 自行修改进入和退出 AOD 表盘的输入掩码和输出掩码。
- 可以修改 “\*\_event\_filter.c”，将不同的触发 AOD 事件转换成输入掩码，例如将盖屏事件转换成 “ENTER\_AMBIENT” 或 “TURN\_ON\_SCREEN”。
- 可修改 “display\_action.c”，将不同的输出掩码与进入 AOD 表盘函数对应起来，例如将 “SET\_DISPLAY\_TO\_IDLE\_MODE” 对应到切换 AOD 表盘，将 “SET\_DISPLAY\_TO\_NORMAL\_MODE” 对应到切换到主表盘。

注意，如需关闭 AOD 模式，请按照以下步骤修改文件内容：

- 步骤 1 display\_state\_machine.c 中屏蔽 ambient 相关代码。

```
// 当前状态为Ambient
// {
```

```
// SCREEN_AMBIENT,
// GET_ACTION_MASK(TURN_ON_SCREEN),
// SCREEN_ON,
// GET_ACTION_MASK(SET_DISPLAY_ON) |
// GET_ACTION_MASK(SET_TE_ON) |
// GET_ACTION_MASK(SET_DISPLAY_TO_NORMAL_MODE) |
// GET_ACTION_MASK(SET_TP_TO_NORMAL_WORK_MODE) |
// GET_ACTION_MASK(START_TIMER),
// },
// {
// SCREEN_AMBIENT,
// GET_ACTION_MASK(KEEP_CURRENT_STATE) |
// GET_ACTION_MASK(ENTER_AMBIENT) |
// GET_ACTION_MASK(RESET_TIMER),
// SCREEN_AMBIENT,
// GET_ACTION_MASK(SET_DISPLAY_ON) |
// GET_ACTION_MASK(SET_DISPLAY_TO_IDLE_MODE)
// },
// {
// SCREEN_AMBIENT,
// GET_ACTION_MASK(TURN_OFF_SCREEN),
// SCREEN_OFF,
// GET_ACTION_MASK(SET_DISPLAY_OFF) |
// GET_ACTION_MASK(SET_TE_OFF) |
// GET_ACTION_MASK(SET_TP_TO_SLEEP_MODE) |
// GET_ACTION_MASK(STOP_TIMER),
// },
```

步骤 2 display\_action.c 中屏蔽输出掩码操作。

```
if ((action_bitmap & GET_ACTION_MASK(SET_DISPLAY_TO_IDLE_MODE)) != 0) {
    // notify_screen_aod_on_event();
}
if ((action_bitmap & GET_ACTION_MASK(SET_DISPLAY_TO_NORMAL_MODE)) != 0) {
    // notify_screen_aod_off_event();
}
```

----结束



7.2.2 AodView

表7-1 AodView 成员函数

| 方法                                                            | 描述                                 |
|---------------------------------------------------------------|------------------------------------|
| void AodView::OnStart()                                       | 初始化离线表盘和在线表盘。                      |
| void AodView::OnStop()                                        | 释放表盘资源操作，客户可以在此处释放在线表盘相关的资源。       |
| UIViewGroup<br>*AodView::InitAodDial(DialSetting &setting)    | 初始化在线表盘，客户可以在此函数进行扩展，返回自定义的在线表盘。   |
| UIViewGroup<br>*AodView::InitAodOffDial(DialSetting &setting) | 初始化离线表盘。                           |
| UIViewGroup<br>*AodView::InitDefaultAod()                     | 初始化默认表盘，客户可以在此函数进行扩展，返回自定义的默认离线表盘。 |

当状态机进入 Ambient 状态，根据输出掩码，slice 切换到 AodView。AodView 使用表 7-1 的成员函数初始化表盘，具体策略如下：

- 1. 如果当前主表盘是离线表盘，进入 AOD 尝试初始化离线表盘，如果失败就初始化默认表盘。
- 2. 如果当前主表盘是在线表盘，进入 AOD 尝试初始化在线表盘，如果失败就初始化默认表盘。

7.2.3 AOD 离线表盘

AOD 离线表盘方案特性：

- 1. AOD 离线表盘方案完全兼容离线表盘。
- 2. 与普通离线表盘相比，除<dial>标签外，其余标签均新增 “display\_mode” 属性，取值范围为“normal”、“ambient”、“always”，分别表示该标签支持在正常模式下显示、AOD 模式下显示、两种模式都显示。如果不显示指定该属性，则默认值是 “normal”。
- 3. 之前具有 “period” 属性的标签新增 “ambient\_periodic” 属性，值分别是“true”、“false”，分别表示在 AOD 模式下是周期更新、AOD 模式下不周期更新。

4. <dial>标签新增“ambient\_period”属性，表示 AOD 模式下的更新频率。注意，该值只能是{1, 10, 30, 60}之一，否则更新周期会变成无穷大。

#### 须知

<dial>使用新的 python 工具时，“version”属性的值必须是 2.0，否则会出现不可预知错误!!!

## 7.3 表盘制作

使用“graphic\_tools/dial\_converter\_tool/main.py”脚本生成 bin 格式文件。

执行命令参考示例：graphic\_tools/dial\_converter\_tool/README.txt。

xml 编写参考示例：graphic\_tools/dial\_converter\_tool/clock/dial.xml。

xml 支持的标签和属性参考表 7-2 的说明。

#### 须知

支持 AOD 表盘的工具 version 的值必须是“2.0”!

表7-2 xml 标签和属性说明

| 标签   | 标签解释          | 属性：值类型                   | 属性解释                            | 取值范围     | 默认值 | 是否必须 |
|------|---------------|--------------------------|---------------------------------|----------|-----|------|
| dial | 根节点指定表盘的基础信息。 | title: string            | 表盘名。                            | 字符串      | NA  | must |
|      |               | description: string      | 表盘描述。                           | 字符串      | NA  | must |
|      |               | size: int16_t<br>int16_t | 表盘大小。                           | (0, 960] | NA  | must |
|      |               | version: string          | 表盘版本，当值为“2.0”时，表时当前表盘具有 AOD 能力。 | 字符串      | NA  | must |
|      |               | preview_img: path        | 表盘在菜单界面的预览                      | 字符串      | NA  | must |

| 标签           | 标签解释                                   | 属性: 值类型                   | 属性解释                                                                                                    | 取值范围                          | 默认值      | 是否必须   |
|--------------|----------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------|----------|--------|
|              |                                        |                           | 图。                                                                                                      |                               |          |        |
|              |                                        | tile_width: uint8_t       | 该表盘的所有图片资源的压缩宽度。                                                                                        | 4, 6, 8, 16                   | 6        | option |
|              |                                        | ambient_period: uint8_t   | AOD 模式下, 属性 ambient_periodic 为 true 的标签主动更新频率, 如果不设置, 将会是 0, 表示不更新。                                     | 1, 10, 30, 60                 | 0        | option |
| static_img   | 静态图片, 不与数据绑定。可以用作背景。使用 Auto Enable 模式。 | position: int16_t int16_t | 控件绝对位置, 控件的宽高与图片一致。                                                                                     | (-32768, 32767)               | NA       | must   |
|              |                                        | center: int16_t int16_t   | 图片旋转中心点。                                                                                                | (-32768, 32767)               | (0, 0)   | option |
|              |                                        | rotate: float             | 图片旋转角度。                                                                                                 | [0, 360]                      | 0        | option |
|              |                                        | img_name: path            | 图片资源路径。                                                                                                 | 字符串                           | NA       | must   |
|              |                                        | display_mode: string      | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal", "ambient", "always" | "normal" | option |
| sequence_img | 序列帧, 不与数据绑定。                           | img_names: path, path ... | 序列帧的所有图片资源, 中间通过逗号隔开。                                                                                   | 字符串                           | NA       | must   |
|              |                                        | position: int16_t int16_t | 控件绝对位置, 控件的宽高与图片一致。                                                                                     | (-32768, 32767)               | NA       | must   |
|              |                                        | repeat: True              | 是否循环播放。                                                                                                 | True/Fa                       | NA       | must   |

| 标签         | 标签解释                                       | 属性：值类型                     | 属性解释                                                                                                   | 取值范围                          | 默认值      | 是否必须   |
|------------|--------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------|----------|--------|
|            |                                            | False                      |                                                                                                        | lse                           |          |        |
|            |                                            | interval: uint16_t         | 序列帧切换间隔。单位 ms                                                                                          | (0, 32767)                    | 0        | must   |
|            |                                            | display_mode:string        | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal", "ambient", "always" | "normal" | option |
| rotate_img | 可动态旋转的图片，与数据绑定。通过绑定不同的数据，可以实现时间指针自动旋转、图片自动 | img_name: path             | 指定可旋转图片的文件名。                                                                                           | 字符串                           | NA       | must   |
|            |                                            | position: int16_t int16_t  | 控件绝对位置，控件的宽高与图片一致。                                                                                     | (- 32768, 32767)              | (0, 0)   | must   |
|            |                                            | rotate_center: float float | 旋转中心点。                                                                                                 | 浮点数                           | 0.0      | must   |
|            |                                            | rotate_start: int16_t      | 旋转起始角度。                                                                                                | [0, 360]                      | 0        | must   |
|            |                                            | rotate_end: int16_t        | 旋转结束角度。                                                                                                | [0, 360]                      | 0        | must   |
|            |                                            | bind_data: int16_t         | 绑定旋转的数据，数据值为比例值。                                                                                       | [0, 32767)                    | 0        | must   |
|            |                                            | period: uint8_t            | 1s 内获取数据的频率。<br>该属性为非必须传参的必要属性，未使用时或者传参为 0 时，被动获取数据频率。                                                 | 0, 1, 10, 30, 60              | 0        | option |
|            |                                            | display_mode:string        | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是                                                                   | "normal", "ambie              | "normal" | option |

| 标签           | 标签解释                                       | 属性：值类型                    | 属性解释                                                                                                 | 取值范围                                | 默认值      | 是否必须   |
|--------------|--------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------|-------------------------------------|----------|--------|
|              | 旋转。<br><br>使用 Auto Enable 模式。              |                           | normal 下显示。<br>"ambient": 该控件只在 aod 模式下显示。<br>"always": 两种模式下都显示。                                    | nt",<br>"always"                    |          |        |
|              |                                            | ambient_periodic:bool     | "true": 该控件在 AOD 模式下主动更新, 且更新频率由标签 dial 的 ambient_period 属性控制。<br>"false": 该控件在 AOD 模式下不主动更新。        | "true",<br>"false"                  | "false"  | option |
| option_image | 可动态变化的图片, 与数据绑定。<br><br>使用 Auto Enable 模式。 | img_names: path,path ...  | 动态选择的所有图片资源, 中间通过逗号隔开。                                                                               | 字符串                                 | NA       | must   |
|              |                                            | position: int16_t int16_t | 控件绝对位置, 控件的宽高与图片一致。                                                                                  | (-32768, 32767)                     | (0, 0)   | must   |
|              |                                            | bind_data: int16_t        | 绑定的数据类型。                                                                                             | [0, 32767)                          | 0        | must   |
|              |                                            | period: uint8_t           | 1s 内获取数据的频率。<br>该属性为非必须传参的必要属性, 未使用时或者传参为 0 时, 被动获取数据频率。                                             | 0, 1, 10, 30, 60                    | 0        | option |
|              |                                            | display_mode:string       | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br>"ambient": 该控件只在 aod 模式下显示。<br>"always": 两种模式下都 | "normal",<br>"ambient",<br>"always" | "normal" | option |

| 标签          | 标签解释                                             | 属性: 值类型                         | 属性解释                                                                                              | 取值范围                                                                                    | 默认值     | 是否必须   |
|-------------|--------------------------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|---------|--------|
|             |                                                  |                                 | 显示。                                                                                               |                                                                                         |         |        |
|             |                                                  | ambient_periodic:bool           | "true": 该控件在 AOD 模式下主动更新, 且更新频率由标签 dial 的 ambient_period 属性控制。<br><br>"false": 该控件在 AOD 模式下不主动更新。 | "true", "false"                                                                         | "false" | option |
| digital_img | 可表达整数的图片组合, 与数据(数字)绑定。<br><br>使用 Auto Enable 模式。 | img_names: path, path...        | 数字图片, 0~9。                                                                                        | 字符串                                                                                     | NA      | must   |
|             |                                                  | sign_img_name: path             | 符号位。                                                                                              | 字符串                                                                                     | NA      | option |
|             |                                                  | space: uint16_t                 | 图片间隔。                                                                                             | [0, 32767)                                                                              | 0       | must   |
|             |                                                  | align: left   center   right    | 图片对齐方式。                                                                                           | left   center   right                                                                   | left    | must   |
|             |                                                  | align_pos: uint16_t<br>uint16_t | 图片对齐坐标点。                                                                                          | [0, 32767)                                                                              | (0, 0)  | must   |
|             |                                                  | integer_length: uint8_t         | 指定整数长度:                                                                                           | <ul style="list-style-type: none"><li>0, 使用实际长度。</li><li>&gt;0, 限定长度, 实际长度不足,</li></ul> | 0       | option |

| 标签 | 标签解释 | 属性：值类型                     | 属性解释                                                                                                   | 取值范围                          | 默认值      | 是否必须   |
|----|------|----------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------|----------|--------|
|    |      |                            |                                                                                                        | 则前方补零，超过则报错。                  |          |        |
|    |      | img_decimal_point: path    | 如果配置该变量，意味着显示的为浮点数。                                                                                    | 字符串                           | NA       | option |
|    |      | decimal_precision: uint8_t | 小数精度，必须配置img_decimal_point才生效。                                                                         | [0, 255]                      | 0        | option |
|    |      | bind_data: int16_t         | 绑定的数据类型。                                                                                               | [0, 32767)                    | 0        | must   |
|    |      | period: uint8_t            | 1s 内获取数据的频率。<br>该属性为非必须传参的必要属性，未使用时或者传参为 0 时，被动获取数据频率。                                                 | 0, 1, 10, 30, 60              | 0        | option |
|    |      | display_mode: string       | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal", "ambient", "always" | "normal" | option |
|    |      | ambient_periodic: bool     | "true": 该控件在 AOD 模式下主动更新，且更新频率由标签 dial 的 ambient_period 属性控制。                                          | "true", "false"               | "false"  | option |

| 标签    | 标签解释                        | 属性: 值类型                                              | 属性解释                                                               | 取值范围                                     | 默认值          | 是否必须   |
|-------|-----------------------------|------------------------------------------------------|--------------------------------------------------------------------|------------------------------------------|--------------|--------|
|       |                             |                                                      | "false": 该控件在 AOD 模式下不主动更新。                                        |                                          |              |        |
| label | 文本约束:<br>只支持一行, 不支持从右到左的文本。 | color: uint32_t (ARGB8888)                           | 字体颜色。                                                              | 32 位十六进制正整数                              | 0x00000000   | must   |
|       |                             | font: string                                         | 系统的字库名; 仅能指定一个, 最大长度限制为 64Byte。                                    | 字符串                                      | NA           | must   |
|       |                             | break_mode: adapt stretch wrap ellipsis marquee clip | 换行方式 (只考虑单行的)。<br>注意: adapt 模式不支持文本对齐。                             | adapt stretch wrap ellipsis marquee clip | ellipsis     | must   |
|       |                             | rect: int16_t int16_t int16_t int16_t                | 文本显示区域。入参方式为: 区域位置和区域宽高 (x y w h)。                                 | (-32768, 32767)                          | (0, 0, 0, 0) | must   |
|       |                             | text: string                                         | 文本, 可以含有通配符, 与绑定的数据相匹配。<br>注意: 不支持 "%%", 如果需显示 "%", 使用 "%s" 格式化输出。 | 字符串                                      | NA           | must   |
|       |                             | font_size: uint8_t                                   | 文本字体大小。                                                            | [0, 255]                                 | 0            | must   |
|       |                             | align: left   center   right                         | 文本对齐方式。                                                            | left   center   right                    | left         | must   |
|       |                             | bind_data: int16_t                                   | 绑定的数据类型, 与通配符配合使用。                                                 | [0, 32767)                               | 0            | option |
|       |                             | period: uint8_t                                      | 1s 内获取数据频率。<br>该属性为非必须传参的必要属性, 未使用时或者传                             | 0, 1, 10, 30, 60                         | 0            | option |



| 标签        | 标签解释 | 属性：值类型                     | 属性解释                                                                                                           | 取值范围                                | 默认值      | 是否必须   |
|-----------|------|----------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------|----------|--------|
|           |      |                            | 参与 0 时，被动获取数据频率。。                                                                                              |                                     |          |        |
|           |      | display_mode:string        | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是 normal 下显示。<br><br>"ambient": 该控件只在 AOD 模式下显示。<br><br>"always": 两种模式下都显示。 | "normal",<br>"ambient",<br>"always" | "normal" | option |
|           |      | ambient_periodic:bool      | "true": 该控件在 AOD 模式下主动更新，且更新频率由标签 dial 的 ambient_period 属性控制。<br><br>"false": 该控件在 AOD 模式下不主动更新。               | "true",<br>"false"                  | "false"  | option |
| arc_label | 弧形文本 | center: int16_t int16_t    | 中心点。                                                                                                           | (-32768, 32767)                     | (0, 0)   | must   |
|           |      | radius: uint16_t           | 半径。                                                                                                            | [0, 32767)                          | 0        | must   |
|           |      | start_angle: int16_t       | 起始角度。                                                                                                          | [0, 360]                            | 0        | must   |
|           |      | end_angle: int16_t         | 结束角度。                                                                                                          | [0, 360]                            | 0        | must   |
|           |      | color: uint32_t (ARGB8888) | 字体颜色。                                                                                                          | 32 位十六进制正整数                         | 0x000000 | must   |
|           |      | font: string               | 系统的字库名。                                                                                                        | 字符串                                 | NA       | must   |
|           |      | text: string               | 文本，可以含有通配符，与绑定的数据相匹配。                                                                                          | 字符串                                 | NA       | must   |

| 标签        | 标签解释 | 属性: 值<br>类型                  | 属性解释                                                                                                    | 取值范围                          | 默认值          | 是否必须   |
|-----------|------|------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------|--------------|--------|
|           |      |                              | 注意: 不支持 “%%” , 如果需显示 “%” , 使用 “%s” 格式化输出。                                                               |                               |              |        |
|           |      | font_size: uint8_t           | 文本大小。                                                                                                   | [0, 255]                      | 0            | must   |
|           |      | align: left   center   right | 文本对齐方式。                                                                                                 | left   center   right         | left         | must   |
|           |      | bind_data: int16_t           | 绑定的数据类型, 与通配符配合使用。                                                                                      | [0, 32767)                    | 0            | option |
|           |      | period: uint8_t              | 1s 内获取数据频率。<br>该属性为非必须传参的必要属性, 未使用时或者传参为 0 时, 被动获取数据频率。                                                 | 0, 1, 10, 30, 60              | 0            | option |
|           |      | display_mode:string          | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal", "ambient", "always" | "normal"     | option |
| box_progr | 条形进度 | ambient_periodic:bool        | "true": 该控件在 AOD 模式下主动更新, 且更新频率由标签 dial 的 ambient_period 属性控制。<br>"false": 该控件在 AOD 模式下不主动更新。           | "true", "false"               | "false"      | option |
|           |      | rect: int16_t int16_t        | 进度条的显示范围。入参方式为: 区域位置和区域                                                                                 | (- 32768,                     | (0, 0, 0, 0) | must   |

| 标签  | 标签解释 | 属性: 值<br>类型                                                                                          | 属性解释                                                                                                            | 取值范围                                                                                   | 默认值               | 是否必须   |
|-----|------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-------------------|--------|
| ess | 条    | int16_t<br>int16_t                                                                                   | 宽高(x y w h)。                                                                                                    | 32767)                                                                                 |                   |        |
|     |      | img_name:<br>path                                                                                    | 进度条的前景图片<br>(img_name   color  <br>(gradient_colors &<br>gradient_colors) 三选<br>一) 。                            | 字符串                                                                                    | NA                | option |
|     |      | color:<br>uint32_t<br>(ARGB88<br>88)                                                                 | 进度条的颜色<br>(img_name   color  <br>(gradient_colors &<br>gradient_colors) 三选<br>一) 。                              | 32 位十<br>六进制<br>正整数                                                                    | NA                | option |
|     |      | gradient_c<br>olors:<br>uint32_t<br>uint32_t ...                                                     | 线性渐变的过渡颜色, 与<br>gradient_stops 匹配使用<br>(img_name   color  <br>(gradient_colors &<br>gradient_colors) 三选<br>一) 。 | 32 位十<br>六进制<br>正整数                                                                    | NA                | option |
|     |      | gradient_s<br>tops: float<br>float...                                                                | 线性渐变的过渡点, 与<br>gradient_colors 匹配使<br>用。                                                                        | 0 ~ 1                                                                                  | NA                | option |
|     |      | cap_type:<br>none  <br>round                                                                         | cap 样式。                                                                                                         | none  <br>round                                                                        | none              | must   |
|     |      | direction:<br>left_to_rig<br>ht  <br>right_to_le<br>ft  <br>top_to_bot<br>tom  <br>bottom_to<br>_top | 进度条增长方向。                                                                                                        | left_to_<br>right  <br>right_to<br>_left  <br>top_to_<br>bottom  <br>bottom<br>_to_top | left_to<br>_right | must   |
|     |      | backgroun<br>d_color:ui<br>nt32_t                                                                    | 背景色。                                                                                                            | 32 位十<br>六进制                                                                           | 0x000<br>00000    | option |

| 标签           | 标签解释  | 属性: 值类型                               | 属性解释                                                                                                   | 取值范围                          | 默认值          | 是否必须   |
|--------------|-------|---------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------|--------------|--------|
|              |       |                                       |                                                                                                        | 正整数                           |              |        |
|              |       | bind_data: int16_t                    | 绑定的数据类型。                                                                                               | [0, 32767)                    | 0            | must   |
|              |       | period: uint8_t                       | 1s 内获取数据频率。<br>该属性为非必须传参的必要属性，未使用时或者传参为 0 时，被动获取数据频率。                                                  | 0, 1, 10, 30, 60              | 0            | option |
|              |       | display_mode:string                   | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal", "ambient", "always" | "normal"     | option |
|              |       | ambient_periodic:bool                 | "true": 该控件在 AOD 模式下主动更新，且更新频率由标签 dial 的 ambient_period 属性控制。<br>"false": 该控件在 AOD 模式下不主动更新。           | "true", "false"               | "false"      | option |
| arc_progress | 弧形进度条 | rect: int16_t int16_t int16_t int16_t | 进度条的显示范围。                                                                                              | (-32768, 32767)               | (0, 0, 0, 0) | must   |
|              |       | img_name: path                        | 进度条的前景图片 (img_name   color   (gradient_colors & gradient_colors) 三选一)。                                 | 字符串                           | NA           | option |

| 标签 | 标签解释 | 属性: 值类型                                | 属性解释                                                                                           | 取值范围             | 默认值        | 是否必须   |
|----|------|----------------------------------------|------------------------------------------------------------------------------------------------|------------------|------------|--------|
|    |      | color: uint32_t (ARGB8888)             | 进度条的颜色 (img_name   color   (gradient_colors & gradient_colors) 三选一)。                           | 32 位十六进制正整数      | NA         | option |
|    |      | gradient_colors: uint32_t uint32_t ... | 线性渐变的过渡颜色, 与 gradient_stops 匹配使用 (img_name   color   (gradient_colors & gradient_colors) 三选一)。 | 32 位十六进制正整数      | NA         | option |
|    |      | gradient_stops: float float...         | 线性渐变的过渡点, 与 gradient_colors 匹配使用。                                                              | 0 ~ 1            | NA         | option |
|    |      | cap_type: none   round                 | cap 样式。                                                                                        | none   round     | none       | must   |
|    |      | radius: uint16_t                       | 半径。                                                                                            | [0, 32767)       | 0          | must   |
|    |      | start_angle: int16_t                   | 起始角度。                                                                                          | [0, 360]         | NA         | must   |
|    |      | end_angle: int16_t                     | 结束角度。                                                                                          | [0, 360]         | NA         | must   |
|    |      | background_color: uint32_t             | 背景色 (默认无背景)。                                                                                   | 32 位十六进制正整数      | 0x00000000 | option |
|    |      | line_width: int16_t                    | 线宽。                                                                                            | [0, 32767)       | 0          | must   |
|    |      | bind_data: int16_t                     | 绑定的数据类型。                                                                                       | 整数               | 0          | must   |
|    |      | period: uint8_t                        | 1s 内获取数据频率。<br>该属性为非必须传参的必要属性, 未使用时或者传                                                         | 0, 1, 10, 30, 60 | 0          | option |

| 标签    | 标签解释 | 属性：值类型                                            | 属性解释                                                                                                           | 取值范围                                | 默认值          | 是否必须   |
|-------|------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------|--------------|--------|
|       |      |                                                   | 参与 0 时，被动获取数据频率。                                                                                               |                                     |              |        |
|       |      | display_mode:string                               | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有，默认是 normal 下显示。<br><br>"ambient": 该控件只在 AOD 模式下显示。<br><br>"always": 两种模式下都显示。 | "normal",<br>"ambient",<br>"always" | "normal"     | option |
|       |      | ambient_periodic:bool                             | "true": 该控件在 AOD 模式下主动更新，且更新频率由标签 dial 的 ambient_period 属性控制。<br><br>"false": 该控件在 AOD 模式下不主动更新。               | "true",<br>"false"                  | "false"      | option |
| video | 视频   | rect:<br>int16_t<br>int16_t<br>int16_t<br>int16_t | 视频控件的显示范围。入参方式为：区域位置和区域宽高(x y w h)。<br><br>注意：宽高必须与视频的帧宽高保持一致。                                                 | (-32768, 32767)                     | (0, 0, 0, 0) | must   |
|       |      | source:string                                     | 指定视频的路径。                                                                                                       | 字符串                                 | NA           | must   |
|       |      | color_key:<br>uint32_t                            | ARGB8888, 指定 ColorKey 的颜色。<br><br>ColorKey 会将图层背景改为当前指定颜色，并实现透视。alpha 分量必须是 FF。                                | 32 位十六进制正整数                         | 0x00000000   | must   |
|       |      | preview_fr                                        | 指定视频预览帧的路径。                                                                                                    | 字符串                                 | NA           | must   |

| 标签    | 标签解释   | 属性: 值类型                               | 属性解释                                                                                                                    | 取值范围                                | 默认值          | 是否必须   |
|-------|--------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-------------------------------------|--------------|--------|
|       |        | ame: string                           |                                                                                                                         |                                     |              |        |
|       |        | display_mode: string                  | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br><br>"ambient": 该控件只在 AOD 模式下显示。<br><br>"always": 两种模式下都显示。         | "normal",<br>"ambient",<br>"always" | "normal"     | option |
| click | 点击跳转配置 | rect: int16_t int16_t int16_t int16_t | 可点击区域。入参方式为: 区域位置和区域宽高 (x y w h)。                                                                                       | (-32768, 32767)                     | (0, 0, 0, 0) | must   |
|       |        | action_slice: uint32_t                | 跳转到 slice, 与 action_bundle 二选一。<br>( "application\wearable\nativeapp\nativelauncher\include\AppViewIDs.h" 查看 AppViewId) | 正整数                                 | NA           | option |
|       |        | action_bundle: bundle_name            | 跳转到 JS 页, 与 action_slice 二选一。                                                                                           | 字符串 (bundle_name 是 JS 的 ID 名字)。     | NA           | option |
|       |        | display_mode: string                  | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br><br>"ambient": 该控件只在 AOD 模式下显示。<br><br>"always": 两种模式下都            | "normal",<br>"ambient",<br>"always" | "normal"     | option |

| 标签           | 标签解释 | 属性: 值类型                      | 属性解释                                                                                                    | 取值范围                                | 默认值      | 是否必须   |
|--------------|------|------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------|----------|--------|
|              |      |                              | 显示。                                                                                                     |                                     |          |        |
| kaleidoscope | 万花筒  | img_name : path              | 万花筒图片。                                                                                                  | 字符串                                 | NA       | must   |
|              |      | crown_rotate_step : float    | 表冠旋转步进值。                                                                                                | 浮点型                                 | 1.0      | option |
|              |      | animator_rotate_step : float | 动画播放旋转步进值。                                                                                              | 浮点型                                 | 0.0      | option |
|              |      | display_mode:string          | "normal": 只在正常亮屏模式下导入该控件。如果该属性没有, 默认是 normal 下显示。<br>"ambient": 该控件只在 AOD 模式下显示。<br>"always": 两种模式下都显示。 | "normal",<br>"ambient",<br>"always" | "normal" | option |

#### 说明

- 默认值生效场景: must 属性传输错误时; option 属性未设置时。
- 各标签的 bind\_data 属性赋值, 参照 "application\wearable\nativeapp\nativeui\main\include\dial\DialDataBase.h" 文件中的枚举值。值的范围匹配不同的数据类型, 具体参照表 7-3。
- 功能实际主动刷新频率, 会按照 xml 内各 period 属性的最高频率刷新。
- 当前所有设置图片资源的属性, 请使用标准 png 或者 jpg 格式图片。支持图片格式: RGB565、RGB888、ARGB8888。
- 若标签为周期性的, 需要绑定 data type; 非周期性标签可以不绑定。

表7-3 bind\_data 范围定义

| 取值范围        | 匹配数据类型 | 对应接口                             |
|-------------|--------|----------------------------------|
| (0, 0x2fff) | 单浮点类型  | void HandleFloatData(float data) |



| 取值范围             | 匹配数据类型 | 对应接口                                                       |
|------------------|--------|------------------------------------------------------------|
| (0x2fff, 0x4fff) | 文本类型   | void HandleTextData(const std::string* data, uint16_t num) |
| (0x4fff, 0x7fff) | 多浮点类型  | void HandleFloatData(const float* data, uint16_t num)      |

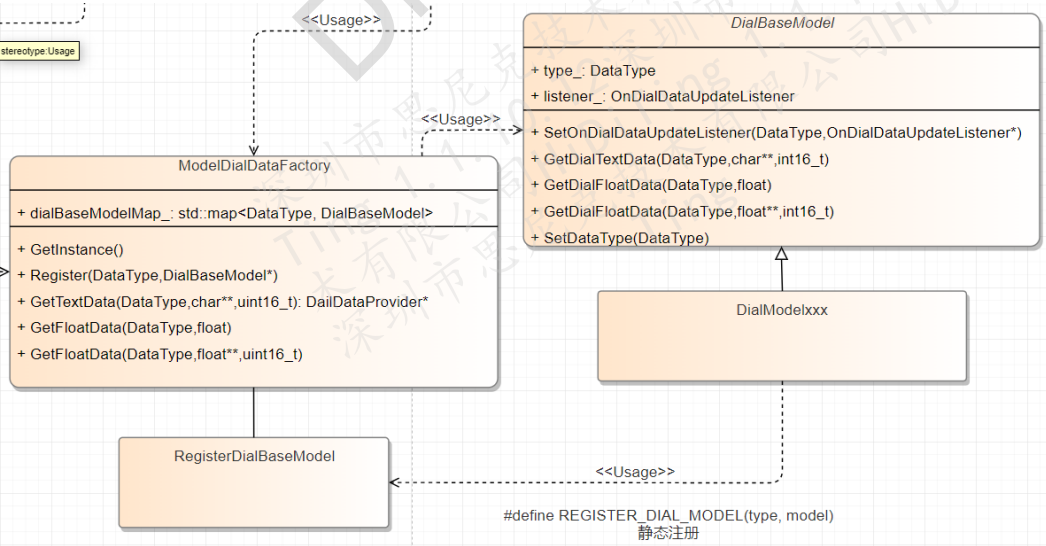
7.4 数据类型 model

离线表盘数据由 model 提供，分为主动获取与被动通知两种方式。当标签设置为周期类，绑定一个 data 后，周期主动获取数据；非周期类，绑定了 data 的，则注册回调，被动更新数据。

model 采用 factory 模式管理，方便扩展以及对上层解耦。当前支持了一部分数据类型，客户可以根据自身情况进行类型扩展。

model 设计如图 7-3 所示。

图7-3 model 设计



ModelDialDataFactory 向上提供查询及注册接口；DialBaseModel 为基类；DialModelxxx 继承 DialBaseModel，并通过 REGIST\_DIAL\_MODULE 静态往 factory 中注册，factory 中管理 model 链表。详细信息，可参考

“application\wearable\nativeapp\nativeui\main\src\ModelListenerSample.cpp” 代码实现。

当前数据类型分三类，float、multi float、text 三类，分别对应三个分段 DATA\_TYPE\_FLOAT\_BASE、DATA\_TYPE\_MULTI\_FLOAT\_BASE、DATA\_TYPE\_TEXT\_BASE。如需扩展，请按数据类型新增对应的枚举值。每个枚举值只能对应一个 model，而一个 model 可能对应多个枚举类型，比如 DialModelTime。

扩展新的 model 类型，需新增 datatype，继承 DialBaseModel 并重写需要的数据接口，然后通过 REGIST\_DIAL\_MODULE 进行注册即可。详细可参考现有的 dial model。

## 7.5 相关类介绍

### 7.5.1 DialViewGroup

DialViewGroup 继承于 CardPage，可直接作为 crossview 的一个页面使用；同时，它也继承于 OnDialDataUpdateListener，用于 model 数据更新时用于通知上层的回调。主要函数如表 7-4 所示。

表7-4 DialViewGroup 主要函数

| 函数                                     | 介绍                   |
|----------------------------------------|----------------------|
| void SetDial(std::string filePath)     | 设置表盘资源路径。            |
| uint8_t GetPeriod(void)                | 获取表盘刷新周期。            |
| void UpdateViewsByPeriodicUpdateData() | 通知表盘更新数据（周期性主动获取数据）。 |

### 7.5.2 DialBaseModel

DialBaseModel 为 model 基类，所有的 model 均继承于此类，主要函数如表 7-5 所示。

表7-5 DialBaseModel 主要函数

| 函数                                                                | 介绍                                    |
|-------------------------------------------------------------------|---------------------------------------|
| void RegisterDialDataListener(OnDialDataUpdateListener* listener) | 注册回调函数，用于被动数据上报方式。当 model 数据变化时，会调用注册 |

| 函数                                                                      | 介绍                                                                                                                                                     |
|-------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                         | 进来的 listener。                                                                                                                                          |
| bool UnRegisterDialDataListener()                                       | 注销回调函数。                                                                                                                                                |
| bool GetDialTextData(DialDataType& type, string*& out, int16_t& strNum) | 获取字符串数据。 <ul style="list-style-type: none"><li>• type: 数据类型。</li><li>• out: 数据指针, 用于带回字符串内容。</li><li>• strNum: 带回的字符串个数。</li></ul>                     |
| bool GetDialFloatData(DialDataType& type, float& out)                   | 获取单个 float 数据。 <ul style="list-style-type: none"><li>• type: 数据类型。</li><li>• out: 带回 float 数据的变量。</li></ul>                                            |
| bool GetDialFloatData(DialDataType& type, float*& out, int16_t len)     | 获取多个 float 数据。 <ul style="list-style-type: none"><li>• type: 数据类型。</li><li>• out: 数据指针, 用于带回 float 数据 buffer。</li><li>• len: 带回的 float 数据个数。</li></ul> |

### 7.5.3 ModelDialDataFactory

ModelDialDataFactory 用于管理 model 链表, 并对上提供数据查询及回调注册接口, 主要函数如表 7-6 所示。

表7-6 ModelDialDataFactory 主要函数

| 函数                                                                                   | 介绍                                             |
|--------------------------------------------------------------------------------------|------------------------------------------------|
| bool RegisterDialDataListener(DialDataType type, OnDialDataUpdateListener* listener) | 注册回调函数: 通过 type 查询到具体的 model, 并调用 model 相应的接口。 |
| bool UnRegisterDialDataListener(DialDataType type)                                   | 注销回调函数: 通过 type 查询到具体的 model, 并调用 model 相应的接口。 |
| bool GetDialTextData(DialDataType& type, string*& out, int16_t& strNum)              | 获取字符串数据: 通过 type 查询到具体                         |

| 函数                                                                   | 介绍                                                  |
|----------------------------------------------------------------------|-----------------------------------------------------|
|                                                                      | 的 model，并调用 model 相应的接口。                            |
| bool GetDialFloatData(DialDataType& type, float& out)                | 获取单个 float 数据：通过 type 查询到具体的 model，并调用 model 相应的接口。 |
| bool GetDialFloatData(DialDataType& type, float*& out, int16_t& len) | 获取多个 float 数据：通过 type 查询到具体的 model，并调用 model 相应的接口。 |

7.6 协议说明

图7-4 Bin 文件的数据说明



说明

- Dial[Type] 支持类：DialStaticImg、DialSequenceImg、DialRotateImg、DialOptionImg、DialDigitalImg、DialLabel、DialArcLabel、DialProgressType、DialBoxProgress、DialArcProgress、DialVideo、DialClick、DialKaleidoscope。
- 具体 DialHeader 和 DialView 结构体成员顺序请参考“application\wearable\nativeapp\nativeui\main\src\dial\DialBinTypes.h”文件。
- DialHeader 和 DialView 结构体成员说明请参考“7.3 表盘制作”章节的标签控件。

# 8 在线表盘

## 8.1 简介

## 8.2 参考实现

## 8.3 注册方案

## 8.1 简介

表盘是手表最为基础的界面，其基本功能是为用户提供时间信息，但随着智能手表的发展，表盘上嵌入了越来越多的元素，比如电量信息、步数、心跳等，方便用户快速获取更多关键信息，如图 [在线表盘样例](#) 所示。为了适应不同客户的定制需求，我们支持在线表盘和离线表盘两种类型：

- 离线表盘：通过加载和解析本地文件存储的表盘描述信息来生成表盘，用户只需要根据约定好的规则去制作表盘描述文件即可，不需要太关心代码实现，开发门槛低，但这也一定程度降低了灵活性。
- 在线表盘：通过直接编程来生成表盘，所有表盘的元素都直接通过代码描述，用户需要深度理解代码，掌握图形界面开发技能，开发门槛较高，但同时开发者可以更灵活的实现想要的效果。

本章主要介绍在线表盘的开发指导。

图8-1 在线表盘样例



## 8.2 参考实现

新增一个在线表盘，用户至少需要实现两个文件：

1. 表盘头文件，参考：  
application/wearable/nativeapp/nativeui/include/clock/MainClockView.h
2. 表盘源文件，参考：  
application/wearable/nativeapp/nativeui/clock/src/MainClockView.cpp

表盘实现关键点如下：

- 新增的表盘类应继承 `UICardPage`，`UICardPage` 是 `UIViewGroup` 的子类，提供容器功能，同时 `UICardPage` 被设计用于和 `UICrossView` 配合，比如在表盘支持上下滑动、左右滑动进入其它页面。
- 新增表盘需要实现构造函数，并在其中设置界面的位置起点、宽高。
- 新增表盘需要重载 `PreLoad` 接口，并在其中完成各种表盘元素的创建和初始化，并调用 `SetCoverable(true)`，以支持其它应用界面遮盖表盘界面。
- 在 `PreLoad` 接口中，如果是定制模拟表盘场景，可创建 `UISweepClock` 实例，其提供了扫描时钟相关的基本能力，同时它最终继承自 `UIViewGroup`，提供了容器功能，表盘上的所有 UI 元素都应该被添加到这个容器中，然后 `UISweepClock` 实例应该被添加到新增的表盘类。



- 注意：因为主表盘是常用界面，为了提升回到主表盘流畅度，请不要在 UnLoad 接口中销毁释放表盘资源，相关操作应该放在表盘的析构函数中。
- 除此之外，用户还需要修改  
“application/wearable/nativeapp/nativelauncher/include/UiConfig.h”，在 DialId 中增加新增表盘的枚举。

## 8.3 注册方案

为了将用户定制表盘的实例化与框架解耦，支持在线表盘静态注册方案，用户注册后，框架会根据运行模式和表盘类型，自适应创建对应表盘的实例。用户需要在新增的表盘源文件中，使用注册宏完成注册，调用形式为：REGIST\_WATCH\_DIAL(mode, id, U)，其中参数解释如下：

- mode：显示模式，定义在“UiConfig.h”中的 DialDisplayMode 中，公板支持主表盘（DIAL\_DISPLAY\_MODE\_MAIN）和 AOD 表盘（DIAL\_DISPLAY\_MODE\_AOD），用户根据实际情况选择入参。
- id：表盘 ID，定义在“UiConfig.h”的 DialId 中，需要将用户自定义的表盘 id 作为入参。
- U：表盘类模板，需要将自定义的表盘类作为入参。

注册一个 main 模式下的表盘，参考样例如下：

```
REGIST_WATCH_DIAL(DIAL_DISPLAY_MODE_MAIN, DIAL_CLOCK, MainClockView);
```

# 9 桌面样式

- 9.1 简介
- 9.2 参考实现
- 9.3 注册方案

## 9.1 简介

桌面样式指的是应用图标在桌面上的组织和呈现方式，最典型的桌面样式为列表风格(如图 [列表风格桌面](#))，通过上下滑动翻页，寻找想要的应用，再点击启动应用。除了最基本的列表模式外，我们还提供了多种个性化的桌面样式，包括足球风格、蜂窝风格和瀑布风格，随着版本迭代，后续可能会增加更丰富的桌面样式。



图9-1 列表风格桌面



## 9.2 参考实现

新增一个桌面样式，用户至少需要实现两个文件：

- 桌面样式头文件，参考：  
application/wearable/nativeapp/nativeui/include/applist/xxxDesktopFragment.h
- 桌面样式源文件，参考：  
application/wearable/nativeapp/nativeui/applist/src/xxxDesktopFragment.cpp

桌面样式实现关键点：

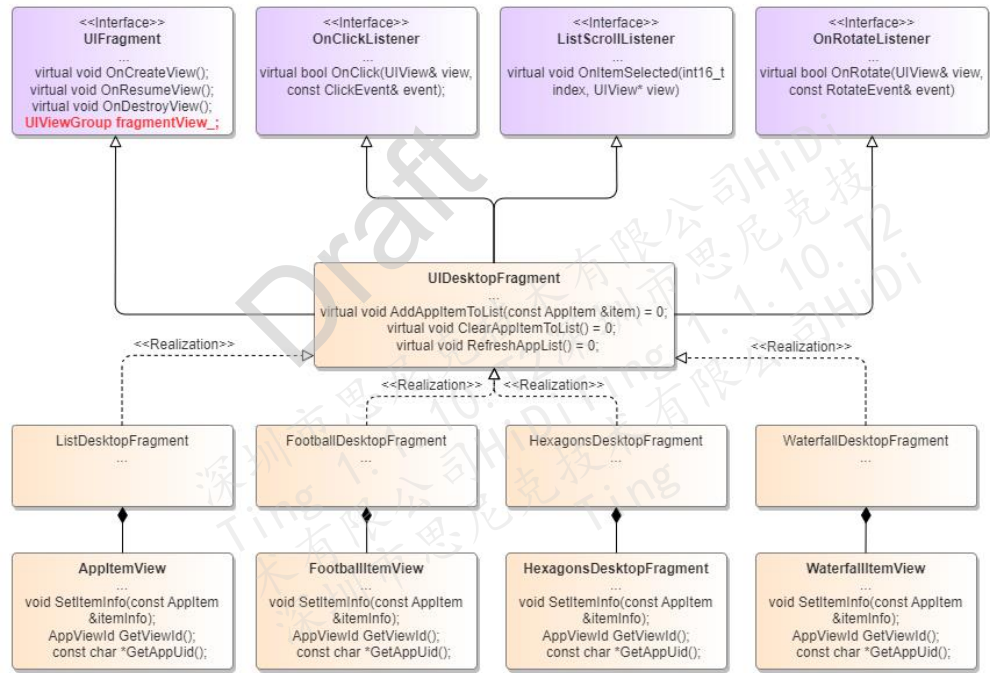
1. 新增的桌面样式类应继承并实现 `UIDesktopFragment` 接口（其定义如[图 UIDesktopFragment 定义](#)），`UIDesktopFragment` 通过继承多个父类，拥有自己的生命周期，支持处理点击、滚动和旋转事件，内部持有 `UIViewGroup` 实例，可提供容器功能；以上关键类的关系如[图 UIDesktopFragment 类图](#)所示。

图9-2 UIDesktopFragment 定义

```
class UIDesktopFragment : public UIFragment, public UIView::OnClickListener,
                        public ListScrollListener, public UIView::OnRotateListener {
public:
    virtual void AddAppItemToList(const AppItem &item) = 0;
    virtual void ClearAppItemToLis() = 0;
    virtual void RefreshAppList() = 0;
    void SetPosition(int16_t x, int16_t y, int16_t width, int16_t height);

protected:
    uint16_t imgSize_ = 0;
    uint16_t imgDistance_ = 0;
    void OnCreateView(void* data) override {};
    void OnDestroyView() override {};
    bool SwitchView(AppViewId viewId, char *uid, TransitionType type, DesktopStyle style);
};
```

图9-3 UIDesktopFragment 类图



UIDesktopFragment 关键接口说明如表 9-1 所示。

表9-1 UIDesktopFragment 关键接口说明

| 接口                | 接口描述                                       |
|-------------------|--------------------------------------------|
| AddApplItemToList | 将应用添加到桌面，并生成对应风格的图标，是一个纯虚函数，客户定制桌面必须实现此接口。 |

| 接口                 | 接口描述                                                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| ClearApplItemToLst | 清空桌面，并销毁桌面图标相关的资源，是一个纯虚函数，客户定制桌面必须实现此接口。                                                                                                  |
| RefreshAppList     | 刷新桌面图标布局，在更新桌面应用后被调用，是一个纯虚函数，客户定制桌面必须实现此接口。                                                                                               |
| OnCreateView       | 在创建桌面时被调用，开发者应该在此接口中创建、初始化对应风格的容器，并调用 fragmentView_.Add(xxx) 将容器添加到 UIFragment 持有的容器中（UIFragment 持有的容器已被添加到 RootContainer）；客户定制桌面必须重载此接口。 |
| OnDestroyView      | 在销毁桌面时被调用，开发者应该在这个接口中销毁所有资源，客户定制桌面必须重载此接口。                                                                                                |
| OnResumeView       | 这个接口没有在 UIDesktopFragment 中显示声明，它是继承自 UIFragment 的接口，用于处理桌面显示前的定制动作，客户可按需实现。                                                              |
| SwitchView         | 用户点击桌面图标时，切换到对应的应用界面，客户定制桌面的 OnClick 回调中，建议统一调用此接口。                                                                                       |
| OnClick            | 响应用户点击事件，进入到对应的应用界面，内部调用 SwitchView 接口完成切换即可，客户定制桌面必须重载此接口；参考表 <a href="#">OnClickListener 回调函数</a> 。                                     |
| OnRotate           | 响应用户旋转桌面的事件，重新绘制桌面，客户可按需实现；参考表 <a href="#">OnRotateListener 回调函数</a> 。                                                                    |

2. 新增桌面样式应继承 UIImageView 并实现对应风格的 xxxItemView 类，用于管理桌面图标，可以将相关代码和主类放在一个文件中；这部分实现较为简单，不做赘述，客户可参考上述 xxxDesktopFragment.cpp(.h)中的 FootballItemView、HexagonsItemView、WaterfallItemView 等实现。

新增实现以上类之外，用户还需要以下操作：

步骤 1 修改

“application/wearable/nativeapp/nativeui/include/settings/model/SettingDesktopModel.h”，在 DesktopStyle 中增加新增桌面的枚举类型。

步骤 2 在 setting 模块中，增加对应桌面选择样式的入口。

----结束

## 9.3 注册方案

为了将客户定制桌面样式的实例化与框架解耦，支持桌面样式静态注册方案，用户注册后，框架会根据设置的风格自适应创建对应桌面的实例。用户需要在新增的桌面样式源文件中，使用注册宏完成注册，调用形式为：REGIST\_DESKTOP\_STYLE(style, D)，其中参数解释如下：

- style：桌面样式风格，定义在“SettingDesktopModel.h”的 DesktopStyle 中，需要将用户自定义的桌面 style 作为入参。
- D：桌面风格类模板，需要将自定义的桌面类作为入参。

注册一个新的桌面样式，参考样例如下：

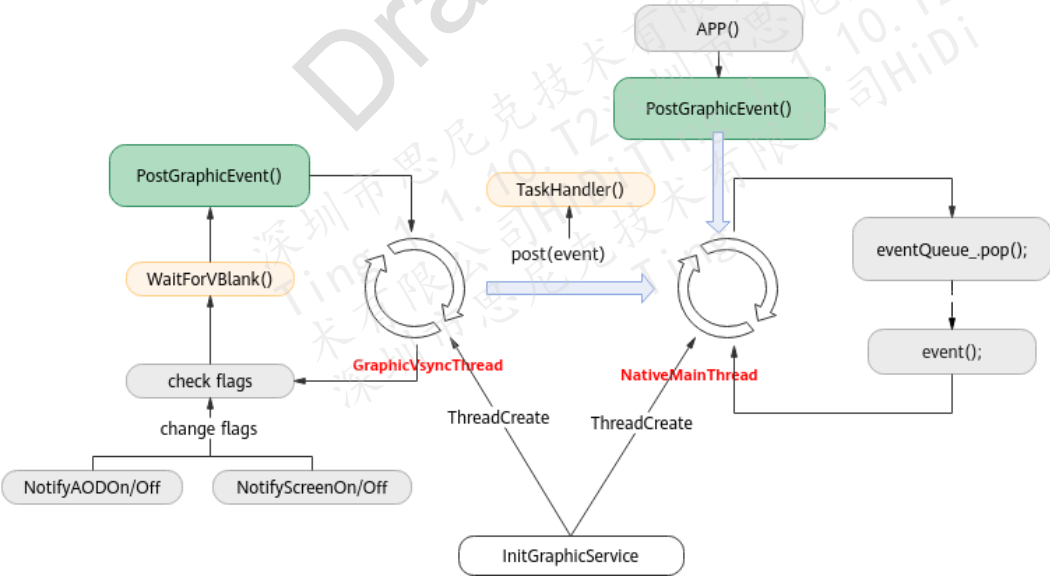
```
REGIST_DESKTOP_STYLE(APPLIST_STYLE, ListDesktopFragment);
```

# 10 Graphic service

- 10.1 整体方案
- 10.2 对外接口

## 10.1 整体方案

图10-1 Graphic service 总体方案



Graphic service 总体上分为两部分：

- 初始化 graphic。

- 创建两个任务线程，分别完成主任务执行，及 vblank（显示器扫描线完成最后一行后，需要重返左上角的过程，也叫 VBI--Vertical Blank Interval）、AOD（Always On Display，常亮显示）、Screen 等硬件信息监测。

通过 InitGraphicService()调用，完成对 gralloc、display HAL、input、TaskManager 等的初始化，准备 graphic 执行环境。同时会启动两个线程。

- main task：只做一件事，执行 event queue 中的任务，若队列空则睡眠等待信号量。
- vsync thread：监测 screen、AOD 信息，若是灭屏或低功耗模式，图形将睡眠等待信号量；否则获取 vblank 信号，并以此为周期将 TaskHandler 推送到 main task 执行。

所有外部模块对图形的调用，均需通过 PostGraphicEvent()接口推送到 event queue，等待 vblank 周期执行。

## 10.2 对外接口

表10-1 graphic service 对外 API

| 函数                                                           | 说明                                                                                                 |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| void<br>PostGraphicEvent(const<br>OHOS::GraphicEvent& event) | 向图形 service 里面推送任务，任务将在后续的 Vsync 周期被执行。<br><br>注意：仅推送涉及 UI 属性变更或刷新的任务，切勿推送无关且耗时的处理，这会阻塞 UI，导致显示卡顿。 |
| void<br>NotifyScreenOn()                                     | 设置 screen flag 为 1（on）。                                                                            |
| void<br>NotifyScreenOff()                                    | 设置 screen flag 为 0（off）。                                                                           |

### 说明

若是 C 语言代码，通过 graphic\_service\_wrapper 中对应的接口调用。

# 11 表情包支持

## 11.1 方案介绍

## 11.2 制作表情包数据库和表情包映射表

## 11.3 绘制原理

### 11.1 方案介绍

不同于文字使用矢量字库绘制，表情包使用图片进行绘制，支持自定义表情包，表情包自定义流程如下：

- 步骤 1 制作表情包压缩数据（请参考“[6.4 图片资源打包](#)”）。
- 步骤 2 在头文件 `ui_emoji_table.h` 中定义 unicode 码点和表情包图片的映射关系。
- 步骤 3 导入表情包压缩数据并使用。

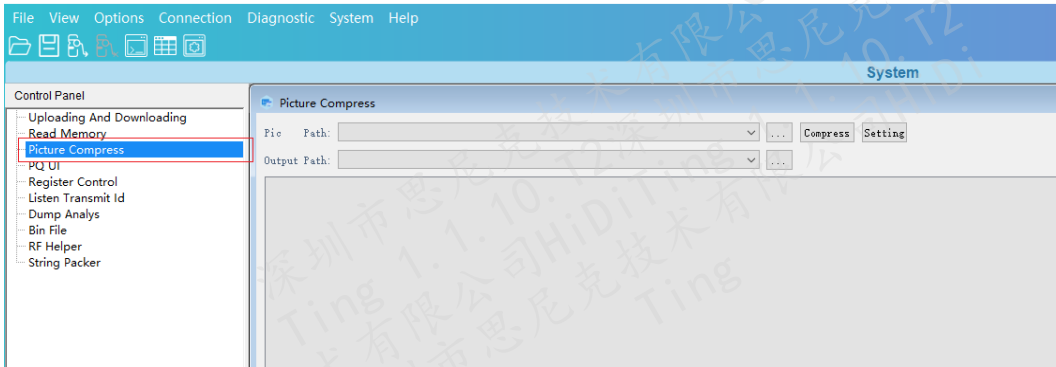
----结束

### 11.2 制作表情包数据库和表情包映射表

制作表情包数据库和表情包映射表流程如下：

- 步骤 1 将使用的表情包用 Debugkits 以 package 的方式进行压缩，并将名字保存为 `emoji.bin`。





步骤 2 将表情包与码点的映射关系放到头文件  
“middleware/services/gui/uikit/proprietary/include/common/ui\_emoji\_table.h” 中去，  
同时通过变量 “EMOJI\_PATH” 配置 emoji.bin 文件的位置。



步骤 3 将 emoji.bin 放入配置的路径中去。

----结束

### 11.3 绘制原理

普通的码点（unicode）首先通过查找字库，获取矢量数据进行绘制。对于找不到的码点会先在自定义的表情包数据库中寻找，如果找到，会提取表情包的图片数据，并进行绘制；如果在表情包映射表中也找不到，将会在配置的多语言字库中查找码点数据。



# 12 DFX

“DFX” 通常指的是 Design for eXcellence，其中 X 也可以看作是通配符。它强调 DFT（设计便于测试）、DFD（设计便于调试）、DFR（设计可靠性）和 DFP（设计功耗）等，以提升整体设计的卓越性，满足应用领域和制程技术需求。

DFX 能力体系涵盖了产品全生命周期的 14 个关键维度：可靠性、节能减排、归一化、可服务性、可安装性、可制造性、可维修性、可采购性、可供应性、可测试性、可修改性/可扩展性、成本、性能和安全性。这些质量属性共同构成了产品在市场中的差异化竞争力。

## 12.1 内存占用

### 12.2 帧率

### 12.3 截图

### 12.4 模拟事件

### 12.5 时延统计

### 12.6 控件树打印

### 12.7 内存泄漏定位

### 12.8 Native 状态打印

## 12.1 内存占用

- dumpmem

打印 UIKit 占用内存（仅 UIKit 内部的 UIMalloc 和 new 分配的内存被记录在内。其中，图片、文字、绘制 Buffer 等使用的预分配内存不计入该内存占用中）。

注意，如果当前 dumpmem 数值在慢慢变大，一般说明出现内存泄漏，请参考“12.7 内存泄漏定位”；如果当前 dumpmem 数值在慢慢减小，说明出现内存重复释放，需要自行定位。

- dumpimg

打印 UIKit 中图片资源缓存信息，以及相关控件所持有的图片资源信息。

#### 📖 说明

在串口工具中发送指令，指令格式为：AT^UIKIT\_DFX=具体命令，后续不再赘述。

## 12.2 帧率

- printfps

- -h: 打印帧率说明。

- duration n: n 秒，持续打印帧率 n 秒[n 次]。

仅输入 printfps，则只打印一次帧率。

- showfps

- -h: 显示帧率说明。

- -p: 指定显示起始位置。参数格式：x, y。

仅输入 showfps，则默认显示在屏幕中间。

其坐标位置仅在第一次输入 showfps 时有效，后续再调用均无效。

- hidefps

隐藏帧率显示。

帧率 DFX 示例如下：

```
/* 打印帧率 */
AT^UIKIT_DFX=printfps 10
/* 显示帧率 */
AT^UIKIT_DFX=showfps -p 100,100
/* 隐藏帧率 */
AT^UIKIT_DFX=hidefps
```

## 12.3 截图

- dump\_view\_id

串口输入此命令，将当前 RootView 下所有设置过 viewId 的子节点的 viewId 和坐标信息输出在串口中：[viewId]: x, y, w, h。

- screencap
  - -h: 截图指南，无参。
  - -v: 控件截图，截取指定控件的数据。参数格式：viewId。
  - -c: 区域截图，截取指定区域的数据。参数格式：x, y, w, h。参数非负，且不能超过屏幕大小[454, 454]。
  - -f: 输出文件，截取数据保存至该输出文件。

#### 📖 说明

- -f 需配置为最后一个选项。如果-f 前无其他选项，则判定为全屏截图。
- 文件保存的数据格式为 RGB888。
- 文件像素宽度为 16 倍数对齐，即真实宽度为：

$$\left\lceil \frac{w}{16} \right\rceil * 16$$

- 串口回显：  
screencap result: 1. (表示执行成功)。  
screencap result: 0. (表示执行失败)。

截图 DFX 示例如下：

```
/* 区域截图 */
AT^UIKIT_DFX=screencap -c 0,0,100,100 -f /bin/vs/sd0p0/rect.bin

/* 组件截图 */
dump_view_id
AT^UIKIT_DFX=screencap -v mainclickactivity -f /bin/vs/sd0p0/view.bin

/* 全屏截图 */
AT^UIKIT_DFX=screencap -f /bin/vs/sd0p0/screen.bin
```

## 12.4 模拟事件

### 模拟旋转表冠事件

- inject\_rotate n
  - 参数 n: 旋转刻度 (有符号整形类型)。
  - 示例：

AT^UIKIT\_DFX=inject\_rotate 10

## 模拟按键事件

- inject\_key keyId state
  - 参数 keyId: 键值 (字符串类型), 合法值[power, func]。
  - 参数 state: 按键状态 (无符号整形类型), 合法值[0, 1, 2]。0 代表松开, 1 代表按下, 2 代表长按。
  - 示例:  
AT^UIKIT\_DFX=inject\_key power 1

## 模拟点击事件

- inject\_click x y
  - 参数 x: X 坐标 (无符号整形类型)。
  - 参数 y: Y 坐标 (无符号整形类型)。
  - 示例:  
AT^UIKIT\_DFX=inject\_click 100 200

## 模拟拖动事件

- inject\_drag startX startY endX endY duration
  - 参数 startX: 起始 X 坐标 (无符号整形类型)。
  - 参数 startY: 起始 Y 坐标 (无符号整形类型)。
  - 参数 endX: 结束 X 坐标 (无符号整形类型)。
  - 参数 endY: 结束 Y 坐标 (无符号整形类型)。
  - 参数 duration: 拖动时长 ms (无符号整形类型)。
  - 示例:  
AT^UIKIT\_DFX=inject\_drag 200 400 200 100 30

## 模拟长按事件

- inject\_lp x y
  - 参数 x: X 坐标 (无符号整形类型)。
  - 参数 y: Y 坐标 (无符号整形类型)。
  - 示例:  
AT^UIKIT\_DFX=inject\_lp 100 30

## 12.5 时延统计

- showtime param  
参数 param：要统计时延的类型，可取值 point、tp、render、close。
  - point：打印 UI 一秒内尝试查询 tp 点的次数及相关信息。
  - tp：实时打印 tp 报点。
  - render：打印绘制耗时。
  - close：关闭时延统计。
  - 示例：  
AT^UIKIT\_DFX=showtime render
- showtime point
- showtime tp
- showtime render count flush
  - 含义：渲染过程中每一帧软硬件耗时、task 耗时和 event 耗时。
  - 示例
    - 示例 1：  
AT^UIKIT\_DFX=showtime render 30 0  
30：打印接下来 30 帧软硬件绘制指令。  
0：第一帧不强制刷新。
    - 示例 2：  
AT^UIKIT\_DFX=showtime render 30 1  
30：打印接下来 30 帧软硬件绘制指令。  
1：第一帧强制刷新。

打印结果（截取部分）：

```
E 72: 33038
E 73: 14145
T 0: 61
T 1: 93
T 2: 61
T 3: 13841
isNativeRunning 1, freq 60
Cmd count: 28 time: 12856, {0 0 0 0}
0. hw_rect: 0, 231, {0, 0, 453, 453}
1. hw_blit: 521, 306
2. hw_path: 2613, 1537, 33
3. hw_submit: 49, 1969, 1
```

```
4. sw_rect: 15, 107, {47, 210, 406, 211}
5. hw_arc_rect: 51, 263
6. hw_submit: 49, 239, 1
7. sw_rect: 66, 202, {47, 302, 76, 331}
8. hw_arc_rect: 60, 119
9. hw_submit: 39, 249, 1
10. sw_rect: 45, 178, {102, 302, 131, 331}
11. hw_arc_rect: 56, 118
12. hw_submit: 38, 240, 1
13. sw_rect: 47, 176, {157, 302, 186, 331}
14. hw_arc_rect: 57, 107
15. hw_submit: 33, 239, 1
16. sw_rect: 46, 176, {212, 302, 241, 331}
17. hw_arc_rect: 61, 109
18. hw_submit: 34, 234, 1
19. sw_rect: 45, 200, {267, 302, 296, 331}
20. hw_arc_rect: 56, 108
21. hw_submit: 35, 488, 1
22. sw_rect: 50, 175, {322, 302, 351, 331}
23. hw_arc_rect: 57, 125
24. hw_submit: 39, 235, 1
25. sw_rect: 47, 175, {377, 302, 406, 331}
26. hw_submit: 59, 173, 1
27. sw_rect: 12, 98, {47, 300, 406, 301}
```

格式 1: E xxx: xxx, 表示事件消耗时间, 例如 E 73: 14145, 表示 73 号事件消耗时间是 14145 微秒, 每个事件可能包含一系列任务, 也可能不包含。此处 73 号事件接下来的几行都是属于事件 73 的具体时间消耗。

格式 2: T xxx: xxx, 表示任务消耗时间, 例如 T 0: 61, 表示 0 号任务消耗时间是 61 微秒, 每个任务可能包含帧打印时间消耗, 也可能不包含, 紧跟着任务打印并且缩进一格的帧时间消耗打印从属于这一任务。

格式 3: 帧格式, 从 isNativeRunning 1, freq 60 开始。

第一行解释: 是否 native、推荐帧率。

第二行解释: 表示有 28 条指令, 耗时总共 12856 微秒, 绘制矩形范围{左、上、右、下}。

第三行以及之后打印格式: 指令序号、简短描述、与上一条指令相隔时间(指令间隔时间)、该条指令用时以及必要的额外信息。

特殊指令额外信息:

- 矢量绘制 hw\_path: 矢量绘制 hw\_path 在一帧的执行个数可能非常多, 因此统计时会将所有矢量绘制指令汇总到一起, 例如 “2. hw\_path: 236, 464, 7” 的额外信息 “7” 表示有 7 矢量绘制指令。
- 绘制提交 hw\_submit: 本例 “21. hw\_submit: 35, 488, 1”, 0 表示异步提交, 1 表示同步提交。

## 12.6 控件树打印

- dump\_root\_view

以 json 格式打印当前整个控件树, 打印信息会输出到屏幕上, 同时会将信息保存到 “/user/res/view\_tree.json”。

- 示例:

```
AT^UIKIT_DFX=dump_root_view
```

## 12.7 内存泄漏定位

- leak\_panic cnt size

该条指令需要结合代码分析, 在存在内存泄漏的代码区间 (例如某个类的构造和析构之间) 两端分别调用如下代码:

```
MemCheck::GetInstance()->EnableLeakCheck(true); // 检测开始  
  
// some code  
  
MemCheck::GetInstance()->EnableLeakCheck(false); // 检测结束
```

结果会输出这个区间内没有释放的内存, 包含信息为 “申请次序+大小”, 多次运行这段代码, 内存泄漏会在某次固定存在, 在下次运行这段代码前发出指令

“AT^UIKIT\_DFX=leak\_panic cnt size”。其中 cnt 表示第几次, size 表示大小, 该指令会在第 cnt 次申请内存时主动崩溃, 并输出调用栈信息, 通过调用栈可以定位内存泄漏位置。

- 示例:

```
AT^UIKIT_DFX=leak_panic 16 24
```

## 12.8 Native 状态打印

- dumpstate

打印 Native UI 的运行状态。打印的信息包含：

- NativeRunning：应用运行状态，用于区分 Native 和 JS 的状态。1 代表 Native 正在运行中，0 代表 JS 正在运行中。
- IsScreenOn：屏幕量灭屏状态。1 代表处于亮屏状态，0 代表处于灭屏状态。
- GraphicTaskTriggeredCount：已出发的 Native 任务个数。

Draft



# 13

## 支持标准解码 JPEG 和 PNG

如果需要开启标准解码功能，按照如下操作进行：

- 步骤 1 打开 “middleware\services\gui\hal\common\include\graphic\_hardware\_config.h”  
(图像硬件参数配置文件)，将该目录下的两个 CAPABILITY\_HW\_JPEG\_DECODE  
参数都设置为 0。
- 步骤 2 打开 “middleware\services\gui\ui\utils\interfaces\innerkits\graphic\_config.h” (图  
像参数配置文件)，将该目录下的所有的 ENABLE\_JPEG\_AND\_PNG 和  
CACHE\_JPEG\_AND\_PNG 参数设置为 1。
- 步骤 3 打开 “build\config\target\_config\brandy\config.py” (编译选项配置文件)，在  
brandy-native-js 的编译选项 defines 中加入 XTS\_SUPPORT、ram\_component 中加  
入 xts\_set。

----结束